

ANDRÉ GUY BRUNEAU

**Définition d'un cadre d'intégration d'application
dans le contexte d'une base de métadonnées**

**Mémoire
présenté
à la Faculté des études supérieures
de l'Université Laval
pour l'obtention
du grade de maître ès sciences (M.Sc.)**

**Département d'informatique
FACULTÉ DES SCIENCES ET DE GÉNIE
UNIVERSITÉ LAVAL**

Novembre 1997

© André Guy Bruneau, 1997.



National Library
of Canada

Acquisitions and
Bibliographic Services

395 Wellington Street
Ottawa ON K1A 0N4
Canada

Bibliothèque nationale
du Canada

Acquisitions et
services bibliographiques

395, rue Wellington
Ottawa ON K1A 0N4
Canada

Your file Votre référence

Our file Notre référence

The author has granted a non-exclusive licence allowing the National Library of Canada to reproduce, loan, distribute or sell copies of this thesis in microform, paper or electronic formats.

The author retains ownership of the copyright in this thesis. Neither the thesis nor substantial extracts from it may be printed or otherwise reproduced without the author's permission.

L'auteur a accordé une licence non exclusive permettant à la Bibliothèque nationale du Canada de reproduire, prêter, distribuer ou vendre des copies de cette thèse sous la forme de microfiche/film, de reproduction sur papier ou sur format électronique.

L'auteur conserve la propriété du droit d'auteur qui protège cette thèse. Ni la thèse ni des extraits substantiels de celle-ci ne doivent être imprimés ou autrement reproduits sans son autorisation.

0-612-25518-2

Canada

Résumé

Les entreprises font actuellement face à un problème d'intégration de leurs ressources informationnelles. Cette intégration est caractérisée par la présence de systèmes opérationnels évoluant dans des environnements hétérogènes et distribués. L'approche retenue pour résoudre ce problème est l'utilisation d'une base de métadonnées qui permet de conserver (1) les modèles décrivant les diverses bases de données, (2) les règles de distribution de l'information entre ces diverses bases de données et (3) les règles de conversion de l'information entre les différents systèmes, fournissant ainsi un modèle consolidé de l'entreprise.

Le schéma de la base de métadonnées (*Global Information Ressource Dictionary*; GIRD ; [Bouziane91, Hsu et al.91, Hsu96]) regroupe toutes les métadonnées décrivant les différentes applications, permettant ainsi le partage des connaissances entre celles-ci. La structure de la base de métadonnées facilite la résolution de requêtes globales par le biais d'un système approprié (*Global Query System*; GQS ; [Cheung91, Cheung et Hsu96]). De plus, grâce à l'implantation d'une architecture concurrente adaptable (*Rule-Oriented Programming Environment*; ROPE ; [Babin93, Babin et Hsu96]), le partage des connaissances distribuées de l'entreprise se fait de façon transparente. La structure d'intégration est telle que les applications devant être intégrées ne nécessitent aucune modification ou adaptation. Cependant, l'intégration d'une application dans une base de métadonnées est relativement complexe. Il est nécessaire de définir les métadonnées décrivant cette application et ensuite de les intégrer dans le contenu de la base de métadonnées.

Ce projet de recherche a pour but d'étudier en détail les techniques déjà utilisées, et d'identifier les informations minimales requises, pour l'intégration d'une application dans le modèle consolidé de l'entreprise, tout en maintenant l'intégrité et la cohérence du contenu de la base de métadonnées. De plus, ce travail élabore les spécifications fonctionnelles pour effectuer cette intégration par le biais du logiciel IBMS (*Information Based Modeling System*; [Hsu et al.92a])

Dédicaces

À ma tendre épouse Renate et à David.

À ma mère Yolande.

À mon père Louis et son épouse Lise.

À mon grand-père René.

À ma belle-mère Hedwig.

À mes copains de travail Zakaria, Radhouan, Sylvain et Sofienne.

Avant-Propos

Je fais part de toute ma gratitude à M. Gilbert Babin, mon directeur de recherche, pour m'avoir soutenu et aidé durant la réalisation de ce mémoire. Je suis profondément reconnaissant pour les encouragements et les conseils qu'il m'a prodigués. Je tiens aussi à le remercier pour ses qualités humaines.

Je remercie M. André Gamache et M. Nadir Belkhiter, qui m'ont fait l'honneur de lire et juger ce travail de recherche, et de fournir leurs commentaires.

J'exprime mes remerciements au corps enseignant, ainsi qu'au personnel du département d'informatique.

J'ai consacré à la réalisation de ce mémoire bon nombre de soirées et de fins de semaine normalement passées en famille. Je remercie chaleureusement mon épouse Renate et David pour leur compréhension affectueuse, leurs encouragements et leur soutien tout au long de ce projet.

J'aimerais témoigner tout spécialement ma reconnaissance envers ma mère Yolande, mon père Louis et son épouse Lise qui m'ont accordé toute leur confiance et ont su m'encourager tout au long de mes études.

Enfin, je remercie mes confrères d'étude pour bon nombre de discussions fructueuses et pour leur participation à la création d'une ambiance de travail sympathique.

Table des Matières

Résumé	i
Dédicaces	ii
Avant-Propos	iii
Table des Matières	iv
Liste des Figures	viii
Glossaire	ix
1 Introduction	1
1.1 Architecture de la base de métadonnées	4
1.2 Problématique	7
1.3 Motivations	8
1.4 Objectifs	8
1.5 Organisation du mémoire	9
2 Différentes approches d'intégration	10
2.1 Bases de données fédérées	10
2.1.1 Discussion sur les multibases de données	12
2.2 CARNOT	12
2.2.1 Dimension de l'intégration sémantique	14
2.2.2 Métamodélisation	15
2.2.3 Transaction entre la vue globale et les vues locales	16
2.2.4 Développement des axiomes	16
2.2.5 Discussion sur Carnot	17
2.3 RODIN	18
2.3.1 Disco	19
2.3.2 Discussion sur Rodin	21
2.4 TSIMMIS	22
2.4.1 Traducteur et modèle commun	22
2.4.2 Médiateur	23
2.4.3 Les interfaces utilisateurs	24
2.4.4 Gestion de contraintes	24

2.4.5	Classification et extraction	24
2.4.6	Discussion sur TSIMMIS	25
2.5	CORBA	25
2.5.1	Concepts de la distribution : objet ou composante	25
2.5.2	Architecture	27
2.5.3	CORBA et les métadonnées	28
2.5.4	CORBA et les services offerts	30
2.5.5	Discussion sur CORBA	31
3	Méthode de modélisation TSER	33
3.1	Métamodélisation	33
3.2	Modèle fonctionnel	35
3.3	Modèle structurel	36
3.4	Algorithmes de transformation	40
3.5	IBMS	40
4	Schéma de l'entreprise	42
4.1	Principes de base du GIRD	42
4.2	Niveau fonctionnel du GIRD (GIRD/SER)	45
4.3	Niveau structurel du GIRD (GIRD/OER)	46
4.3.1	Sous-modèle Vue-Application	47
4.3.2	Sous-modèle Vue-Fonctionnelle	47
4.3.3	Sous-modèle Vue-Structurelle	49
4.3.4	Sous-modèle Vue-Ressource	50
4.3.5	Consolidation des quatre sous-modèles	51
5	Système de requêtes globales	54
5.1	Définition d'un système de requête générique	54
5.1.1	Interface usager pour la formulation de requêtes	56
5.1.2	Objectifs du système de requêtes globales	57
5.1.3	Exigences de base pour une requête globale	58
5.2	Traitement d'une requête	59
5.2.1	Algorithme de résolution d'une requête globale	60
5.2.2	Modèle minimal pour une base de connaissances en direct	66
5.3	Utilisation de MDB pour l'implantation d'une assistance en direct	67
5.3.1	Formulation de la requête	67
5.3.2	Conversion à base de règles	71
5.3.3	Langage de requête globale	73
6	Architecture concurrente adaptable	74
6.1	Architecture concurrente pour l'intégration adaptable	74
6.2	ROPE : méthode de gestion informationnelle dans l'entreprise	77
6.2.1	Modèle ROPE	78
6.2.2	Structure statique de ROPE	82
6.2.3	Structure dynamique de ROPE	84

6.3	Création d'un nouveau shell	86
7	Cadre d'intégration	88
7.1	Architecture d'intégration locale et globale	88
7.2	Modèle autonomie-coopération	90
7.3	Cellule d'intégration locale autonome	91
7.4	Fonctions des métaentités et métarelations	92
7.4.1	Vue Application	94
7.4.2	Vue Fonctionnelle	94
7.4.3	Vue Structurelle	97
7.4.4	Vue Ressources	99
7.5	Informations minimales	102
7.5.1	MétaEntités	102
7.5.2	MétaRelation	104
7.6	Modifications fonctionnelles de IBMS	105
7.6.1	Nouvelle version de IBMS	107
7.6.2	Synthèse	108
8	Conclusion	109
	Bibliographie	113
	Annexe A	122
	Annexe B	126

Liste des Figures

1.1	L'architecture de la base de métadonnées	5
1.2	Processus d'intégration des modèles dans la base de métadonnées	8
2.1	Architecture des schémas d'une base de données fédérée	11
2.2	Architecture de Carnot	13
2.3	Cyc inter-reliant des formalismes de modélisation populaire	15
2.4	Base de connaissances Cyc	16
2.5	Architecture de TSIMMIS	22
2.6	Évolution des composantes et les limites de l'infrastructure	27
2.7	Concepts de l'approche électronique	27
2.8	Interopérabilité de composantes spécifiées en IDL	28
2.9	Architecture de CORBA	29
3.1	Comparaison entre TSER et les méthodes traditionnelles	35
3.2	Concepts du modèle fonctionnel de TSER	35
3.3	Concepts du modèle structurel de TSER	36
3.4	Structure d'un métamodèle	39
4.1	Les couches logiques du concept GIRD	44
4.2	Le modèle fonctionnel GIRD (GIRD/SER)	45
4.3	Le sous-modèle Vue-Application	47
4.4	Le sous-modèle Vue-Fonctionnelle	48
4.5	Le sous-modèle Vue-Structurelle	50
4.6	Le sous-modèle Vue-Ressource	51
4.7	Le modèle structurel du GIRD (GIRD/OER)	53
5.1	Les deux méthodes de navigation du modèle	69
5.2	Table des équivalences	72
5.3	Conversion et dérivation des données selon l'approche à base de règles .	72
6.1	L'architecture concurrente utilisant la base de métadonnées	76
6.2	Le traitement des requêtes globales	79
6.3	Le traitement automatique de la connaissance de l'entreprise	80
6.4	Le processus de mise à jour des règles	80
6.5	Le modèle fonctionnel du <i>shell</i> de ROPE	81
6.6	Le modèle structurel du <i>shell</i> de ROPE	82
6.7	La structure statique d'un <i>shell</i> de ROPE	83

7.1	L'architecture d'intégration globale	89
7.2	Approche d'intégration autonomie-coopération	90
7.3	Couches de métadonnées	93
7.4	Représentation détaillée de la vue ressource	101
7.5	Modèle structurel minimal du GIRD	102
7.6	Phases d'intégration d'une nouvelle application	106

Glossaire

Définition des termes

Terme	Description
ANSI	Institut de normalisation national américain (<i>American National Standard Institute</i>).
ASCII	Code américain normalisé pour l'échange d'information (<i>American Standard Code for Information Interchange</i>).
BD	Base de données (<i>database</i>). Une base de données est un ensemble structuré de données. C'est en quelque sorte la représentation organisée et codée d'une réalité en évolution dans le temps. Son architecture peut être centralisée ou répartie.
CASE	Génie logiciel assisté par ordinateur (<i>Computer-Aided Software Engineering</i>). Consiste à utiliser les technologies informatiques pour améliorer la productivité au niveau du développement de systèmes.
Contexte	Concept de modélisation représentant l'information dynamique du système telles que les règles opérationnelles, les règles de contrôle et les règles behavioristes.
CORBA	Architecture commune pour courtier de requêtes objects (<i>Common Object Request Broker Architecture</i>).
DBA	Administrateur de base de données (<i>Database Administrator</i>). Sa fonction est de gérer la création et le fonctionnement général d'une base de données, particulièrement le placement des données, la performance des accès et les autorisations aux utilisateurs. Il est également responsable du fonctionnement du réseau et de celui des ordinateurs-serveurs.

Terme	Description
DDL	Langage de définition de données (<i>Data Definition Language</i>) qui permet de construire le schéma d'une base de données
DML	Langage de manipulation de données (<i>Data Manipulation Language</i>) qui permet d'instancier les données d'une base de données.
Données	des faits, des statistiques enregistrés.
Entité	Concept de modélisation utilisé pour représenter des objets possédant des clés primaires singulières.
FR	Relation fonctionnelle (<i>Functional Relationship</i>). Concept de modélisation qui représente une contrainte d'intégrité référentielle sous-entendant l'existence d'une clé étrangère. Cette relation est de type plusieurs à un .
GIRD	Dictionnaire global des ressources informationnelles (<i>Global Information Resources Dictionary</i>) définissant le schéma de la base de métadonnées.
GQS	Système de requêtes globales (<i>Global Query System</i>). Approche fondamentale pour les requêtes globales permettant aux usagers d'accéder aux données contenues dans des bases de données distribuées et hétérogènes.
GUI	Interface utilisateur facilitant les interactions personne-machine (<i>Graphical User Interface</i>).
IBMS	Outil de conceptualisation (<i>Information Base Modeling System</i>) qui permet de modéliser un système quelconque en se basant sur la méthodologie TSER.
IDL	Langage de définition d'interfaces (<i>Interface Definition Language</i>) de CORBA.
Information	Données placées dans un contexte d'interprétation
IRDS	Système de dictionnaire d'interfaces (<i>Information Resources Dictionary Standard</i>).
LAN	Réseau local (<i>Local Area Network</i>).
LIC	Cellule d'intégration locale (<i>Local Integration Cells</i>) gérant l'intégration d'applications.

Terme	Description
MDB	Base de métadonnées (<i>Metadatabase</i>). Base de données contenant des métadonnées. Elle permet de conserver les modèles décrivant les différentes bases de données, les règles de distribution de l'information entre ces diverses bases de données et les règles de conversion de l'information entre les différents systèmes, fournissant ainsi un modèle intégré de l'entreprise.
MDBMS	Système de gestion de la base de métadonnées (<i>Metadatabase Management System</i>)
Métadonnées	Données décrivant des données. Elles décrivent l'utilisation, le contenu et les interactions entre des bases de données et des bases de règles, comme par exemple les schémas et les modèles informationnels.
MGQS	Système de requêtes globales assisté par modèle (<i>Model-assisted Global Query System</i>). Approche basée sur la connaissance pour améliorer la formulation et le traitement des requêtes dans la base de métadonnées.
MQL	Langage de requêtes globales (<i>Metadatabase Query Language</i>), qui permet la formulation des requêtes globales au niveau de la base de métadonnées. Avec MQL l'utilisateur peut interroger une collection de systèmes d'informations autonomes de façon non-procédurale.
MR	Relation obligatoire (<i>Mandatory Relationship</i>). Concept de modélisation représentant les contraintes existentielles entre les entités, de telle sorte que si l'entité propriétaire est détruite, les entités possédées le sont aussi. Elle est une relation de type un vers plusieurs .
OE	Entité Opérationnelle (<i>Operational Entity</i>), voir la description d'entité.
OER	Modèle structurel de TSER (<i>Operational Entity/Relationship model</i>) qui est approximativement équivalent aux diagrammes entités/relation classiques, quoique ce modèle peut être aussi utilisé pour la modélisation logique. Le modèle structurel est ordinairement dérivé du modèle fonctionnel.
OMG	Consortium favorisant l'utilisation des méthodes orientées-objets <i>Object Management Group</i> . OGM est à l'origine de CORBA
ORB	Courtier de requêtes objets (<i>Object Request Broker</i>).

Terme	Description
PR	Relation plurale (<i>Plural Relationship</i>). Concept de modélisation pour représenter une relation de type plusieurs à plusieurs entre deux ou plusieurs entités.
ROPE	Environnement de programmation orienté-règles (<i>Rule-Oriented Programming Environment</i>). Il facilite l'implantation et la gestion de l'architecture concurrente de la base de métadonnées.
RPC	Appel de procédure à distance (<i>Remote Procedure Call</i>).
Schéma	La représentation structurel (OER) d'une base de données, d'une base de règles ou d'une base de métadonnées.
SE	Entité Sémantique (<i>Semantic Entity</i>). Voir la description de sujet.
SER	Modèle fonctionnel de TSER (<i>Semantic Entity Relationship model</i>). Permet de modéliser les applications et les données qu'elles utilisent de façon non-normalisée.
SGBD	Système de Gestion de Base de Données (<i>Database Management System; DBMS</i>). Le SGBD est un ensemble de procédures partagé par les utilisateurs pour la création, la manipulation et la sécurité des données en assurant la cohérence dans les opérations et la persistance des traitements transactionnels. Ces procédures coopèrent pour exploiter et gérer les structures de données complexes à savoir, le chaînage, l'adressage calculé (<i>hashing</i>) et le B-arbre.
SQL	Langage structuré de requêtes pour les bases de données (<i>Structured Query Language</i>) souscrit par l'institut de normalisation nationale américain (ANSI).
Sujet	Concept de modélisation utilisé pour représenter l'information statique d'un système d'information, tels que les principes, les objets ou les faits pour lesquels l'application a été conçue.
TSER	Modèle entité/relation à deux phases (<i>Two Stage Entity Relationship</i>) contenant les modèles fonctionnel (SER) et structurel (OER) pour la représentation d'applications.
WAN	Réseau à grande échelle (<i>Wide Area Network</i>).

Chapitre 1

Introduction

Plongée dans un environnement sans cesse plus complexe, plus concurrentiel et toujours changeant, l'entreprise doit tout mettre en oeuvre pour survivre et progresser. Cela implique l'acquisition ou la disponibilité de technologies de l'information adéquates. Utiliser les technologies de l'information afin de se tailler et maintenir une place dans un marché compétitif était impensable il y a une trentaine d'années. À cette époque, on concevait des systèmes de traitement de données qui transformaient des données recueillies aux préalables. Ces données étaient emmagasinées pendant un certain temps dans des fichiers en vue d'un traitement ultérieur. Plus on avance dans le temps, plus on constate que les technologies de l'information ont évolué. Au milieu des années 1980, les personnes oeuvrant en technologie de l'information ont cessé de simplement chercher à satisfaire les besoins perçus. Elles ont plutôt participé à la création et à la réalisation du plan directeur de l'entreprise, en vue d'harmoniser le développement technologique et la croissance de l'entreprise.

Le modèle de Nolan [Nolan79] permet d'évaluer le niveau d'intégration des technologies de l'information à l'intérieur d'une entreprise. Cette approche se base sur l'évolution des dépenses concernant les technologies de l'information dans une entreprise. Le modèle se divise en six étapes, soit : (1) le lancement, (2) la contagion, (3) la gestion, (4) l'intégration, (5) l'administration des données et (6) la maturité.

La plupart des entreprises d'aujourd'hui se situent soit à l'étape de l'intégration, soit à l'étape de l'administration des données. La quatrième étape se caractérise par une intégration du développement des technologies et de celui de l'entreprise. Ainsi, on gère mieux les dépenses liées aux technologies de l'information. On commence également la transformation des applications existantes en vue de les adapter aux bases de données. Désormais, le développement des systèmes au sein de l'organisation se fait dans le cadre d'un cycle de vie soigneusement sélectionné en fonction de la réalité de l'entreprise.

À la cinquième étape, celle de l'administration des données, l'information est dorénavant considérée comme une ressource accessible à tous les usagers. Pour ce faire, il faut gérer l'information avec efficacité et en évaluer les coûts en fonction de son utilité. On emmagasine uniquement les données utiles et on assure leur mise à jour

constante de telle sorte que tous les usagers puissent toujours avoir rapidement accès à des données exactes et fiables.

Les technologies de l'information sont d'une aide précieuse pour la concertation des efforts organisationnels afin que l'entreprise soit exploitée de façon efficiente et qu'elle puisse faire face adéquatement à l'ouverture de nouveaux marchés. Les technologies de l'information représentent une valeur ajoutée pour l'organisation. Les systèmes d'information sont considérés comme membre à part entière de l'organisation. De plus, ils doivent s'intégrer aux diverses fonctions de l'organisation, soutenir le travail d'équipe, la planification et la gestion. Il existe cinq types de systèmes d'information [Kroenke et Hatch94] :

1. **Système transactionnel**

Il supporte les opérations quotidiennes de l'entreprise en conservant les données détaillées de celles-ci. Ce système aide l'entreprise à gérer ses opérations et à les mémoriser.

2. **Système d'information de gestion**

Il produit des rapports de façon régulière et répétitive. Ces rapports peuvent alors être utilisés pour planifier et organiser les opérations de l'entreprise.

3. **Système d'automatisation**

Il améliore les communications à l'intérieur de l'entreprise. Cela est grandement influencé par les différentes technologies de l'information et par l'évolution de celles-ci. L'avènement du bélinographe en est un exemple.

4. **Système d'information pour dirigeants**

Il permet d'obtenir l'information provenant des différentes sources de l'entreprise de façon condensée et de rendre cette information accessible et facilement utilisable. Il doit également permettre aux dirigeants de pouvoir atteindre les données de façon précise. Bien sûr, les données doivent être fiables et à jour.

5. **Système d'aide à la décision**

Il est surtout conçu pour aider à résoudre des problèmes particuliers ou pour aider les entreprises à envisager de nouvelles opportunités. Il facilite l'élaboration de solutions. Par conséquent il doit être flexible et adaptable.

Dans ces environnements d'exploitation, l'information est une ressource vitale et stratégique ; sa disponibilité et sa qualité sont d'une importance capitale [Thacker89]. Tandis que les approches de traitement centralisées permettent de réaliser des économies substantielles, elles empiètent sur la flexibilité et l'expansion (*flexibility and scalability*) de l'infrastructure informationnelle de l'organisation [Babin et al.94].

Les systèmes d'information très diversifiés des années 90 offrent beaucoup de possibilités. Cependant, les buts de ces systèmes restent simples : traiter, gérer et partager de l'information. Toutes ces données sont emmagasinées dans des bases de données de différents types, soit réseau, hiérarchique, relationnel, objet ou autres. Toutes ces

bases de données doivent supporter plusieurs applications, souvent hétérogènes et distribuées, d'où la complexité et la diversité de ces systèmes, les rendant difficiles à gérer et à intégrer.

Une équipe de recherche du département de génie industriel de l'école polytechnique de Montréal propose une technique permettant de quantifier le degré d'intégration informatique d'une entreprise [Alsène et Auclair94, Alsène et Liman96]. Cette mesure du degré d'intégration se base sur deux éléments essentiels, soit : (1) les systèmes informatiques et (2) les liaisons informatiques entre ceux-ci. Elle s'exprime de la manière suivante : $I = F * E$, où I représente le degré d'intégration informatique de l'entreprise concernée, F , le degré d'informatisation des fonctions de celle-ci, et E , le degré d'informatisation des échanges entre les systèmes. Cependant aucune indication n'est donnée quant à l'évaluation de F et de E .

Une telle mesure se révèle utile à bien des points de vue. Tout d'abord, sa constitution fournit une définition à la fois plus précise et plus opérationnelle du concept d'intégration informatique. Une fois constitué, cet indice permet de comparer les entreprises entre elles, facilitant ainsi la recherche de liens entre le degré mesuré et d'autres facettes de l'organisation. Par surcroît, cette mesure n'incorpore pas, dans sa définition ou dans son mode d'évaluation, d'autres aspects de l'organisation.

Il devient de plus en plus important de fournir aux entreprises des approches d'intégration et des outils pour consolider les bases informationnelles disponibles par le biais de ces systèmes distribués, tout en maintenant leur autonomie et leur indépendance. Les divers concepts d'intégration sont supportés par des processus interopérables, où les systèmes concernés peuvent mutuellement échanger des informations, indépendamment de l'hétérogénéité et de leurs contraintes de distribution. L'objectif principal est de permettre l'échange d'information entre différents partenaires et ce, indépendamment de leurs standards.

Les approches d'intégration traditionnelles effectuent l'interopérabilité au niveau des applications (entre les systèmes locaux) via un schéma global ou un langage de manipulation commun [Dilts et Wu91, Dilts et Wu92]. La conception d'un schéma global implique la combinaison de différentes sortes de domaines dans un modèle, qui est généralement difficile à créer et à entretenir. Par conséquent, la définition d'un langage commun représente un défi de taille, car chaque partenaire possède ses propres standards et besoins. La plupart de ces approches traditionnelles ont fourni un support au niveau de la représentation et spécification sans vraiment prendre en considération la flexibilité et la distributivité.

Indéniablement, l'intégration de systèmes est à la fine pointe de la modernisation technologique des entreprises et cette activité se compose, à l'heure actuelle de trois grandes classes de systèmes informatiques [Alsène95], soit :

1. systèmes d'applications intégrés

Ils comprennent éventuellement de très nombreux programmes, formant un tout et procurant des applications à diverses fonctions de l'entreprise ;

2. systèmes d'applications modulaires

Ce sont des systèmes composés de regroupements d'applications (ou modules) qui sont reliés les uns aux autres, mais qui fonctionnent aussi de manière plus ou moins autonomes les uns par rapport aux autres. Les liens entre ces modules sont assurés de trois façons différentes, soit : (1) par des interfaces, (2) par des bases de données communes ou (3) des bases de données distribuées ;

3. métasystèmes d'application

Ils renvoient également à la mise en place d'une interface, d'une base de données commune ou d'une base de données distribuée, mais cette fois entre diverses applications et/ou systèmes indépendants, déjà existants ou nouveaux.

L'approche traitée dans ce mémoire est en relation avec la troisième classe de systèmes. Les travaux de recherche sur la base de métadonnées se sont concentrés sur la création d'un environnement d'intégration et sur la définition de ses principales composantes, créant ainsi une solution pour résoudre les problèmes d'interopérabilité dans les entreprises [Hsu96, Hsu91, Hsu et Babin93, Hsu et al.92a, Hsu et Rattner93, Hsu et al.91, Hsu et Rattner90, Hsu et al.93].

1.1 Architecture de la base de métadonnées

L'administration de l'information dans une entreprise moderne se fait par le biais de bases de données hétérogènes et distribuées. La plupart des approches d'intégration de bases de données autonomes utilisent un mécanisme de contrôle central des transactions entre les diverses bases de données, créant ainsi un goulot d'étranglement. La base de métadonnées permet de conserver les modèles décrivant les différentes bases de données, les règles de distribution de l'information entre ces diverses bases de données et les règles de conversion de l'information entre les différents systèmes. Une base de métadonnées gère l'information de ces bases de données. Cette information peut être distribuée sur différents sites selon les besoins, diminuant ainsi la possibilité d'apparition d'un goulot d'étranglement. L'architecture de la base de métadonnées est illustrée à la figure 1.1.

Le système de gestion de la base de métadonnées (*MetaDataBase Management System*; MDBMS) agit comme système d'exploitation et, à ce titre, coordonne et contrôle toutes les activités facilitant la gestion et le traitement des métadonnées. Il rend également transparent à l'utilisateur les disparités de stockage et de traitement grâce à des interfaces bien adaptées.

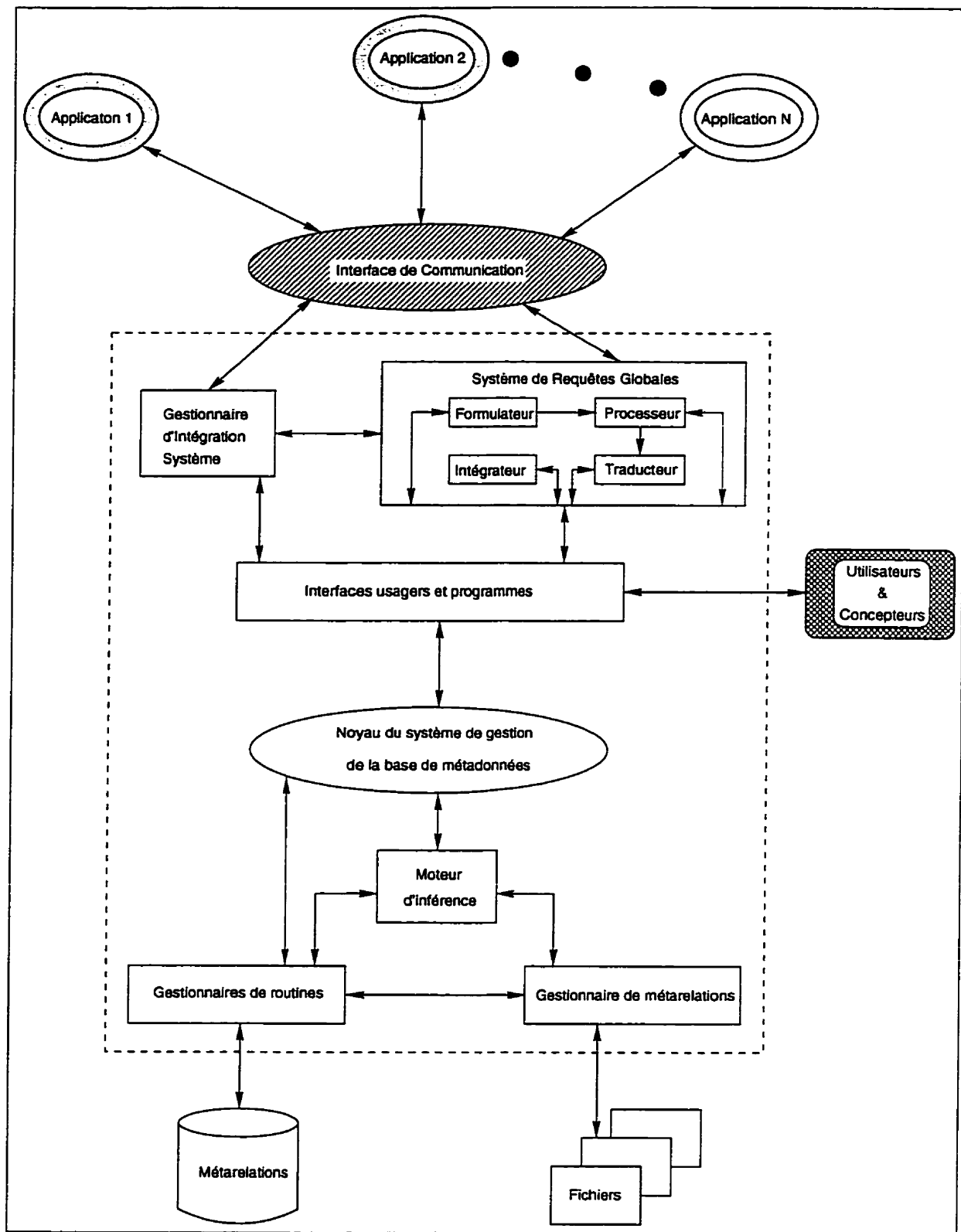


Figure 1.1 : L'architecture de la base de métadonnées

Le dictionnaire global des ressources informationnelles (*Global Information Resources Dictionary*; GIRD) [Bouziane91] est le schéma de la base de métadonnées. La représentation des applications dans la base de métadonnées est effectuée à l'aide de la méthode de modélisation *Two-Stage Entity-Relationship* (TSER) [Hsu91]. Cette méthode est en quelque sorte une extension naturelle du modèle Entité-Relation. Le formalisme TSER permet de définir les modèles fonctionnels et structurels des applications de l'entreprise. TSER propose également des règles de construction du modèle structurel à partir du modèle fonctionnel ; cette méthode de modélisation permet de concevoir trois niveaux [Hsu et al.93] :

1. **Modélisation**

C'est une méthode neutre de représentation commune où tous les paradigmes peuvent être transférés.

2. **Intégration des modèles**

Le métamodèle est un schéma de métadonnées, qui synthétise et structure tous les modèles de l'entreprise dans la base de métadonnées.

3. **Gestion des informations**

Le métamodèle définit un modèle intégré de l'entreprise, tel que décrit dans la base de métadonnées.

Les principaux avantages de l'utilisation d'un métamodèle sont la simplification de la complexité du problème d'intégration par le développement d'un schéma conceptuel et la minimisation des impacts de changements au niveau des systèmes existants, par rapport au reste de l'entreprise.

Le concept de la base de métadonnées est une représentation simplifiée de l'architecture des systèmes distribués, des modèles d'informations hétérogènes et du contrôle des bases de connaissances. La base de métadonnées fournit un modèle intégré de l'entreprise englobant les multiples systèmes d'information, leurs ressources ainsi que les connaissances qu'ils véhiculent. Le modèle de la base de métadonnées présente une architecture qui fournit trois types de fonctionnalités [Hsu et Rattner93] :

1. **Fonctionnalité passive**

La base de métadonnées fournit une représentation consolidée des modèles de données et de la connaissance opérationnelle/contextuelle. Elle peut aussi être utilisée comme une ressource d'information autonome pour les utilisateurs.

2. **Fonctionnalité semi-active**

Utilisée sous ce mode, la base de métadonnées fournit aux utilisateurs la possibilité de formuler des requêtes globales par le biais du *Global Query System* (GQS) [Cheung91] ; cela est un moyen de lier les différents systèmes locaux. Ce type de fonctionnalités supporte les requêtes des utilisateurs ne possédant pas une connaissance technique avancée. L'intervention des métadonnées dans le cadre des requêtes globales est importante : elle permet : (1) de formuler les requêtes, (2) de décomposer les requêtes et (3) d'intégrer les résultats au niveau local.

3. Fonctionnalité active

En se basant sur les modèles de données et la connaissance emmagasinée dans la base de métadonnées, et en utilisant les caractéristiques de ROPE (*Rule-Oriented Programming-Environment*) [Babin93], le système actif gère les informations globales. Cela est assuré grâce à l'implantation de la connaissance appropriée dans les systèmes locaux.

Étant donné la diversité des différents systèmes de gestion de bases de données, il est difficile d'avoir un système qui soit capable d'interagir avec chacun des systèmes. Pour cela, l'implantation d'une base de métadonnées, qui contient la représentation des systèmes d'information déjà en place dans l'entreprise, peut faciliter les différentes interactions entre ces systèmes, puisqu'elle utilise une architecture concurrente, permettant d'avoir un système global ouvert. Lorsqu'on désire intégrer ou enlever une application dans l'entreprise, tout se fait à un seul endroit, c'est-à-dire dans la base de métadonnées.

1.2 Problématique

Le travail nécessaire pour intégrer une nouvelle application est complexe. Pour débiter, il faut modéliser le nouveau système qui doit être incorporé dans l'environnement informationnel de l'entreprise. Une méthode unique de modélisation n'est pas requise. Cependant, il faut prendre en considération la sémantique pour ainsi capturer la logique de la nouvelle application. Le formalisme TSER est une approche très riche et puissante [Hsu et al.93]. Elle est utilisée pour représenter des modèles déjà existants qui ont été créés en utilisant différents paradigmes et méthodologies. Toute nouvelle application à intégrer dans la base de métadonnées est modélisée avec TSER, soit directement, soit par traduction de paradigme [Hsu et al.93]. Le résultat de la phase de modélisation est un modèle fonctionnel et un modèle structurel. Une fois le modèle créé, il doit être intégré avec le modèle concurrent existant (cf. figure 1.2).

La gestion de l'architecture concurrente suit ce cheminement : en premier lieu, le modèle courant de l'entreprise est extrait de la base de métadonnées pour y intégrer le modèle de la nouvelle application. Cette intégration se fait au niveau fonctionnel grâce aux contextes qui peuvent exister entre ces applications. Ces contextes contiennent les connaissances opérationnelles entre les différents systèmes. Par la suite, les attributs équivalents sont définis entre les applications, sans oublier le développement de règles de conversion, lorsque nécessaire. Au niveau structurel, les connaissances déclarées sur les données équivalentes sont utilisées pour ajouter le nouveau modèle structurel à l'ancien, résultant ainsi en un nouveau modèle consolidé de l'entreprise, c'est-à-dire, avec sa nouvelle application intégrée dans la base de métadonnées.

Une fois l'application intégrée au modèle de l'entreprise, on peut définir son *shell* en utilisant le langage de définition des *shells*. Les *shells* peuvent être mis à jour avec la connaissance décrivant le comportement global de l'application nouvellement intégrée. Pour créer un *shell*, quatre étapes sont nécessaires [Babin93] : (1) préparer les liens de

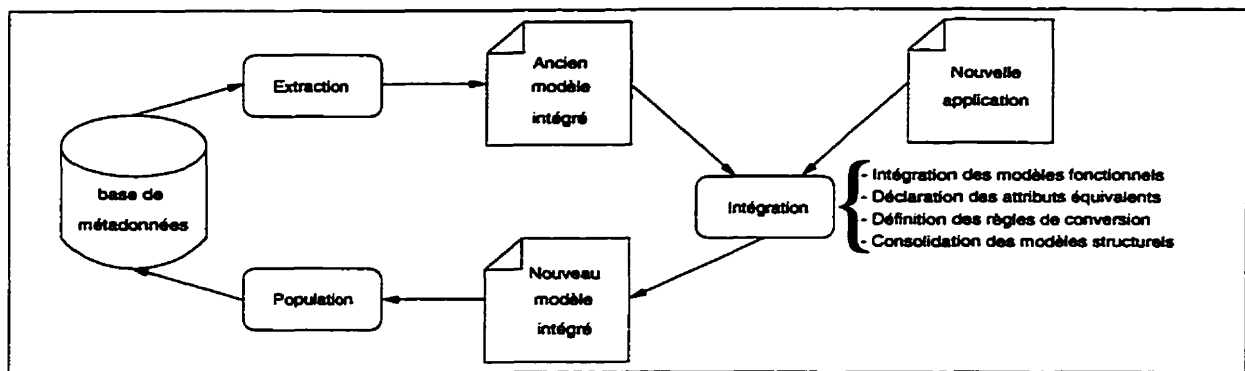


Figure 1.2 : Processus d'intégration des modèles dans la base de métadonnées

communication entre le nouveau et l'ancien système, (2) définir les liens entre le *shell* et sa base de données locale, (3) générer le *shell* en utilisant le langage de définition des *shells* et (4) instancier le *shell* avec les différentes règles incluant les règles de conversion. L'insertion d'une nouvelle application fait intervenir la base de métadonnées et les *shells*. Pour effectuer l'intégration, il faut posséder une connaissance approfondie du modèle fonctionnel du GIRD, c'est-à-dire connaître l'architecture des différentes vues représentant un ensemble d'applications.

1.3 Motivations

La première motivation concerne le développement d'une solution viable et efficiente pour résoudre le problème d'intégration d'applications dans un contexte de base de métadonnées. L'intégration de plusieurs systèmes déjà existants dans une entreprise provoque des difficultés majeures. Le principe de la base de métadonnées peut faciliter l'implantation d'un système global en milieu de production.

La deuxième motivation vient du fait que le processus d'intégration d'une application dans une base de métadonnées est relativement complexe. La tâche doit être réalisée manuellement en plusieurs étapes. Le développement d'un logiciel prévu à cette effet, facilitera de beaucoup la tâche d'intégration.

La troisième motivation provient de la continuité des travaux menés à *Rensselaer Polytechnic Institute* sur un environnement avec une seule base de métadonnées [Hsu et al.91].

1.4 Objectifs

Le projet de recherche a pour but d'étudier en détail la technique déjà utilisée pour l'intégration de nouvelles applications, d'explorer des domaines connexes et de proposer des améliorations pour alléger et faciliter le processus d'intégration d'une nouvelle application. L'étude de la structure du dictionnaire global des ressources informationnelles

permet de déterminer les informations minimales requises pour l'intégration d'une application dans le modèle structurel global consolidé de l'entreprise, tout en maintenant l'intégrité et la cohérence du contenu de la base de métadonnées.

Les principaux objectifs visés par ce mémoire sont :

1. Étudier la structure et les fonctionnalités de la base de métadonnées
 - L'étude de la structure de la base de métadonnées, c'est-à-dire les modèles fonctionnels et structurels du GIRD.
 - L'étude du système de requêtes globales (GQS).
 - L'étude de l'environnement de programmation orienté-règles (ROPE)
2. Définir les contraintes de structuration d'une cellule locale d'intégration.
3. Établir l'information minimale nécessaire à l'intégration d'une application.
4. Déterminer les modifications fonctionnelles et informationnelles pour effectuer cette intégration.

1.5 Organisation du mémoire

Le mémoire est composé de huit chapitres. Après une présentation de l'architecture de la base de métadonnées et de la problématique au **chapitre 1**, le **chapitre 2** présente différentes approches d'intégration tandis que le **chapitre 3** explique les concepts de la méthode de modélisation TSER. Pour ce qui est du **chapitre 4**, il décrit en détails les modèles fonctionnel et structurel du dictionnaire des ressources informationnelles (objectif 1). Le **chapitre 5** démontre le fonctionnement du système de requête globale (objectif 1), tandis que le **chapitre 6** explique le fonctionnement de l'architecture concurrente adaptable de la base de métadonnées (objectif 1). Le **chapitre 7** présente les contraintes de structuration (objectif 2) ainsi qu'une description des métadonnées requises pour l'intégration d'une nouvelle application (objectif 3) et les modifications fonctionnelles devant être apportées à l'outils d'intégration IBMS. La conclusion de ce mémoire est rédigée dans le **chapitre 8** ; elle présente les perspectives de recherche et un compte rendu du projet de recherche dans son ensemble.

Chapitre 2

Différentes approches d'intégration

L'évolution des technologies de l'information et l'ampleur des systèmes de télécommunication obligent les organisations à adopter une structure répartie pour répondre adéquatement aux nouvelles exigences que crée cette évolution. Une telle structure implique le partage des ressources d'une entreprise, particulièrement les systèmes d'information, ce qui conduit au développement d'environnements autonomes et distribués, chacun d'eux supportant les besoins de ses utilisateurs. Les interactions coopératives entre ces environnements peuvent être conduits dans un système interopérable. Dans ce chapitre, nous présentons quatre approches de systèmes globaux de partage d'information, soit : (1) les bases de données fédérées, (2) le système CARNOT, (3) le projet RODIN et (4) le système TSIMMIS. Étant donné que les techniques mentionnées précédemment utilisent l'architecture orientée-objets et que cette dernière est en voie de devenir le standard pour l'interopérabilité entre objets distribués, nous examinerons en plus le protocole de communication et d'échange d'information CORBA d'OMG.

2.1 Bases de données fédérées

Les systèmes de partage de données globales ont été un secteur majeur de recherche durant la dernière décennie. Un grand nombre de solutions commerciales et académiques ont été proposées, tel que ADDS, Calida, Dataplex, IDS, IISM, IMDAS, Ingres/Star, Mermaid, MRDSM, Multibase, NDMS, Omnibase, Preci et Sabrina [Reddy et Subhash93].

La principale justification de cette approche est d'assurer l'autonomie totale des différentes bases de données et de privilégier leur administration, gestion et manipulation de manière indépendante. L'utilisateur d'une base de données fédérée (*multidatabase*) [Litwin et al.90] ne perçoit pas une base de données globale mais un ensemble de bases de données inter opérables. L'absence de schéma global permet de tolérer les conflits sémantiques qui causent des problèmes lors de la conception des bases de données réparties hétérogènes. Cependant, cette technique rend ardu le processus d'intégration de nouvelles applications, car elle nécessite la mise à jour des schémas des différentes

fédérations qui composent le système global.

L'absence de schéma global a un impact décisif sur l'architecture et l'utilisation d'une base de données fédérée. Une architecture des schémas et leurs différents niveaux, tels que proposés par [Litwin88], sont présentés de manière ascendante à la figure 2.1 : au plus bas niveau apparaissent les schémas internes et conceptuels de chaque base de données existante. Ces schémas correspondent aux modèles locaux d'une base de données répartie et peuvent être confondus, par exemple, dans le cas d'une base de données réseau.

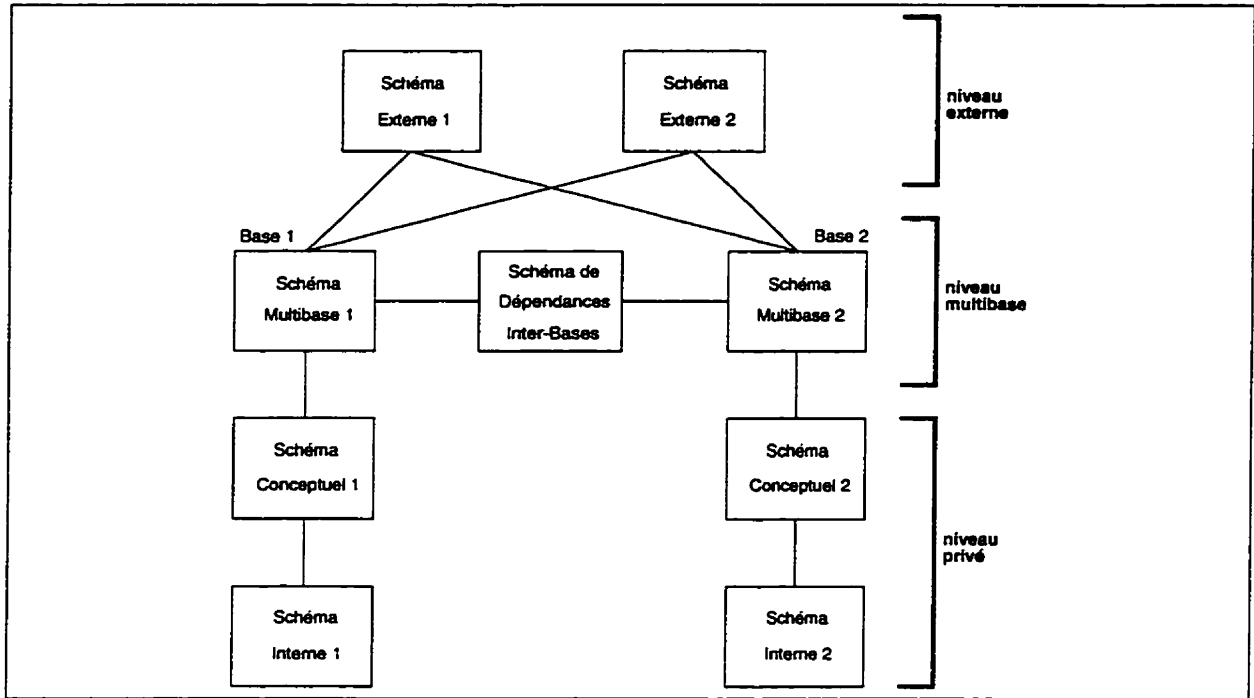


Figure 2.1 : Architecture des schémas d'une base de données fédérée

Un schéma multibase, appelé aussi schéma exporté [Heimbigner et McLeod85], décrit dans un modèle commun de données le sous-ensemble d'une base de données existante faisant partie de la fédération. Le modèle commun de données, similaire au modèle pivot d'une base hétérogène, permet de décrire de manière uniforme les données représentées dans différents modèles et fournit donc l'indépendance aux SGBD. Le schéma de données relationnel est également le choix naturel du modèle commun. Dans le cas d'hétérogénéité des bases de données, un traducteur est nécessaire pour convertir les données et requêtes depuis le modèle commun vers le modèle de la base de données.

Un schéma des dépendances inter-bases permet de lier deux ou plusieurs bases de données par des contraintes d'intégrité sémantiques et de protection inter-bases. En l'absence de schéma global, ce schéma est le seul moyen de préserver la cohérence des données des différentes bases. L'ensemble des schémas multibases et des schémas de

dépendances décrit toute la base de données fédérée.

Finalement, au niveau le plus haut, des schémas externes peuvent être construits afin de faciliter la manipulation des données appartenant à différentes bases. De plus, l'utilisateur peut accéder directement à des bases de données multiples au moyen d'un langage multibase sans passer par un schéma externe.

Les nouvelles fonctions nécessaires à la gestion d'une base de données fédérée sont fournies par le langage multibase. Afin d'éviter le développement d'un langage complètement nouveau, une bonne approche consiste à incorporer les concepts multibases dans un langage conçu pour une base de données centralisée. Le langage multibase comporte en fait deux langages, un pour la définition des données multibases et un autre pour leur manipulation.

2.1.1 Discussion sur les multibases de données

Les difficultés techniques limitant le développement des SGBD multibases sont nombreuses. L'approche multibase fournit à l'utilisateur potentiel une collection de schémas locaux décrivant l'architecture et le fonctionnement d'un ensemble de bases de données. Par contre, l'usager se voit dans l'obligation de comprendre et résoudre les conflits interbases et de formuler correctement sa requête, tout en respectant la sémantique des bases de données concernées par celle-ci. Chacune des BD doit maintenir des connaissances concernant la structure et la sémantique de ses consœurs. L'instauration d'un schéma global éliminerait les problèmes en relation avec la fonctionnalité des requêtes interbases de données.

L'autonomie et le manque d'intégration des bases de données hétérogènes rendent inapplicables la plupart des solutions apportées dans le contexte des bases de données réparties [IEEE87]. En particulier, les hypothèses simplificatrices concernant la transparence de la répartition des données, la durée et l'atomicité des transactions ainsi que la cohérence des données doivent être réexaminées dans le contexte multibase. Les fonctions les plus complexes restent l'évaluation de requêtes et la gestion de transactions multibases.

2.2 CARNOT

Carnot [Woelk et al.93] est développé par la Microelectronics and Computer Technology Corporation (MCC) à Austin (Texas). Le principal objectif de ce projet est d'unir des architectures physiquement distribuées, c'est-à-dire des systèmes d'information hétérogènes présents dans les organisations et ce, par le biais de liens logiques d'unification. Carnot offre un ensemble d'options génériques focalisant sur la résolution de problèmes d'intégration de ces ressources informationnelles. Ses options sont réparties en cinq services distincts (cf. figure 2.2), soit : (1) l'accès, (2) la sémantique, (3) la distribution, (4) le support et (5) la communication.

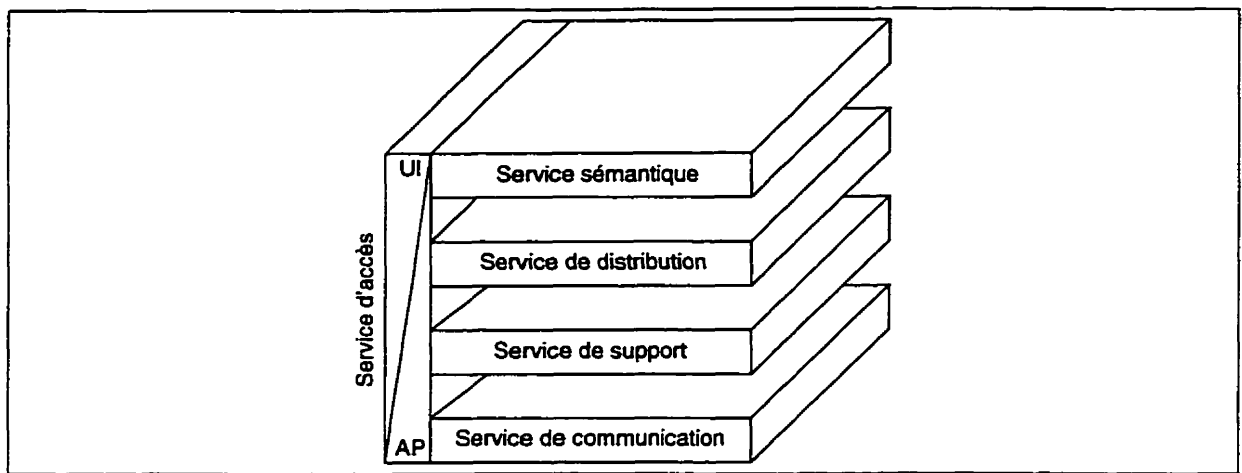


Figure 2.2 : Architecture de Carnot

Service d'accès

Le service d'accès fournit des mécanismes d'interaction aux développeurs, via des interfaces graphiques conviviales, pour la gestion et la manipulation des quatre autres services de Carnot, ce qui donne la possibilité aux concepteurs de construire un système d'intégration global. L'interface utilisateur de Carnot inclut des outils de visualisation de modèles 2D ou 3D et un environnement de traitement éductif orienté-objets, c'est-à-dire le LDL++, qui est entièrement intégré au C++ et optimisé pour le traitement de requêtes récursives.

Service sémantique

Le service sémantique permet d'obtenir une visualisation globale de toutes les ressources intégrées de l'entreprise. La modélisation de ces ressources est réalisée par le biais des modèles d'intégration de Carnot, qui utilisent une base de connaissances commune pour définir un schéma global et des mécanismes de fédération pour l'expression de concepts d'intégration cohérents à l'intérieur d'un ensemble de modèles représentant l'infrastructure informationnelle de l'entreprise [Huhns et al.92]. Le service sémantique utilise une série extensible de propriétés fonctionnelles pour la représentation d'un modèle déclaratif à l'intérieur du schéma global de l'entreprise et pour la construction de flux de données bidirectionnels entre le modèle déclaratif et son schéma global.

Service de distribution

Le service de distribution supporte les processeurs de transactions (*relaxed transaction processors*) qui gèrent les incompatibilités informationnelles et les fonctionnalités des agents distribués qui interagissent avec : les applications clientes, les services de répertoires, les gestionnaires de stockage et les contraintes des ressources déclaratives de Carnot nécessaires à la construction des *scripts* pour les *shells* ESS (*Extensible Services Switch*) dédiés au transport de certaines fonctions d'affaires [Woelk et al.92]. Les *scripts* de flux de travail accumulent la culture corporative et exécutent des

tâches reflétant fidèlement les réalités d'affaires courantes. Les contraintes de ressources déclaratives se manifestent en un ensemble de prédicats exprimant des règles d'affaires, des dépendances inter-ressources, des contrôles et des stratégies d'uniformités et ce, à la grandeur de l'entreprise.

Service de support

Le service de support implante les fonctionnalités de base du réseau de communication pour l'architecture intégrée de l'entreprise. Une composante primordiale de ce service est l'environnement de *shells* distribués ESS [Tomlinson et al.92]. Un ESS fournit des accès aux ressources de communications, aux ressources d'information locales et aux applications du site courant.

Service de communication

Le service de communication intègre les diverses plates-formes d'exploitation d'applications qui se retrouvent dans une entreprise et ce, par le biais d'une méthode uniforme d'interconnection d'équipements et de ressources hétérogènes.

2.2.1 Dimension de l'intégration sémantique

La fonctionnalité de Carnot est basée sur une approche dite *composite*, avec quatre améliorations supplémentaires, ce qui la rend plus flexible. Premièrement, Carnot utilise la base de connaissances Cyc [Doug et Guha90] où sont comparés et fusionnés les modèles locaux des sources de données, ce qui facilite la création du schéma global de l'entreprise.

Deuxièmement, Carnot permet de capturer la description structurelle des modèles locaux, en plus de toute autre information disponible, incluant les connaissances relatives : (1) aux schémas, comme par exemple les structures de données, les contraintes d'intégrité et les définitions des opérations permises ; (2) aux ressources, comme par exemple les descriptions des services supportés tels que les modèles de données et les langages, les définitions lexicales des noms des objets, les commentaires des concepteurs en relation avec les ressources en traitement et les données elles-mêmes ; (3) à l'organisation, comme par exemple, les règles de corporation gouvernant l'utilisation des ressources.

Troisièmement, Carnot saisit les relations entre les modèles locaux et le schéma global en définissant un ensemble d'axiomes (*articulation axioms*), qui fournissent un moyen pour la traduction de termes en un langage commun, donnant ainsi la possibilité de partager l'information inter-applications.

Quatrièmement, Carnot prend en considération les types des ressources, qu'il s'agisse d'une base de données, d'un modèle de traitement ou d'un système à base de connaissances (SBC). L'approche d'intégration d'un SBC dépend du rôle qu'il joue :

1. Un SBC peut être une ressource informationnelle ; dans ce cas, il est intégré de la même manière qu'une base de données.
2. Un SBC peut être une application accédant à d'autres ressources ; dans ce cas, l'intégration de ses ressources est effectuée en relation avec le SBC lui-même.
3. Un SBC peut faire partie d'un mécanisme de fédération servant d'intermédiaire entre les applications et les bases de données faisant partie de cette fédération et ce, au niveau sémantique ; dans ce cas, l'ontologie de ce SBC devra être fusionnée avec le schéma global. Le SBC maintient les représentations des schémas de bases de données dont il a la charge.

2.2.2 Métamodélisation

L'utilisation de la base de connaissances commune Cyc, comme mécanisme de fédération des modèles locaux, assure la cohérence d'intégration du schéma global. De plus, Carnot réfère à la méthode de représentation des connaissances de Cyc pour exprimer les structures d'information statiques et les processus dynamiques d'une entreprise. La méthode Cyc supporte plusieurs formalismes de modélisation, tels que IRDS, AD/Cycle d'IBM, CLDM de Bellcore et le modèle d'information d'Andersen Consulting. La relation entre Cyc et ces formalismes est illustrée par la figure 2.3, tandis que les relations entre les formalismes et leurs instances spécifiques le sont par la figure 2.4.

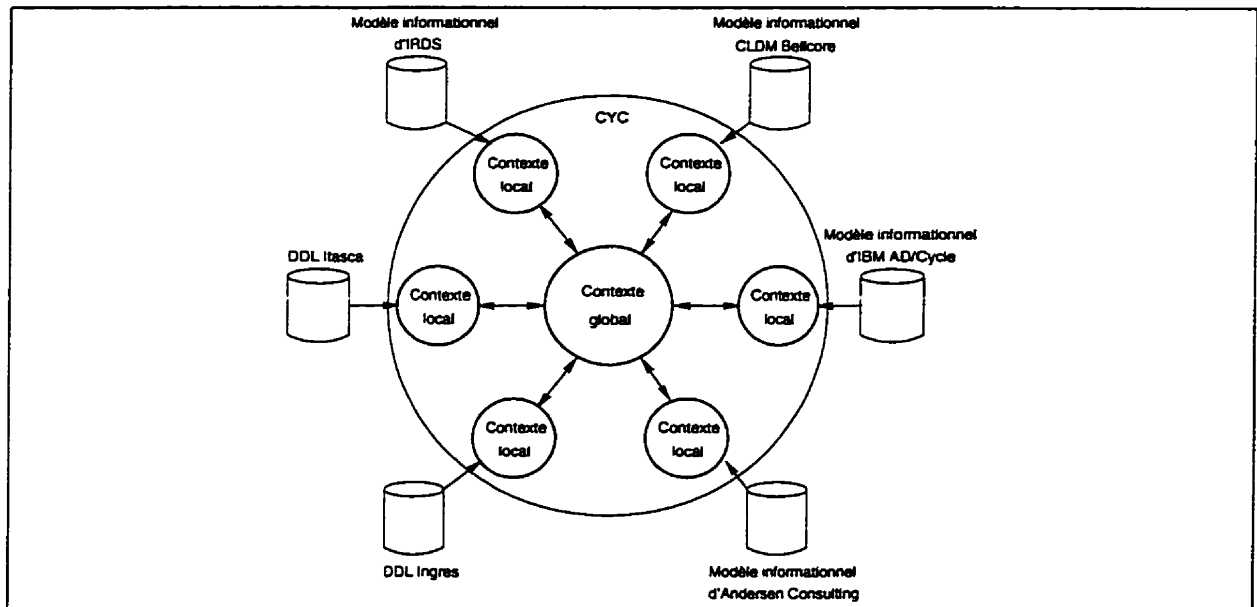


Figure 2.3 : Cyc inter-reliant des formalismes de modélisation populaire

Carnot peut fournir la sémantique de n'importe quelle instance de données contenue dans le métamodèle, qui est le schéma global. Par exemple, Carnot s'assure que pour une entité donnée, sa sémantique est la même à la grandeur de l'entreprise et ce, peu importe la ressource accédant à cette entité.

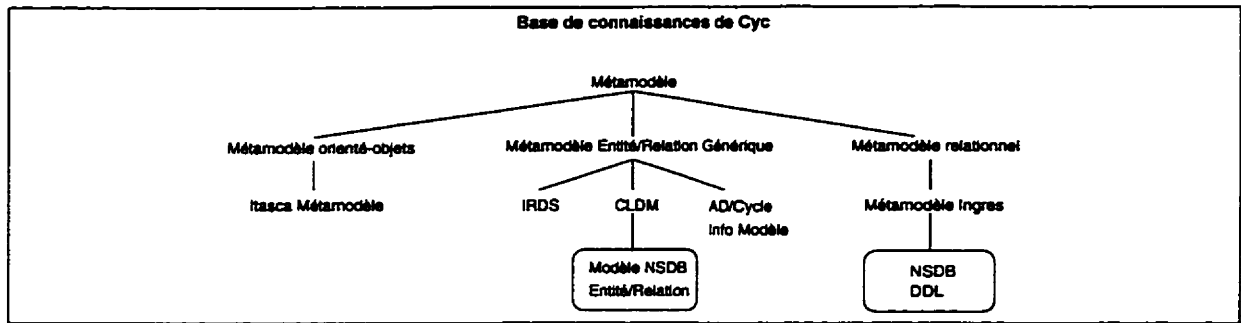


Figure 2.4 : Base de connaissances Cyc

Les relations entre le concept d'un modèle local et un ou plusieurs concepts du schéma global sont exprimées par le biais d'axiomes, qui manipulent le contenu du schéma global et inter-relient les modèles locaux d'une organisation. Ces derniers peuvent être intégrés indépendamment les uns des autres et les axiomes qui en résultent ne nécessitent aucune mise à jour lors de l'intégration d'un nouveau modèle local.

2.2.3 Transaction entre la vue globale et les vues locales

Une ressource est intégrée en spécifiant une traduction syntaxique et sémantique entre elle-même et le schéma global. La traduction syntaxique fournit une spécification opérationnelle entre le langage d'une ressource locale, *RML_i* (*Remote Manipulation Language*), et le langage du schéma global, *GCL* (*Global Concept Language*), qui est basé sur une version étendue de la logique du premier ordre. La traduction sémantique est une représentation entre deux expressions *GCL* ayant une signification équivalente. Ce processus est réalisé par le biais d'un ensemble d'axiomes. Chacun des axiomes a la forme $ist(G \theta) \Leftrightarrow ist(C_i \psi)$, où θ et ψ sont des expressions logiques et *ist* est un prédicat qui signifie "est vrai dans le contexte". Cet axiome exprime que la signification de θ dans le schéma global *G* est la même que ψ dans le modèle local *C_i*. Au plus, *n* ensembles d'axiomes sont nécessaires pour *n* ressources locales.

Après l'intégration d'une ressource quelconque, celle-ci peut accéder aux autres ressources via la vue globale, donnant ainsi l'illusion d'une ressource informationnelle unique, mais qui requiert l'utilisation du *GCL*. Des opérations de requêtes et de mises à jour peuvent être issues d'une ou plusieurs vues locales. Ces opérations sont premièrement traduites en *GCL* et en second lieu dans les différents langages locaux *RML_i*, et par la suite, acheminées aux ressources informationnelles appropriées.

2.2.4 Développement des axiomes

Carnot possède un logiciel graphique, le MIST (*Model Integration Software Tool*), qui automatise les routines d'intégration de modèles, tout en affichant à l'utilisateur les informations nécessaires à la bonne conduite de cette opération. L'outil génère des axiomes pour les trois étapes suivantes :

1. MIST représente automatiquement les caractéristiques d'une ressource informationnelle en une représentation locale équivalente, devenant ainsi une instance d'un formalisme supporté par Cyc. La représentation est déclarative et utilise un ensemble extensible de propriétés sémantiques.
2. Les contraintes de propagation fournies par l'utilisateur, permettent de jumeler les concepts d'un modèle local avec les concepts du schéma global.
3. Pour chacun des jumelages, MIST construit automatiquement un axiome en instanciant des gabarits d'axiomes (*axiom templates*).

Grâce au logiciel d'intégration, l'analyste peut naviguer dans le schéma global et les modèles locaux ; de plus, il a la possibilité de visualiser les axiomes qui sont contenus dans la base de connaissances, ce qui permet à l'opérateur de modifier ces informations à son gré.

2.2.5 Discussion sur Carnot

L'approche d'intégration Carnot est similaire à celle de la base de métadonnées (*MetaDataBase*; MDB) et ce, en trois points :

1. Les caractéristiques des ressources hétérogènes réparties d'une entreprise sont, dans Carnot, emmagasinées dans une base de connaissances représentant un schéma global. MDB offre déjà ces services, en plus d'une architecture bien définie facilitant la gestion de la cohérence du contenu de la base de connaissances et ce, par le biais du dictionnaire global des ressources informationnelles (voir chapitre 4).
2. La manipulation des données distribuées à la grandeur de l'entreprise est effectuée à l'aide du langage de requêtes globales, soit GCL pour Carnot et MQL (*Metadata Query Language*) pour MDB. Les requêtes globales dans l'approche de la base de métadonnées sont effectuées via le système de requêtes globales (voir chapitre 5).
3. Les services de distribution et de support de Carnot possèdent une structure de *shells* offrant des fonctionnalités de communication inter-applications. L'architecture concurrente adaptable (voir chapitre 6) de la MDB possède également une structure de *shells* basée sur la programmation orientée-règles. Cette caractéristique permet d'intégrer les applications sans devoir les modifier ou les adapter sous aucune forme. L'intégration peut être réalisée grâce à la métamodélisation des applications avec la méthode TSER via le logiciel IBMS (voir chapitre 3).

Le processus d'intégration dans Carnot est plus évolué que celui de la base de métadonnées. L'outil MIST offre un ensemble complet de fonctionnalités facilitant la gestion de la base de connaissances Cyc ; cela n'est pas le cas pour le système de gestion de la base de métadonnées. Ce projet de maîtrise vise à établir et définir les caractéristiques du processus d'intégration d'une nouvelle application dans une MDB.

Il serait intéressant d'obtenir une collaboration étroite entre les deux équipes de chercheurs. Ils pourraient ainsi partager leurs résultats et expertises en collaborant à l'établissement d'un prototype efficient en milieu de production.

2.3 RODIN

L'objectif général du projet Rodin est de concevoir et d'expérimenter de nouvelles techniques afin d'améliorer les fonctionnalités et les performances de la gestion de bases de données hétérogènes. Ces techniques prennent la forme de langages ou d'algorithmes qui peuvent être conceptualisés dans des outils ou composantes de bases de données.

Depuis 1996, le groupe Rodin de l'Institut National de Recherche en Informatique et en Automatique (INRIA) de France, a mis l'accent sur l'étude des problèmes de BD qui se posent dans deux grands domaines d'applications, soit : (1) le commerce électronique, en particulier les bourses d'affaires électroniques, et (2) les systèmes d'information dans le domaine de l'environnement. Ce projet se concentre sur les points suivants :

1. Extension de la flexibilité et de l'expressivité des modèles de données objets supportés par les SGBD existants.
2. Aide à la conception de ressources informationnelles actives, dans lesquelles les contrôles d'intégrité et les règles de gestion sont automatiquement vérifiés, et centralisés dans les bases de données au lieu d'être gérés par les applications. Ceci est réalisé par le biais d'outils d'aide à la conception d'applications.
3. Découverte et recherche de l'information pertinente dans des sources de données hétérogènes et autonomes sur un réseau global de type inforoute (Internet, Intranet, Extranet). Le but est de fournir des outils facilitant la gestion de répertoires de métadonnées et d'index, et leur utilisation pour optimiser l'accès à l'information.
4. Mise en fédération de sources de données hétérogènes et réparties dans un réseau. L'objectif est de permettre une intégration cohérente des données et de garantir par la suite que les règles de gestion formalisées par les participants à une fédération seront respectées lors de la mise à jour de façon pseudo-autonome des bases de données.
5. Gestion d'objets persistants dans un environnement réparti, multi-utilisateurs et tolérant aux fautes. Ce domaine a pour objectif la conception de mécanismes et d'outils performants qui seront à la base des futurs systèmes de gestion de base de données orientée-objets. Le projet s'intéresse spécialement aux systèmes utilisant l'architecture client/serveur.

La valorisation industrielle est importante pour le mégaprojet Rodin. D'une part, elle permet de confronter des solutions développées à l'INRIA dans un contexte d'utilisation réel ; d'autre part, la collaboration avec des partenaires industriels est à la base

de la définition de nouveaux problèmes de recherche pour l'INRIA. Les principaux partenaires industriels sont Bull et O2 Technology, avec qui le groupe de recherche mène des opérations de transfert technologique. Les principaux utilisateurs sont le ministère de l'environnement de la France, le CEMAGREF et l'IFEN.

Dans la prochaine partie, nous présentons une description d'un système de recherche d'information dans des sources de données hétérogènes, puisque le projet Disco est directement relié aux points 3 et 4 mentionnés précédemment.

2.3.1 Disco

Le nombre, la variété et la taille des sources de données accessibles dans un réseau global comme Internet sont en expansion rapide et introduisent deux problèmes majeurs. D'abord, il est difficile de savoir quelles sont les données pertinentes et leur localisation. Ensuite, il reste difficile d'accéder et d'intégrer rapidement des données hétérogènes sur un réseau global à partir de critères de recherche complexes. Pour aborder ces problèmes, le projet Disco (*Distributed Information Search COmponent*) [Bonnet et Tomasic97, Kapitskaia et al.97, Naacke et al.97, Raschid et al.96. Raschid et Valduriez95], dont l'objectif est de construire un système de recherche d'informations dans des sources de données hétérogènes, a été lancé en 1995 à l'INRIA, sous la tutelle du mégaprojet Rodin. Cette recherche est axée sur trois aspects complémentaires : (1) définition de l'architecture de Disco, (2) reformulation de requêtes hétérogènes et (3) exécution de requêtes réparties.

Définition de l'architecture de Disco

Pour uniformiser la représentation des données, Disco s'appuie sur le modèle standard de l'ODMG (*Object Database Management Group*) en étendant légèrement les langages ODL (*Object Definition Language*) avec des définitions de vues globales et OQL (*Object Query Language*). Disco adopte l'architecture récente des systèmes de bases de données hétérogènes qui évite la gestion trop rigide d'un schéma global. Cette architecture repose sur trois types de composantes :

1. les médiateurs, qui encapsulent la représentation des sources de données ;
2. les traducteurs, qui convertissent les requêtes sur les sources de données locales ;
3. les catalogues, qui répertorient toutes les composantes du système réparti et leur localisation.

Un médiateur est un serveur pour les applications ou pour d'autres médiateurs. Pour faciliter l'ajout de sources de données dans un médiateur, Disco modélise les connexions aux sources de données par des objets et supporte les conversions de type (*casting*), c'est-à-dire du niveau local au niveau global. Les médiateurs gèrent également des répertoires de métadonnées et d'index qui sont utilisés pour optimiser l'accès aux

informations. Un médiateur fournit des fonctions de bases de données gérant l'accès concurrent aux métadonnées et intégrant les résultats des requêtes. Le traitement des requêtes dans Disco repose sur des techniques de reformulation et d'optimisation, et prend en considération l'indisponibilité des sources données pendant l'exécution.

Un premier prototype a été conçu au début de l'année 1996 en réutilisant des modules de l'optimiseur Flora. Le langage Flora offre un modèle objets générique et un langage algébrique qui sert de support pour l'optimisation de requêtes objets. Ce prototype peut accéder à diverses sources d'information telles que BibTex, Wais et O2. Par la suite, un second prototype de médiateur a été construit en Java. Il supporte la définition de schéma global, de schéma local et de vue, et peut décomposer les requêtes depuis le niveau global vers le niveau local. Une interface HTML devra permettre l'accès sur le Web. La technologie traducteur est développée par Bull dans le contexte de GIE Dyade et sera intégrée avec un médiateur pour une application de bourses d'affaires électroniques.

Reformulation de requêtes hétérogènes

Un système tel que Disco fournit à l'utilisateur une vision commune des données hétérogènes grâce au modèle de l'ODMG étendu ; ainsi, l'utilisateur peut interroger ces données de manière transparente via un médiateur avec le langage OQL. La traduction de l'information contenue dans des sources de données est réalisée par le biais de règles d'équivalences. Il est cependant nécessaire de maintenir d'autres connaissances sémantiques concernant l'intégrité ou la duplication des données.

Dans ce contexte, le traitement des requêtes hétérogènes, exprimées en OQL, s'effectue en trois étapes essentielles : (1) reformulation de la demande en des requêtes équivalentes portant sur les interfaces des sources de données locales, (2) décomposition des ces demandes en des requêtes locales pouvant être traitées par les traducteurs et (3) consolidation des résultats obtenus. La seconde étape doit faire l'objet d'optimisation pour choisir la meilleure décomposition possible.

L'étape de reformulation est compliquée par la diversité des connaissances sémantiques qui décrivent les données et par la complexité du langage OQL. À partir du module de transformation de requêtes de l'optimiseur Flora, un algorithme de reformulation [Florescu96] a été développé pour exploiter les connaissances exprimées sous forme de règles de réécriture (règles de traduction) avec une technique de *pattern-matching*. L'algorithme est suffisamment générique pour réutiliser les résultats de requêtes précalculées.

Une solution [Florescu et al.96] a été proposée pour exploiter les vues matérialisées, définies par des expressions OQL, et éviter le calcul systématique de certaines expressions pour une requête donnée. Un algorithme polynomial permet d'établir des conditions suffisantes pour décider de l'utilisation d'une vue matérialisée et proposer une réécriture non triviale de la requête.

Exécution de requêtes réparties

L'exécution de requêtes réparties dans un réseau global, tel que l'Internet, cause une difficulté au niveau du temps de réponse de la requête qui peut varier grandement selon la charge du réseau et la disponibilité des sources de données accédées. Pour résoudre ce problème, un algorithme de modification dynamique de plan d'exécution [Amsaleg et al.96b], ou *query scrambling*, a été développé. Cet algorithme modifie les plans d'exécution pendant la résolution de la requête pour compenser les retards imprévus lors de l'accès aux données. Les modifications de plan concernent l'ordonnement des opérations et l'introduction de nouveaux opérateurs. Les résultats de simulation ont démontré que l'algorithme cache effectivement bien les retards de réception des premiers éléments de réponse de la requête.

La technique de *query scrambling* est étendue [Amsaleg et al.96a] pour prendre en considération l'arrivée des données en provenance des sources à un taux imprévu et irrégulier. L'extension principale consiste à matérialiser les données lorsqu'elles arrivent trop vite.

La résolution de requêtes globales dans un système réparti tel que Disco, peut bénéficier naturellement du parallélisme afin de réduire les temps de réponse. En effet, Disco peut être vu comme un système multiprocesseur à mémoire distribuée faiblement couplé. Un modèle d'exécution adaptatif, qui combine la répartition statique des données et l'allocation dynamique des processeurs, a été proposé dans le but de supporter la distributivité de données biaisées qui ont pour effet négatif de réduire l'équilibre de charge. Ce modèle a l'avantage de découpler le degré de parallélisme de la répartition. Le prototype DBS3, le SGBD parallèle réalisé en collaboration avec Bull sur Encore Multimax et KSR1 (72 processeurs), a démontré de bonnes performances au niveau du temps d'exécution et représente un avantage précieux pour le projet Disco.

2.3.2 Discussion sur Rodin

Le mégaprojet Rodin définit plusieurs sujets reliés au domaine des bases de données. La présente discussion se concentre sur la partie en relation avec les problèmes de traitement des ressources informationnelles hétérogènes, c'est-à-dire le projet Disco. Ce projet est semblable à celui réalisé par Waiman Cheung [Cheung et Hsu96], qui a déterminé les paradigmes fondamentaux d'un système de requêtes globales pour bases de métadonnées. L'approche Disco offre une des composantes primordiale pour la création d'un environnement concurrent adaptable (section 6.2.1), c'est-à-dire un système permettant de retrouver l'information et ce, peut importe sa localisation.

Les composantes de l'architecture de Disco, qui sont le médiateur, le traducteur et le catalogue, obligent l'utilisation de formalismes statiques et non génériques. L'ajout d'une nouvelle ressource informationnelle implique un ensemble de mises à jour dans les trois types de composantes. La formule suivante permet de calculer le nombre de mises à jour devant être effectuées lors de l'intégration d'une application : $3 * n$, où 3

est le nombre de composantes dans Disco et n représente le nombre d'applications nécessitant des mises à jour. Si n est grand, le processus d'intégration peut prendre des proportions démesurées, ce qui rend l'approche Disco, pour l'instant, non exploitable dans des environnements organisationnels, puisqu'aucune fonction de prolifération des connaissances (métadonnées) n'a été mise au point.

2.4 TSIMMIS

Les organisations doivent opérer avec de multiples ressources informationnelles disparates. Pour la prise de décision, le comité d'administration a besoin d'informations se trouvant dans ces ressources hétérogènes et distribuées, ce qui cause des difficultés au niveau de la disponibilité et de la consolidation de cette même information.

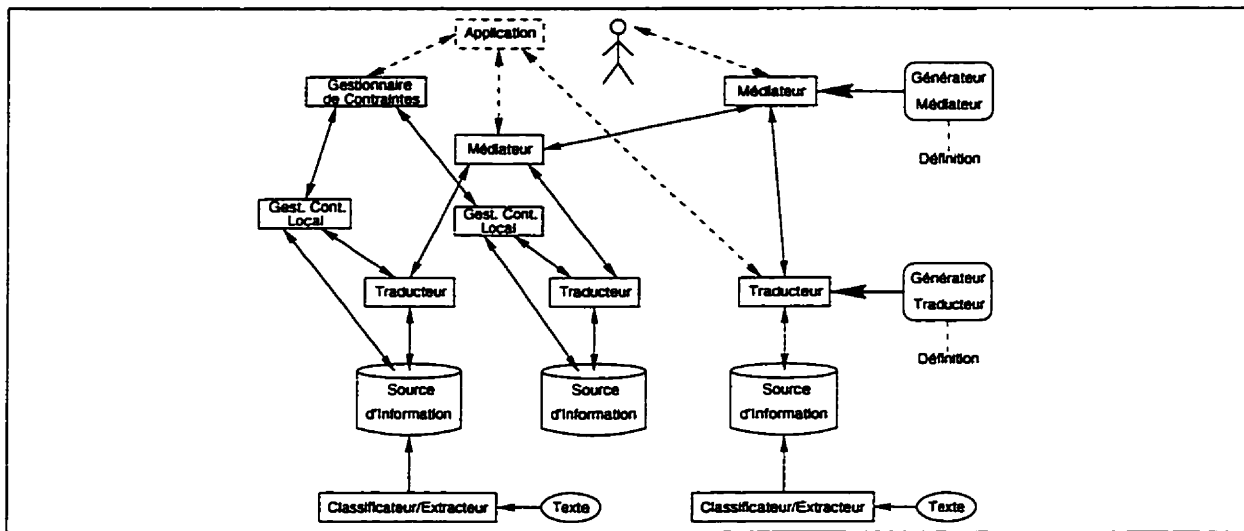


Figure 2.5 : Architecture de TSIMMIS

Le principal objectif du projet TSIMMIS (*The Stanford-IBM Manager of Multiple Information Sources*) [Bergamaschi97, Chawathe et al.94, Gracia-Molina et al.95] est de fournir des outils pour accéder aux diverses sources d'information. Cela est réalisé en créant un système d'exploitation global commun (cf. figure 2.5), ce qui facilite la gestion et l'intégration des systèmes locaux.

2.4.1 Traducteur et modèle commun

La principale fonction d'un traducteur est de convertir, par le biais de prédicats logiques, les données objets du schéma global vers le modèle local de sa source, ou les données objets local sous-jacentes à sa source vers le schéma global. Ces opérations permettent l'exécution de traitements locaux dans les ressources informationnelles d'une entreprise.

L'architecture objets autodescriptive de TSIMMIS, OEM (*Objet Exchange Model*), facilite la gestion de ses données distribuées. Les objets dans OEM possèdent quatre attributs, soit : (1) une étiquette décrivant l'objet, (2) un type, (3) une valeur et (4) un identificateur unique. Par exemple, une entité représentant une température de 80 degrés Fahrenheit pourrait être définie comme suit : <temp-en-Fahrenheit, int, 80>, où la chaîne de caractères "temp-en-Fahrenheit" est une étiquette explicative et compréhensible par les utilisateurs, le type "int" indique une valeur entière, et "80" représente la valeur elle-même. Lors de la définition d'un objet complexe, chacune des composantes de l'objet possède sa propre étiquette ; l'exemple suivant démontre une entité représentant un ensemble de deux températures :

```
<ensemble-de-temperature, set, {cmp1, cmp2}>
    cmp1 : <temp-en-Fahrenheit, int, 80>
    cmp2 : <temp-en-Celsius, int, 20>
```

OEM, malgré son architecture simplifiée, fournit la puissance expressive et la flexibilité nécessaire à l'intégration d'informations réparties dans des sources de données disparates. La sollicitation des objets est effectuée par le biais du langage de requêtes OEM-QL, qui est semblable au SQL standard, mais adapté pour les modèles orientés-objets.

2.4.2 Médiateur

Un médiateur est en quelque sorte un système de raffinement d'information pour une ou plusieurs sources de données. Ce module englobe la connaissance nécessaire au traitement de types spécifiques d'information. Ainsi, lors de la réception d'une requête globale, le médiateur peut réacheminer celle-ci vers les ressources informationnelles locales concernées et ce, après prétraitement si nécessaire (règles de conversion ou d'équivalence). Les médiateurs, qui sont implantés à partir de règles et de modèles, possèdent une structure relativement simpliste. Malgré cette contrainte, les prototypes actuellement implantés fournissent une représentation réelle des organisations.

La génération d'un médiateur est un processus complexe et laborieux ; il est cependant possible d'automatiser certaines fonctionnalités de ce processus. TSIMMIS permet de concevoir des médiateurs à partir d'une description du traitement de l'information qu'ils doivent effectuer. De plus, TSIMMIS fournit un module Traducteur/Générateur qui peut créer des traducteurs OEM en se basant sur les règles de conversion devant être opérées lors du traitement de requêtes globales.

Les concepteurs de l'approche TSIMMIS considèrent qu'un médiateur n'a pas à comprendre toutes les données qu'il manipule, et que l'exploitation d'un schéma global de l'infrastructure informationnelle d'une organisation s'avère inutile pour les traitements répartis. Lorsqu'un médiateur envoie des objets à ses clients, il peut annexer le nom de la ressource informationnelle à l'étiquette afin que le client puisse interpréter correctement les données reçues.

2.4.3 Les interfaces utilisateurs

Les interfaces utilisateurs, qui sont similaires pour les deux modules (traducteur ou médiateur), permettent la saisie de requêtes OEM-QL et l'affichage des résultats. Lors de l'ajout d'une nouvelle source de données, on doit générer son traducteur. Une fois cette opération effectuée, l'utilisateur peut exploiter, via les GUI (*Graphical User Interface*) d'un médiateur, le contenu de cette nouvelle ressource informationnelle et ce, de manière transparente. L'utilisateur peut accéder aux données du système global de l'organisation par le biais d'outils de navigation génériques créés à cette fin. Il est aussi possible d'interagir avec le système à l'aide d'un fureteur tel que Netscape. L'utilisateur définit sa requête sur une page interactive du Web, et par la suite, reçoit ses résultats sous forme de document hypertexte. Le document HTML racine affiche les données objets avec, si nécessaire, des liens hypertextes, permettant ainsi à l'utilisateur de visualiser des portions de la réponse n'apparaissant pas sur le document préliminaire. Cette approche fournit un mécanisme pour l'exploration des sources d'information hétérogènes qui est convivial et basé sur une interface communément utilisée.

2.4.4 Gestion de contraintes

Les transactions d'accès et de contrôle, réalisées dans un réseau à grande échelle, peuvent être effectuées de multiples façons, impliquant ainsi la création et la gestion de contraintes d'intégrité spécifiques aux niveaux global et local et ce, par le biais de modules appropriés. Une contrainte d'intégrité spécifique des exigences sémantiques cohérentes sur l'information emmagasinée.

Le gestionnaire de contraintes, qui est basé sur un formalisme générique de framework, décide de la stratégie de gestion des contraintes non centralisées qu'il exécute, tandis que les gestionnaires de contraintes locales sont responsables de la description et du support des interfaces. Elles spécifient les droits de lecture et d'écriture des attributs d'une source spécifique contrôlés par un gestionnaire de contraintes local.

2.4.5 Classification et extraction

La dernière composante de l'architecture TSIMMIS est le Classificateur/Extracteur. Parfois, certaines sources d'information importantes sont non-structurées. Il est alors impossible de classifier automatiquement les objets de telles sources et d'en extraire les propriétés clés. Le Classificateur/Extracteur effectue cette tâche en se basant sur l'identification de modèles objets simples. L'information recueillie par ce module peut être exportée, par le biais d'un traducteur, si nécessaire, vers le reste du système global, formant ainsi un ensemble de données objets brutes. La composante Classificateur/Extracteur est basée sur le système Rufus développé au centre de recherche IBM à Almaden.

2.4.6 Discussion sur TSIMMIS

Les concepteurs du projet TSIMMIS désirent fournir des méthodes et des outils aux spécialistes et ce, en explorant des technologies facilitant l'intégration de ressources informationnelles hétérogènes réparties. Les efforts de recherche se concentrent sur les points suivants :

- Fournir des techniques d'accès intégrées à de l'information semi-structurée (sans schéma défini), diversifiée, dynamique et hétéroclite.
- Inter-relier le processus de visualisation de l'information et de l'intégration d'applications pour créer des stratégies de consolidation évolutives.
- Automatiser certaines parties du processus d'intégration par le biais de spécifications fonctionnelles pour le générateur de médiateurs.

L'absence de schéma global dans l'architecture agents de TSIMMIS représente une lacune majeure. L'utilisation de la base de métadonnées, en tant qu'agent d'exploitation du système organisationnel, allège de beaucoup les processus de conversion de données, d'intégration d'applications et de gestion des contraintes d'intégrité.

L'utilisation du modèle OEM pour le traitement de l'information s'avère un avantage considérable, puisqu'il : (1) permet la création d'un environnement réaliste d'exploitation de systèmes disparates ; (2) facilite l'intégration d'application et ce. à cause de sa structure simpliste ; (3) peut être, avec un minimum d'effort, adapté à la plupart des protocoles de communication objets.

2.5 CORBA

L'approche naturelle pour intégrer diverses applications dans un environnement à architecture ouverte est de développer un protocole de communication commun et acceptable par l'ensemble des éléments [Babin et Hsu94]. Les concepteurs des différentes techniques d'intégration décrites précédemment suggèrent, pour résoudre ce problème, l'utilisation de l'approche orientée-objets pour la gestion des communications inter-applications et pour l'entreposage de leurs données, d'où l'intérêt de présenter une architecture facilitant l'implantation et l'exploitation d'un environnement d'intégration d'applications supportant cette avenue.

2.5.1 Concepts de la distribution : objet ou composante

Un **objet classique** est un module intelligent qui encapsule du code et des données. L'approche objets fournit beaucoup de services pour la réutilisabilité du code grâce aux concepts de l'héritage, l'encapsulation et de redéfinition de méthodes. Cependant, ces notions sont souvent utilisées au niveau d'un seul programme et seul le langage qui crée ces objets connaît leur existence, ce qui diminue par conséquent les possibilités d'utilisation. À l'opposé, un **objet distribué** est un module intelligent

qui peut vivre à travers le réseau. Il peut être assemblé à l'aide de morceaux de code indépendants. Les objets distribués peuvent être accédés par les utilisateurs distants en invoquant les méthodes appropriées. Lors de la création de serveurs basés sur des objets distribués, il n'est pas nécessaire de spécifier la localisation des objets ni quels systèmes d'exploitation ils adoptent. Un objet a la possibilité d'échanger des messages et de s'adapter dynamiquement.

Une **composante** (*compound*) est un objet qui n'est pas en relation directe avec un programme particulier ou langage de développement/implantation. Elle évolue d'une manière indépendante et peut utiliser l'approche client/serveur pour ses opérations d'interaction. Une composante a les mêmes caractéristiques qu'un objet, puisqu'elle supporte l'héritage, l'encapsulation et le polymorphisme. De plus, elle dispose de caractéristiques additionnelles, qui sont les suivantes : c'est une entité commerciale ; ce n'est pas une application complète ; elle peut être utilisée dans des situations non prévues à l'avance ; elle a une interface bien spécifiée ; elle est interopérable¹ et extensible. Les **super-composantes** (*supercomponents*), un autre type de composantes, sont d'un niveau d'abstraction plus élevé puisqu'elles disposent de caractéristiques intelligentes (*smart*). Ces dernières sont nécessaires pour la création d'objets autonomes, extensibles, couplés légèrement et qui peuvent se déplacer à travers le réseau.

Par définition, les objets distribués sont des composantes et l'infrastructure qui les gère est un cadre d'intégration appelé **framework**. L'importance est de faire collaborer ces composantes en se basant sur des standards, qui fournissent l'ensemble des règles d'engagements de chaque composante envers l'environnement global d'interaction. Dans un cadre d'interaction, une composante se comporte comme un objet d'affaires et modélise les aspects pertinents du domaine d'application.

La Figure 2.6 présente l'évolution des composantes d'une interopérabilité simple — via un bus — vers une collaboration constructive. Les services de niveau système proposent des composantes *supersmarts* alors que les frameworks de niveau application supportent l'échange sémantique d'informations entre les composantes au cours de la collaboration.

L'introduction des concepts de composantes et de frameworks s'inspire de l'analogie avec le domaine électronique. Ainsi, un framework peut être comparé à une plaque métallique où les composantes électroniques (*chips*) viendront se greffer tout en respectant les différentes normes de connexion, par exemple la même polarité. Le fait de positionner les différentes composantes suivant un ordre et un emplacement bien définis permet d'avoir un système fiable qui respecte les différentes règles de conception (cf. Figure 2.7).

¹Interaction des différents constituants d'un environnement, qui est souvent hétérogène et distribué.

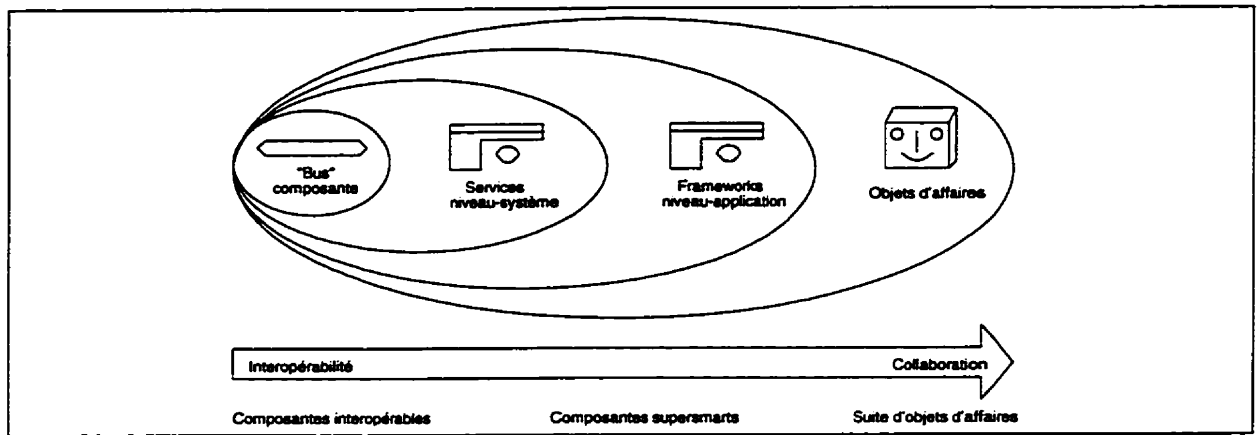


Figure 2.6 : Évolution des composants et les limites de l'infrastructure

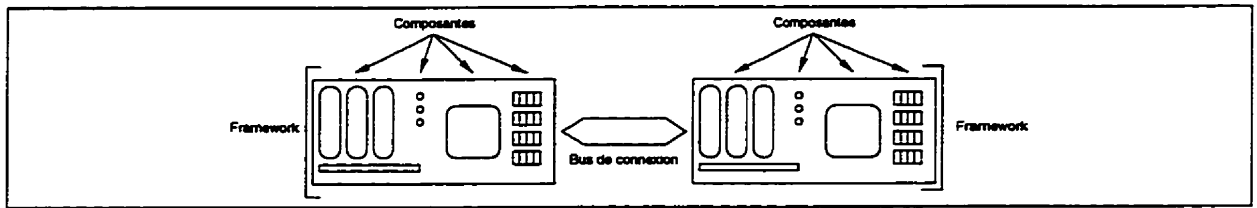


Figure 2.7 : Concepts de l'approche électronique

2.5.2 Architecture

CORBA (*Common Object Request Broker Architecture*) [Mowbray et Zahavi95] est l'un des plus importants projets mis en place par l'industrie du logiciel. Elle est le produit du consortium OMG (*Object Management Group*) [OMTF92], qui est appuyé par plus de 500 compagnies. La seule exception vient de Microsoft avec son propre environnement COM (*Component Object Model*). Le modèle orienté-objets [Manola et Heiler92, Soley93] et l'architecture CORBA proposés par l'OMG visent à réaliser l'interopérabilité entre des applications évoluant dans un environnement réparti. Le modèle établit une base conceptuelle partagée avec, entre autres, un modèle objets commun et un modèle commun d'interaction entre ses composants. Le modèle objets permet une définition détaillée des concepts et offre des méthodes de représentation indépendantes de toute technologie. L'architecture CORBA fournit les mécanismes permettant aux différents sous-systèmes d'effectuer des requêtes et de recevoir des réponses, donc de coopérer et de mettre en place des applications client/serveur orientées-objets.

CORBA est conçu dans le but permettre à des composants intelligentes de se découvrir et d'interagir via un bus d'objet [Watson96]. Cette approche propose plus que l'interopérabilité en spécifiant un ensemble extensible de services. Ceux-là permettent la création et la suppression d'objets, l'accès aux objets par leur nom, leur stockage dans des modules persistants, l'expression de leurs états et la définition de relations ad-hoc entre eux. CORBA offre la possibilité de créer un objet ordinaire et de le rendre

transactionnel, bloquant et persistant. Cela peut se faire grâce à un héritage multiple des services appropriés. La spécification des interfaces des différentes composantes se fait par le langage IDL (*Interface Definition Language*) [OMG94] qui permet de disposer de composantes portables.

Les objets CORBA sont des modules (*bolbs*) intelligents qui peuvent être distribués sur le réseau. Ils sont regroupés comme des composantes, qui assurent aux clients distants un accès à leurs méthodes via des opérations d'invocation. Les procédés utilisés pour créer les serveurs objets sont totalement transparents aux clients. Ces derniers n'ont pas besoin de connaître où l'objet réside ni dans quel système il évolue. Pour l'utilisateur, c'est l'interface d'accès que fournit le serveur objets, qui est important. Cette interface est un contrat ferme entre les clients et les serveurs et est spécifiée à l'aide de IDL d'OMG, qui est un langage purement déclaratif. Il offre des mécanismes qui sont indépendants des interfaces des services et des composantes qui résident sur le bus CORBA et permet aux objets clients et serveurs écrits dans des langages différents d'interagir au niveau du réseau (cf. figure 2.8). IDL est aussi utilisé pour spécifier les attributs des composantes, les classes parents dont elles héritent, les exceptions qu'elles supportent, les types d'événements qu'elles émettent et les méthodes que leurs interfaces supportent, incluant les paramètres d'entrées et de sorties et leurs types de données. La grammaire IDL est un sous-ensemble de C++ avec des alias additionnels pour supporter les aspects de la distribution.

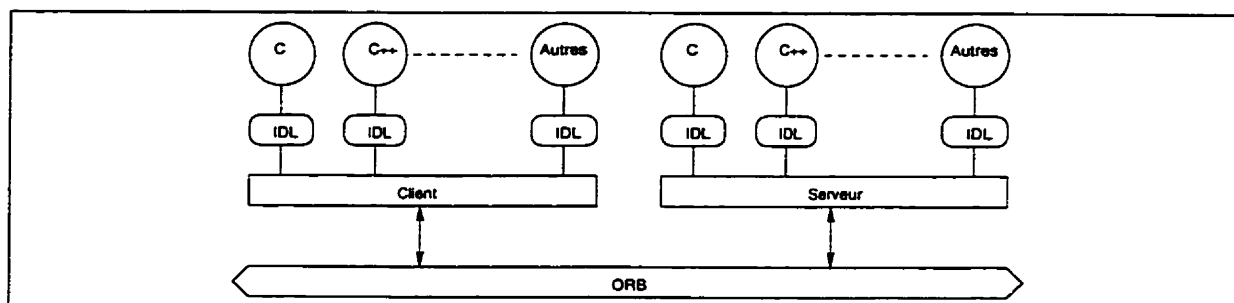


Figure 2.8 : Interopérabilité de composants spécifiés en IDL

La figure 2.9 représente l'architecture de CORBA. Quatre composantes principales s'y trouvent [Henessay et al.96] : (1) l'ORB (*Object Request Broker*) qui définit le bus d'objet, (2) les services objets communs (*Common Object Services*) qui définissent des frameworks de niveau système et qui étendent le bus, (3) les services communs (*Common Facilities*) qui définissent des frameworks de niveau application (services verticaux et horizontaux) et (4) les objets applications (*Application Objects*) qui sont les consommateurs de l'infrastructure CORBA.

2.5.3 CORBA et les métadonnées

Le concept de métadonnées est l'élément qui permet de créer des systèmes client/serveur agiles qui sont auto-descriptifs, dynamiques et reconfigurables. Ils permettent aux composantes de se découvrir au moment de l'exécution en leur fournis-

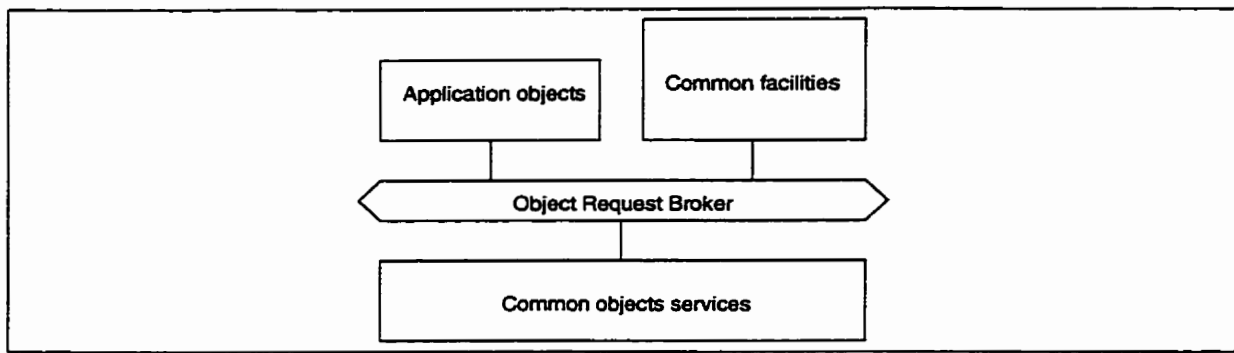


Figure 2.9 : Architecture de CORBA

sant les informations nécessaires au moment de l'interopérabilité. La caractéristique majeure d'un système agile par rapport à un système client/serveur traditionnel est sa capacité d'utiliser des **métadonnées** pour une description consistante de la disponibilité des services, des composantes et des données. Les métadonnées assurent le développement de composantes indépendantes malgré l'objectif de coordination visé initialement. CORBA est considéré comme un système agile et offre la possibilité de concevoir des composantes auto-descriptives. Le langage des métadonnées est IDL alors que le répertoire des métadonnées est *l'Interface Repository*.

L'Interface Repository de CORBA est une base de données qui renferme les définitions des objets. CORBA s'intéresse à cet aspect pour vérifier comment l'information est organisée et retrouvée au sein de cette base. En réalité, les définitions des objets représentent les instances des classes. La base de données obtenue est orientée-objets, hautement flexible et permet de garder trace des différentes classes d'objets utilisées. Un ORB a besoin de comprendre la définition des objets qu'il utilise. Une façon d'avoir ces définitions est d'intégrer l'information dans les *stubs* routines, alors que l'autre manière est d'accéder dynamiquement à *l'Interface Repository*. L'ORB a besoin de ces informations pour les raisons suivantes : (1) fournir un contrôle de types pour les signatures des méthodes, (2) aider les ORB à se connecter entre eux, (3) fournir une information sur les métadonnées aux clients et aux outils, et (4) fournir des objets auto-descriptifs. *L'Interface Repository* est implanté comme un ensemble d'objets qui représentent les informations disponibles. CORBA regroupe les métadonnées en des modules qui représentent les **espaces de nomination** ; les noms des objets sont uniques à l'intérieur d'un module. L'ORB propose des mécanismes standards pour la génération et la gestion des métadonnées. Il est également possible de générer des métadonnées qui décrivent les composantes et leurs relations en utilisant IDL. Ces métadonnées sont stockées dans *l'Interface Repository* qui joue et supporte respectivement le rôle et les opérations d'une base de données traditionnelle. Les clients peuvent questionner l'interface afin de découvrir quelles sont les informations disponibles et comment les exploiter.

2.5.4 CORBA et les services offerts

Nomination, événements, cycle de vie et négociation

Les services nomination (*naming*), événements (*events*), cycle de vie (*life cycle*) et négociation (*trader*) sont considérés comme des services de base. Quand les objets s'exécutent pour la première fois, ils ont besoin de trouver les autres objets qui exploitent le même ORB. Ainsi, le service de nomination permet de retrouver les objets par leur nom. Le service de négociation annonce les services de l'objet en facilitant leur localisation. Le service cycle de vie considère tous les aspects existentiels, incluant la création, la suppression et le remplacement d'objets. Enfin, le service événements supporte les interactions asynchrones entre des objets anonymes.

Transactions et accès concurrentiel

Les transactions sont essentielles pour la construction d'applications efficaces. Le service des transactions objets (*Object Transaction Service*) définit des interfaces en IDL, qui permettent à divers objets distribués sur de multiples ORB de participer à des transactions atomiques. Le service contrôle des accès concurrentiels (*Concurrency Control Service*), donne aux objets la possibilité de coordonner leurs accès aux ressources partagées en utilisant le mécanisme de blocage (*lock*). Un objet doit obtenir un blocage approprié de ce service avant d'accéder à une ressource partagée. Chaque blocage est associé à une seule ressource et à un seul client. Le service définit aussi plusieurs modes de blocage qui correspondent aux différentes catégories d'accès et aux multiples niveaux de granularité.

Persistance

La majorité des objets distribués sont persistants ; cela signifie qu'ils gardent longtemps leur état après la fin de l'exécution du programme. Pour être persistant, l'état d'un objet doit être stocké d'une manière non volatile. Le service des objets persistants (*Persistent Object Service*) permet aux objets d'être persistants par rapport à l'application créatrice ou aux clients utilisateurs. La durée de vie d'un objet peut être relativement courte ou indéfinie. Ainsi, ce service permet à l'état d'un objet d'être sauvegardé dans un support persistant. Parfois, les clients d'un objet ont besoin de contrôler ou d'assister la gestion de la persistance. Le service peut accommoder différents niveaux d'implication des clients. Le cas extrême, c'est la transparence totale aux applications clientes, qui ignorent complètement le mécanisme de persistance. Un autre cas extrême, les applications clientes peuvent utiliser des protocoles spécifiques au stockage, qui leur donnent des détails sur tout le mécanisme de persistance. Ainsi, le client décide du choix du niveau de persistance dans la gestion de données. Le meilleur support physique pour l'entreposage des objets de CORBA est l'utilisation d'une base de données objets.

Les SGBD objets fournissent des architectures client/serveur totalement différents des SGBD relationnels. Un système objets intègre les capacités de gestion de données

avec ceux des langages de programmation orientés-objets, fournit un stockage persistant pour les objets, supporte les accès concurrents aux objets, fournit des mécanismes de blocage et de protection des transactions, protège les objets stockés de tous types de problèmes et prend en considération les tâches traditionnelles, comme le *backup* et le *restore*. Les avantages qu'une gestion basée sur les objets offre : la liberté de créer de nouveaux types d'informations, la rapidité des accès, les vues flexibles sur les structures composées, l'intégration étroite aux langages OO, la personnalisation des structures d'information en utilisant l'héritage multiple, etc. En ce qui concerne CORBA, les composantes d'un système de gestion sont : un langage de définition, un langage de requêtes des objets et des agrégations (*bindings*) avec les autres langages tels que C++ ou Smalltalk.

Requêtes, relations et gestion du système

Le service requêtes aux objets (*Object Request Service*) de CORBA permet de trouver des objets dont les valeurs des attributs coïncident avec ceux précisés au niveau du critère de recherche. Le service requête peut exécuter directement une requête ou la déléguer à un autre évaluateur de requêtes. Il combine les résultats des requêtes en provenance de tous les évaluateurs participants et retourne les résultats finaux au demandeur. Cela signifie qu'il est possible de coordonner plusieurs fédérations. Une requête au niveau de CORBA offre, en plus de trouver des objets, la possibilité de les manipuler via des opérations de création, de suppression et de modification.

2.5.5 Discussion sur CORBA

La prochaine génération des systèmes client/serveur sera inévitablement conçue en utilisant des objets distribués. Cela est essentiellement motivé par les points suivants : (1) l'augmentation rapide du nombre de réseaux à grande échelle, ce qui amène une diminution de la charge des bandes passantes ; et (2) l'apparition de nouveaux environnements supportant le multitâche. La mise en place de systèmes basés sur l'approche client/serveur est intéressante pour diverses raisons :

1. **Variété des transactions** : en plus des transactions ordinaires à supporter, les environnements ont besoin de transactions qui peuvent consulter de multiples serveurs, qui ont une longue durée de vie et qui sont sécuritaires.
2. **Nouveauté des concepts** : avec l'émergence des technologies de l'information, de nouveaux concepts sont suggérés afin de supporter les tendances actuelles en matière de développement, comme par exemple le concept d'agent-logiciel. Les environnements doivent alors supporter ces nouveaux concepts.
3. **Variété des données** : en plus de pouvoir gérer des données traditionnelles, il faut de nouveaux types de données telles que les composantes multi-média. Ces composantes doivent être déplacées, stockées, vues et éditées n'importe où à travers le monde. De plus, chaque noeud du réseau doit pouvoir fournir ce type de composantes et les supporter.

4. **Système intelligent** : un environnement distribué doit donner l'illusion d'un système unique, c'est-à-dire transparent, qui regroupe des millions de machines client/serveur hybrides. Cette transparence doit exister afin de faire apparaître tous les serveurs du réseau comme un seul système. Les utilisateurs et les programmes doivent être capables d'interagir et de se brancher dynamiquement au réseau.

À l'heure actuelle, quatre paradigmes sont impliqués dans le développement d'applications client/serveur : (1) les bases de données, (2) la surveillance des processus de traitement, (3) les logiciels de productivité avec leurs flux de travail et (4) les objets distribués. Ces paradigmes peuvent être combinés et fournir un système client/serveur. Avec les nouvelles tendances de développement et le chevauchement des domaines, l'approche orientée-objets additionnée du concept de distribution se distingue des autres approches. En utilisant des modules et des infrastructures adaptés, les objets peuvent participer à la division des applications client/serveur monolithiques en plusieurs composantes. Chaque composante a son mode de gestion, interagit avec les autres et navigue à travers le réseau. Les objets comparés à des composantes permettent ainsi de créer un nouveau type de système client/serveur et de combiner les données et l'intelligence au cours des opérations de traitement. Aucune distinction n'est à faire entre les composantes clientes et les composantes serveurs. L'affectation des rôles est dynamique.

La technologie des objets distribués est extrêmement intéressante pour la création de systèmes client/serveur flexibles. Les données et les règles de fonctionnement sont encapsulées ensemble à l'intérieur d'objets. Ces derniers sont localisés à travers le réseau. Les objets distribués ont le potentiel de permettre à diverses composantes d'interagir, de s'exécuter sur différentes plates-formes, de coexister avec les systèmes hérités (*legacy systems*), de naviguer, de s'auto-gérer et d'être utilisés par l'approche *plug and play*. Pour que les objets distribués puissent assurer les fonctionnalités d'un système client/serveur, ils ont besoin d'être groupés suivant un cadre d'interaction, qui leur permet d'évoluer selon des suites ou des enchaînements cohérents. L'interaction entre les différents objets se fait grâce à un bus d'échange, qui assure l'invocation des ressources externes d'une manière dynamique ou statique.

En tant que technologie récente, l'architecture CORBA remporte beaucoup de succès et de plus en plus de constructeurs tiennent compte des spécifications de l'OMG lors du développement de nouveaux produits. Les faiblesses en relation avec l'héritage multiple et le polymorphisme ont été supprimées en grande partie.

Chapitre 3

Méthode de modélisation TSER

Les métadonnées permettent de décrire avec exactitude les différentes caractéristiques des systèmes hétérogènes d'une entreprise. La saisie de ces caractéristiques peut être effectuée par le biais de la modélisation de ces systèmes. Dans ce chapitre, nous présentons l'approche de modélisation *Two-Stages Entity-Relationship* (TSER). Cette méthode a été utilisée pour définir la structure de la base de métadonnées. Par surcroît, TSER s'avère un outil très intéressant pour faciliter la création d'un environnement d'intégration d'applications.

3.1 Métamodélisation

Si un métamodèle parfait existait, ce qui est peu probable, il devrait évidemment supporter tous les paradigmes concevables dans le présent et dans le futur. Une attente un peu plus réaliste serait une méthode qui pourrait satisfaire plusieurs exigences dérivées de la littérature et de la vision suivante [Hsu96] :

1. l'habilité de représenter les classes majeures de connaissances concernant les processus industriels reliés au fonctionnement, au contrôle et au domaine décisionnel;
2. l'inclusion de tout modèle pertinent d'analyse multi-niveaux et de conception de cycles de vie;
3. l'architecture isomorphe (*neutral accommodation*) des vues sur des données hétérogènes (surtout les relations et la hiérarchie des objets);
4. la représentation unifiée de l'ensemble de ces métadonnées.

Les flux dynamiques caractérisés par les graphes non-orientés, tels que les flux de données et les flux de contrôles, sont des éléments primordiaux en relation avec la représentation des connaissances. Il n'existe pas de méthode d'encapsulation implicite capable de représenter tous les types de flux existants. Les méthodes d'encapsulation sont surtout orientées vers la localisation des déclencheurs (*triggers*), la définition des contraintes d'intégrités et la description logique d'applications similaires. Une combinaison des méthodes d'encapsulation et des techniques de graphes non-orientés utilisant

certaines primitives génériques suffisent à ce besoin.

Un avantage important pour la conceptualisation de systèmes est l'isomorphisme, qui est l'objectif cherchant à combiner les modèles multi-niveaux. L'évolution de la représentation d'un modèle au niveau X du cycle de vie vers le niveau suivant doit être complète et exacte. L'isomorphisme, un corollaire utile, est un processus réversible de modélisation des paradigmes informationnels résultant de l'analyse et de la conception initiales. Ainsi, ces paradigmes peuvent être récupérés de la base de métadonnées et ce, même après le développement initial des systèmes.

En ce qui concerne la représentation des données, un besoin évident est d'englober l'hétérogénéité telle que démontrée par l'approche relationnelle et les paradigmes orientés-objets, qui sont deux alternatives de modélisation récentes. Chacune de ces approches a révélé certains éléments uniques et fondamentaux concernant la modélisation des données, et ceux-ci doivent être considérés par tous modèles génériques. L'approche relationnelle a établi la théorie des dépendances pour l'intégration des données (ablation des redondances) et le principe de séparer les vues des applications et des usagers de la structure commune des données. Le paradigme orienté-objets soutient l'abstraction hiérarchique des données et l'intégration de certaines classes de transactions avec la modélisation des données.

Présentement, il n'existe pas de méthode idéale qui puisse satisfaire toutes les exigences spécifiées ci-haut. Cependant, la métamodélisation vise directement les **Sujets** des bases de données et leurs **Contextes**, facilitant ainsi la définition des métadonnées, ce qui nous dirige vers une méthode de représentation particulière.

L'approche TSER pour la métamodélisation des informations est conçue pour représenter des systèmes complexes. Elle fournit une méthode de représentation (algorithmes et structures de modélisation) qui inclut la connaissance contextuelle du système d'information et de ses composantes [Hsu et al.93]. La méthode TSER a été initialement développée pour intégrer certaines tâches de l'analyse de systèmes avec la conception des bases de données dans une entreprise complexe et par la suite étendue pour inclure la représentation des connaissances. Elle se compose de deux niveaux de modélisation, un pour les abstractions sémantiques, c'est-à-dire le modèle fonctionnel et un pour la représentation normalisée des données et des règles de production, c'est-à-dire le modèle structurel. Le modèle fonctionnel est semblable au diagramme de flux de données (*Data Flow Diagram*; DFD) et le modèle structurel est similaire au modèle entité/relation (cf. figure 3.1). À partir de ces deux modèles, on peut générer le schéma d'une base de données.

La méthode TSER supporte le développement de systèmes avec l'approche ascendante (*bottom-up*) ou descendante (*top-down*). Elle facilite également le processus de rétro-ingénierie des applications déjà existantes. Dans TSER, il existe des algorithmes rigoureux dans TSER permettant la traduction du modèle fonctionnel vers le modèle structurel. Ces algorithmes assurent que la structure résultante est au minimum à la

troisième forme normale (3FN). Les algorithmes TSER intègrent aussi les vues ; cela permet la consolidation systématique de n'importe quel modèle de données. Les contraintes d'intégrité construites à l'aide des concepts TSER sont utilisées pour faciliter la gestion et le contrôle de la base de métadonnées. TSER est également caractérisée par un outil CASE, *Information Base Modeling System* (IBMS) [Hsu et al.92a], qui assiste les utilisateurs dans la conception du schéma de l'entreprise, dans notre cas, la création du schéma de la base de métadonnées.

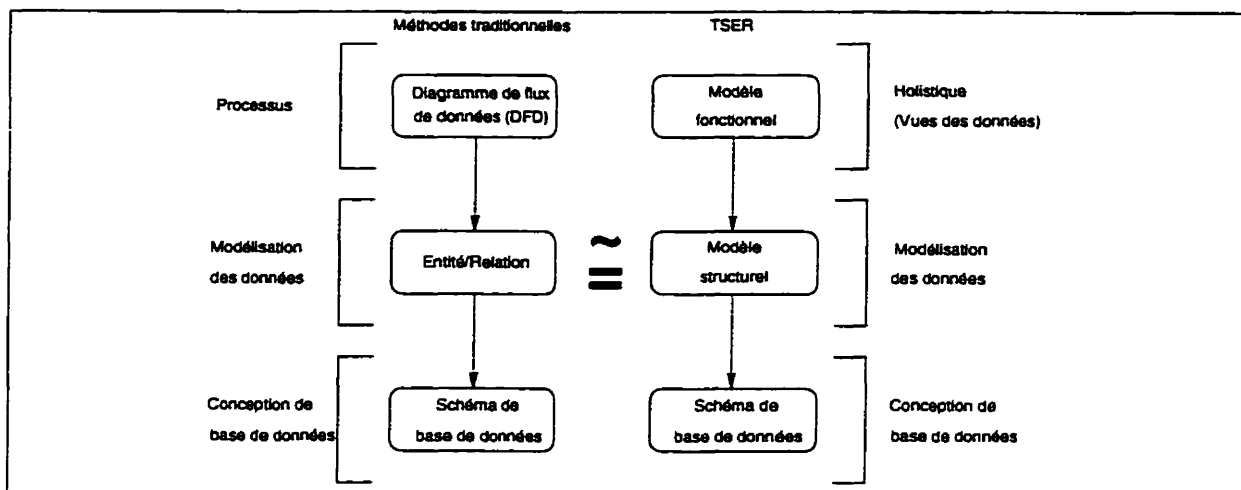


Figure 3.1 : Comparaison entre TSER et les méthodes traditionnelles

3.2 Modèle fonctionnel

Le modèle fonctionnel utilise des concepts de niveau sémantique orientés-utilisateur pour la représentation des processus et la hiérarchisation des objets. Ces concepts sont utilisés pour l'analyse de systèmes et pour déterminer les informations nécessaires à la conception. Il existe deux concepts : (1) le **Sujet** et (2) le **Contexte** (cf. figure 3.2).

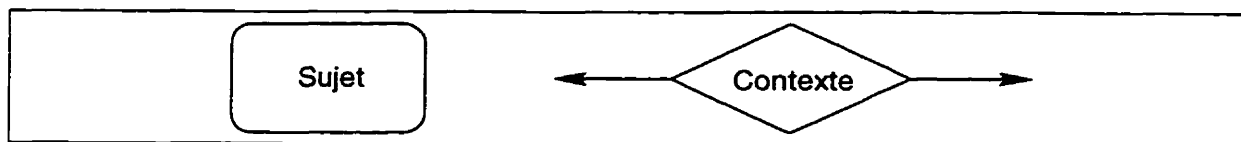


Figure 3.2 : Concepts du modèle fonctionnel de TSER

Sujet :

- **Description :**

représente une unité fonctionnelle ou un regroupement de données (ex. : vues utilisateurs, documents et applications) décrivant les ressources informationnelles de l'entreprise en excluant les problèmes liés aux structures de données, et est analogue au cadre, à l'objet ou à l'activité.

- **Primitives :**

contient des attributs, des dépendances fonctionnelles relatives à ces attributs, des règles *Intra-Sujet* (déclencheurs, événement et définition dynamique des attributs se référant à un seul sujet), et les associations hiérarchiques (sujets généralisés et globaux). Un sujet (sous-sujet) peut être associé avec un ou plusieurs sujets (super-sujet) dans la hiérarchie.

Contexte :

- **Description :**

représente les interactions entre les **Sujets** et le contrôle des connaissances, telles que les règles d'affaires et les procédures opérationnelles, et est analogue au processus logique.

- **Primitives :**

contient les règles *Inter-Sujet* caractérisées par la référence aux attributs appartenant à des sujets multiples ; inclut la direction des flux pour la logique (décision et contrôle) et les données (communication, etc.).

3.3 Modèle structurel

Le modèle structurel permet, à partir du modèle fonctionnel, d'obtenir une représentation normalisée des données sémantiques et des règles de production, facilitant ainsi la conception du schéma d'une base de données. Il existe quatre concepts de base : (1) l'Entité Opérationnelle (OE), (2) la Relation Plurale (PR), (3) la Relation Fonctionnelle (FR) et (4) la Relation Obligatoire (MR) (cf. figure 3.3).

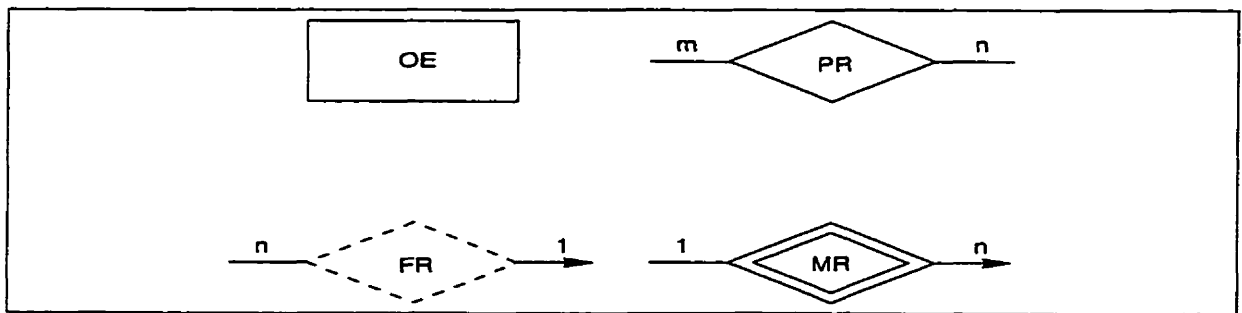


Figure 3.3 : Concepts du modèle structurel de TSER

Entité Opérationnelle :

- **Description :**

notion concernant l'unité logique du niveau le plus bas de système de base de données (ex. : un schéma en 3FN) ; elle est identifiée par une clé primaire singulière, des clés équivalentes (optionnelles) et des attributs non-clés, rendant ainsi son identification unique et indépendante.

- **Intégrité relationnelle :**
aucun attribut clé primaire ne peut prendre la valeur nulle et sa valeur doit toujours être unique (aucun duplicata).

Relation Plurale :

- **Description :**
association d'entités caractérisées par une composition de clés primaires. Elle signifie une relation **plusieurs à plusieurs** et représente une association indépendante.
- **Intégrité Associative :**
chacun des attributs référés par la clé primaire doit être une clé primaire d'une EO et ses valeurs dans la relation plurale doivent être équivalentes à celle de l'entité correspondante.

Relation Fonctionnelle :

- **Description :**
association de **plusieurs à un**. Elle signifie des relations de composition. La relation fonctionnelle représente une contrainte d'intégrité référentielle sous-entendant l'existence d'une clé étrangère. Le sens de la flèche pointe du déterminant au déterminé ; les deux sont liés à des entités opérationnelles ou à des relations plures (caractéristique propre à TSER). La clé primaire du déterminé est considérée comme une clé étrangère dans le déterminant.
- **Intégrité Référentielle :**
la valeur de chacune des clés étrangères dans le déterminant doit, soit être nulle, soit être identique à la valeur d'une clé primaire dans le déterminé.

Relation Obligatoire :

- **Description :**
association fixe de **un à plusieurs**. Elle peut être associée à une relation d'héritage. Une relation obligatoire représente une contrainte de dépendance existentielle. Le sens de la flèche pointe du possédant au possédé : les deux sont liés à des entités opérationnelles ou à des relations plures. La clé primaire du possédant est considérée comme une clé étrangère dans le possédé.
- **Intégrité Existentielle :**
lorsque l'instance propriétaire (possédant) est détruite, toutes les instances associées au propriétaire doivent être détruites ; il y a une clé étrangère impliquée dans l'instance du possédé dont la valeur doit être identique à celle de la clé primaire du propriétaire.

Les relations fonctionnelles et obligatoires peuvent être associées à des relations plures, par opposition au modèle Entité/Relation [Chen76]. Les relations plures peuvent contenir des attributs qui leur sont propres, incluant des clés étrangères. Il est

donc normal qu'une relation plurale puisse être impliquée dans une relation fonctionnelle ou obligatoire.

Quoique TSER possède sa propre méthodologie de conception, il est néanmoins compatible avec trois perspectives de modélisation très répandues, à savoir :

1. Entité/Relation

le modèle informationnel se compose principalement d'entités et des interactions entre ces entités (relations). La différence majeure entre le modèle entité/relation traditionnel et TSER est qu'il n'est plus nécessaire de cataloguer les concepts de représentation en entités ou en relations lors de la modélisation sémantique d'une application. IBMS (voir section 3.5) permet de déterminer ces définitions en utilisant tous les concepts sémantiques fournis et facilite la génération d'entités et de relations structurelles normalisées tout en respectant la théorie de dépendance. Les relations sont représentées en utilisant des règles de production. Les règles internes appartenant à une entité sont directement intégrées dans celle-ci, tandis que les règles externes, c'est-à-dire celles impliquant plusieurs entités, sont regroupées de manière à préserver leur autonomie. Le **sujet** représente les entités sémantiques tandis que le **contexte** conceptualise les règles externes.

2. Orienté-Objets

le modèle informationnel se compose principalement d'objets et de comportements. TSER supporte très bien ce paradigme et offre en plus deux caractéristiques additionnelles. Premièrement, on peut séparer les comportements externes, tels que les flux de contrôle de l'information et d'autres connaissances contextuelles inter-reliant différents objets, permettant ainsi une représentation globale de ces comportements. Deuxièmement, on peut obtenir des objets normalisés, ce qui facilite la création d'un schéma cohérent pour une base de données. Le **sujet** représente les objets tandis que le **contexte** conceptualise les comportements externes. Les comportements internes sont simplement intégrés dans les objets.

3. Processus-Activités

le modèle informationnel se compose principalement d'activités et de flux de contrôle. En comparaison avec les techniques de modélisation sémantique, telles que les DFD, TSER permet de définir les activités en associant les processus avec leurs données correspondantes et ce, grâce aux **sujets**, tandis que les connaissances contextuelles représentées par les flux de contrôle entre les activités sont modélisées par le biais des **contextes**.

En résumé, la modélisation d'un système avec la méthode TSER s'effectue en trois étapes (cf. figure 3.4). Premièrement, on crée un modèle fonctionnel pour chacune des applications ; deuxièmement, chacun de ces modèles fonctionnels est transformé en son modèle structurel correspondant ; troisièmement, les divers modèles structurels sont consolidés en un modèle structurel global en utilisant la théorie de dépendance. Le schéma conceptuel d'une application est basé sur la logique générique de TSER

et contient les quatre types de vues, soit (1) la vue application, (2) la vue fonctionnelle, (3) la vue structurelle et (4) la vue ressource. Ce schéma est également fondé sur les concepts de modélisation de TSER, qui sont les métaentités et les métarelations (plurales, fonctionnelles et obligatoires). La modélisation est générique, à l'exception des ressources logicielles et matérielles qui sont extensibles, dépendant de l'entreprise en question. Une fois le modèle structurel global créé, il sera stable, et l'addition ou la suppression de modèles deviendra une transaction de métadonnées en insérant ou enlevant des informations dans la base de métadonnées.

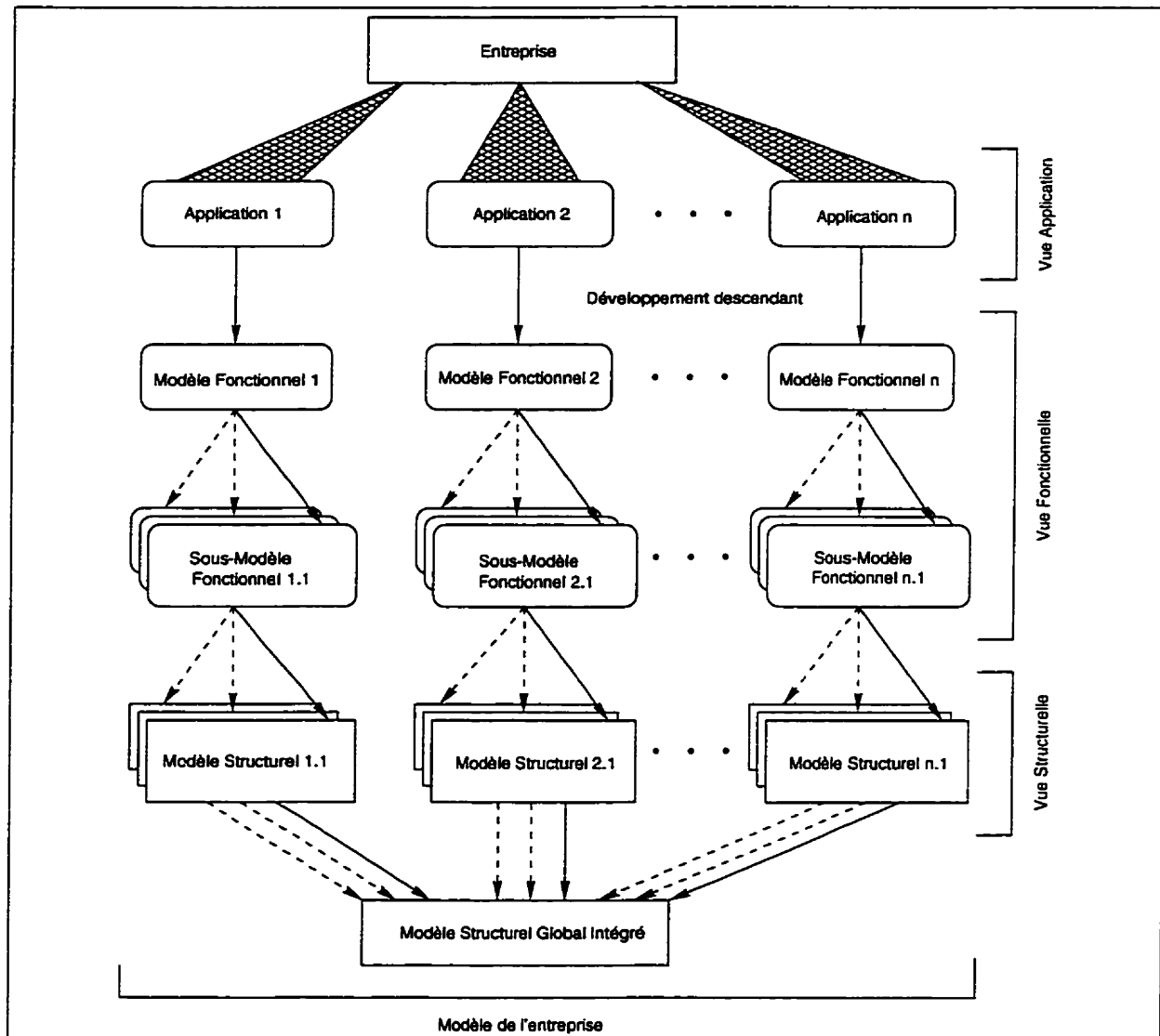


Figure 3.4 : Structure d'un métamodèle

3.4 Algorithmes de transformation

L'approche TSER est supportée par deux types d'algorithmes. Le premier type dérive une structure normalisée (modèle structurel) à partir du modèle fonctionnel. Le deuxième type utilise les structures du schéma obtenu afin de générer le schéma initial des bases de données. Sommairement, le passage du modèle fonctionnel au modèle structurel se fait en trois étapes [Hsu et al.88] :

1. **Décomposition :**
Elle crée des sous-modèles structurels pour chaque sujet de la hiérarchie et analyse ses cardinalités.
2. **Normalisation :**
Elle améliore et simplifie les structures de données à l'intérieur de chaque sous-modèle en se basant sur la théorie des dépendances, assurant ainsi la génération d'un schéma respectant la troisième forme normale.
3. **Consolidation :**
Elle lie l'ensemble des sous-modèles créés pour produire un modèle structurel correspondant au modèle fonctionnel en entrée.

Ces algorithmes génèrent et décomposent des vues, produisent des relations ou des structures de données et déterminent les contraintes d'intégrités pour la conception de schémas [Hsu et al.92a, Hsu et al.93].

3.5 IBMS

IBMS est un logiciel d'aide au développement de systèmes (outil CASE) facilitant la modélisation des applications qui doivent être intégrées dans la base de métadonnées. La méthode de modélisation supportée par IBMS est TSER. Cette outil de modélisation peut être téléchargé à partir de la page Web "Enterprise Integration Modeling" à l'adresse <http://viu.eng.rpi.edu/>.

Ce logiciel facilite la tâche de conceptualisation en permettant :

- la définition du modèle fonctionnel d'une application ;
- la transformation du modèle fonctionnel en son modèle structurel correspondant qui respecte la troisième forme normale.
- la génération du schéma pour la base de données (ORACLE, dBase, Ingres, Rdb, Access, EXPRESS) qui est nécessaire à l'exploitation de l'application en traitement ; cette étape est réalisée à partir des modèles fonctionnel et structurel de l'application ;
- la production de rapports complets détaillant les définitions, les contraintes d'intégrité et les connaissances en relation avec une application ;

- la création d'un fichier d'importation correspondant aux métainformations en relation avec le modèle fonctionnel et structurel d'une application.

IBMS est exploitable dans les environnements supportant les applications Windows et sur le système d'exploitation AIX de IBM. Cet outil est relativement complet. Cependant, un problème majeur demeure ; les métadonnées fournies ne sont pas suffisamment explicites pour assurer une intégration cohérente dans la base de métadonnées. Il n'existe pas de spécifications fonctionnelles dans IBMS permettant la définition des informations minimales d'une application, des ressources logicielles et matérielles.

Chapitre 4

Schéma de l'entreprise

L'utilisation de systèmes répartis dans une entreprise a pour principal objectif le partage de ressources informationnelles, de ressources logicielles et de ressources matérielles [Tanenbaum96]. L'intégration de ces différents sous-systèmes représente un défi de taille et ce, principalement pour trois raisons spécifiques. Premièrement, les structures de données utilisées dans ces sous-systèmes peuvent être complexes et sont souvent incompatibles les unes avec les autres. Deuxièmement, plusieurs tâches requièrent un volume considérable de ressources computationnelles (traitement des données, communications réseaux et autres). Troisièmement, avec l'expansion des technologies de l'information, les utilisateurs requièrent des systèmes plus conviviaux et des accès aux ressources informationnelles en temps réels. La métamodélisation des applications hétérogènes de l'entreprise facilite l'intégration de celles-ci. Cependant, on doit définir un schéma global pouvant accueillir les métadonnées décrivant ces applications. Dans ce chapitre, nous présentons le schéma de la base de métadonnées, le dictionnaire global des ressources informationnelles (*Global Information Resources Dictionary*; GIRD), ainsi que les différents sous-modèles qui le composent.

4.1 Principes de base du GIRD

L'architecture informationnelle de l'entreprise ne peut reposer sur un contrôleur global gérant les différents systèmes distribués de celle-ci. Pour des raisons de performance, d'ordre pratique et théorique, l'architecture doit posséder suffisamment de connaissances pour supporter et gérer les technologies de l'information présentes dans l'entreprise. Cette même connaissance doit être distribuée selon les besoins et imbriquée dans cette architecture de manière à permettre aux utilisateurs d'y référer facilement. Un outil de gestion globale de l'information, des processus de support et des interfaces utilisateurs adaptées, peut être exploité pour la manipulation de cette connaissance : cette connaissance est représentée par les **métadonnées de l'entreprise**. La notion traditionnelle de métadonnées peut être définie comme étant **des données décrivant des données**. Cette notion se divise en quatre catégories de métadonnées, soit [Hsu96] :

1. modèle global des données ;
2. connaissance contextuelle et modèle de traitement ;

3. modèle des ressources logicielles, matérielles et réseaux ;
4. information concernant les utilisateurs et les modèles organisationnels.

Les quatre types de métadonnées sont intégrés dans la base de métadonnées, qui est elle-même construite comme étant une base de connaissances indépendante, permettant ainsi le support de l'architecture organique du système d'information de l'entreprise. Par conséquent, la base de métadonnées fait appel à un schéma pour la représentation des ces types de métadonnées, le GIRD [Bouziane91]. Les concepts génériques de la base de métadonnées sont personnalisés et fournissent un mode de fonctionnement unique et standardisé.

Le GIRD est similaire aux structures de dictionnaires connues, telles que IRDS (*Information Resources Dictionary Systems*) [ANSI89]. Cependant, sa structure logique, permet d'obtenir une représentation unifiée des métadonnées et une vue conceptuelle globale des ressources informationnelles présentes dans l'entreprise. Grâce à la méthodologie TSER (voir chapitre 3), il est possible de modéliser les applications déjà existantes ou les applications devant être insérées dans le modèle global de l'entreprise. Ainsi, chacun des systèmes est visualisé selon deux niveaux, le niveau fonctionnel et le niveau structurel. Ces modèles représentent l'ensemble du contenu de la base de métadonnées. La méthodologie TSER est utilisée pour abstraire le contenu de la base de métadonnées dans un schéma de métadonnées générique, le schéma GIRD.

Le modèle fonctionnel du GIRD est défini en terme de sujets et de contextes, et ils décrivent les catégories de métadonnées et les caractéristiques opérationnelles de la base de métadonnées. Le modèle structurel du GIRD (voir section 4.3) contient un ensemble d'entités et différents types de relations permettant de spécifier le schéma conceptuel sous-jacent à la base de métadonnées. De façon similaire, mais au niveau des applications, le modèle informationnel de l'entreprise contient des sujets et des contextes dans son abstraction fonctionnelle et un ensemble d'entités et de relations dans son abstraction structurelle. De cette manière, les mêmes concepts sont utilisés pour modéliser le GIRD et le modèle informationnel de l'entreprise.

L'utilisation des deux niveaux d'abstraction dans TSER permet de déterminer cinq couches constituant une structure définitive (cf. figure 4.1). Les concepts TSER sont utilisés récursivement à chaque couche pour archiver les types et l'auto-description nécessaires à la gestion des métadonnées. Fondamentalement, la première et la plus haute couche est composée des six primitives génériques pourvues par TSER. Elles sont les **sujets** et les **contextes** au niveau fonctionnel, et les **entités** et les trois **classes de relations** au niveau structurel.

Le niveau sémantique du GIRD (*Global Information Resources Dictionary/Semantic Entity Relationship*; GIRD/SER) pourvoit à un haut-niveau de représentation des classes fonctionnelles de métadonnées, qui sont définies en utilisant les deux primitives du niveau fonctionnel. Ces classes sont ensuite transformées en une série de concepts qui sont représentés par les entités et les différentes primitives

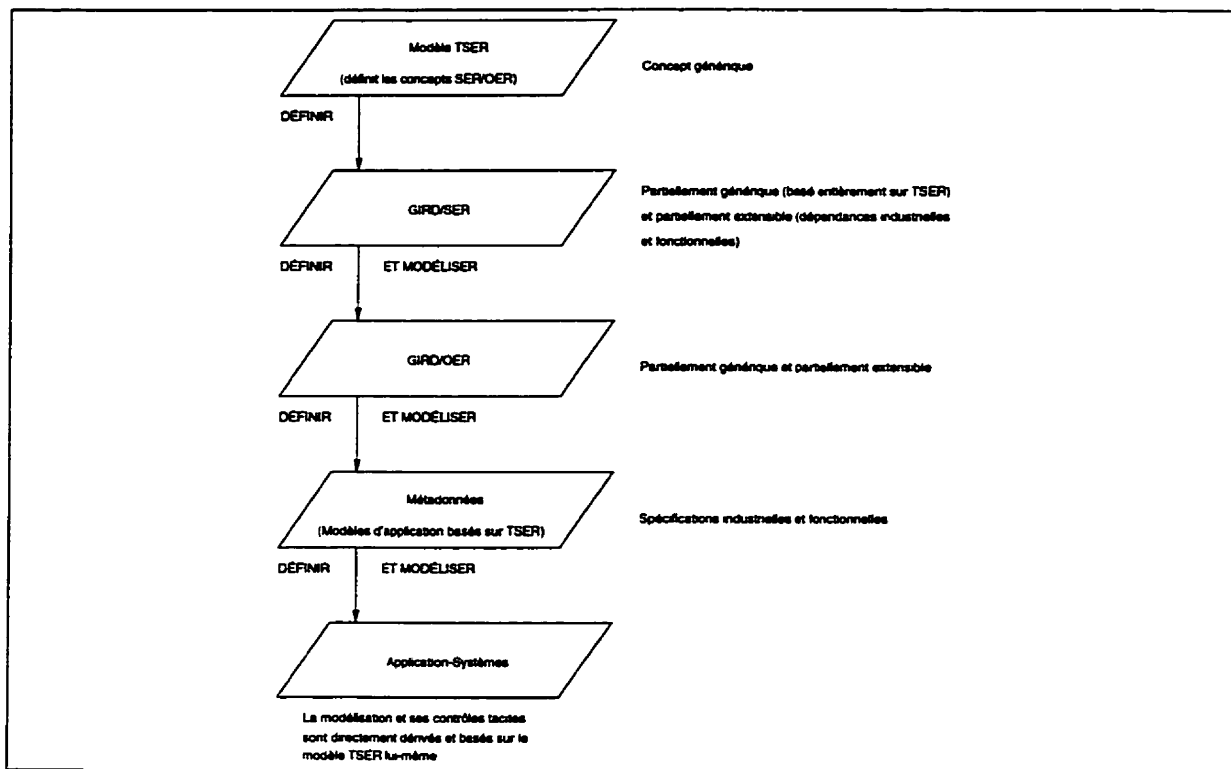


Figure 4.1 : Les couches logiques du concept GIRD

relationnelles. Le résultat est le modèle structurel du GIRD (*Global Information Resources Dictionary/Operational Entity Relation*; GIRD/OER). Cette troisième couche spécifie le schéma de la base de métadonnées. Les deux couches sont étiquetées dans la figure 4.1 comme étant **partiellement générique** et **partiellement extensible**: quelques classes fonctionnelles dans le GIRD/SER et leurs concepts correspondant dans le GIRD/OER sont génériques, tandis que d'autres sont extensibles. Les classes génériques et les concepts représentent des métadonnées conceptualisées à l'aide de TSER et par conséquent, ils ne peuvent être changés ou modifiés d'un environnement à un autre. Les concepts extensibles représentent les modèles d'implantation, comme par exemple : les fichiers, le modèle relationnel ou le modèle orienté-objets. Des changements et des modifications peuvent devenir nécessaires pour accommoder des fonctions spécifiques et/ou des systèmes ayant des dépendances environnementales.

La couche métadonnées est composée d'une variété de modèles informationnels sous-jacents à une application de l'entreprise. Ces modèles sont utilisés pour visualiser et développer les métadonnées de l'entreprise, et ainsi, sont des instances de la couche GIRD/OER. La dernière couche est le portrait fidèle de l'application telle qu'elle existe dans le monde réel de l'entreprise.

L'implantation des applications est basée sur le schéma défini par leurs modèles informationnels sous-jacents, ce qui constitue le contenu de la base de métadonnées. Ainsi, chacun des attributs, des entités et des relations et des attributs de l'application

sont respectivement une instance de type **item**, **entité** et **relation** au niveau de la couche métadonnées en correspondance avec le modèle informationnel. Similairement, pour chacun des types décrits ci-haut, il existe un métaitem, une métaentité ou une métarelation correspondant dans la couche GIRD/OER.

4.2 Niveau fonctionnel du GIRD (GIRD/SER)

En se basant sur le processus général de modélisation de l'information, on peut identifier quatre vues majeures de métadonnées, soient la **Vue-Application**, la **Vue-Ressource**, la **Vue-Fonctionnelle** et la **Vue-Structurale**. Chacune de ces vues est un métasujet dans le modèle fonctionnel du GIRD (cf. figure 4.2, le GIRD/SER). Les vues reflètent la perspective des usagers et permettent une compréhension globale de l'infrastructure informationnelle de l'organisation au niveau sémantique. L'architecture informationnelle de l'entreprise est vue comme un ensemble d'applications effectuant certaines fonctions et des opérations de traitement. Le métasujet **Vue-Application** reflète la compréhension d'une application au niveau exécutif et la façon avec laquelle cette dernière interagit avec chacune de ses congénaires dans l'organisation.

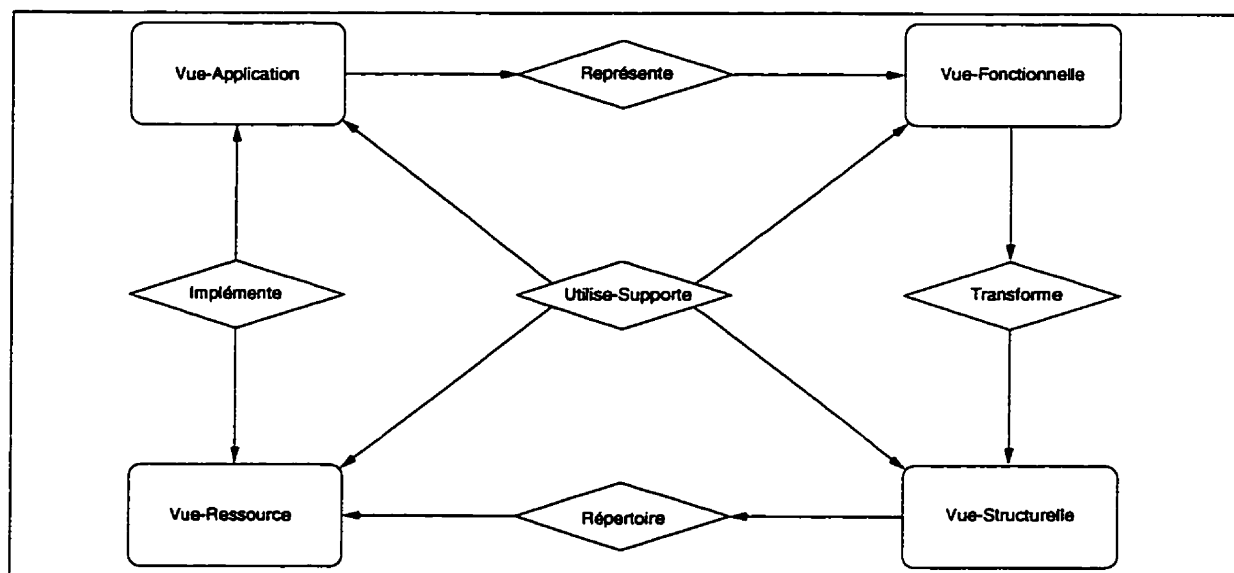


Figure 4.2 : Le modèle fonctionnel GIRD (GIRD/SER)

Le métasujet **Vue-Fonctionnelle** contient les modèles fonctionnels (SER) des applications de l'entreprise. Ces modèles sont détaillés selon leurs domaines, et ils définissent les processus majeurs, la sémantique des données et la logique décisionnelle contenue dans chacun de ces sous-systèmes. Ils reflètent également la vue de l'entreprise selon les gestionnaires qui perçoivent les systèmes d'information comme étant un ensemble d'applications inter-reliées par la logique. Puisque chaque modèle fonctionnel est transformé en un modèle structurel correspondant, l'entreprise peut être également vue comme étant une série de modèles structurels (OER) qui sont intégrés dans le

modèle structurel global de l'entreprise (métasujet **Vue-Structurelle**). Cette classe de métadonnées décrit le schéma sous-jacent de la base de données et de la base de connaissances.

Les trois vues décrites ci-haut représentent l'information minimale requise pour la prise de décision. Chacune d'elles est essentiellement une catégorie de métadonnées déterminée entièrement à partir des concepts du modèle lui-même, qui, à son tour, représente les étapes générales des systèmes d'analyses et des processus de conception. Le niveau le plus bas, qui correspond au métasujet **Vue-Ressource**, n'est pas enraciné dans le processus de modélisation lui-même, mais il est habituellement implémenté dans le modèle informationnel de l'entreprise. Cette vue représente les ressources logicielles et matérielles utilisées pour l'implantation des systèmes et reflète l'implantation, le contrôle et la sécurité de l'information et les applications contenues dans l'entreprise.

Les interactions entre ces différents métasujets sont préservées à l'aide de cinq métacontextes. La logique fonctionnelle, les règles de décision et la conceptualisation sont contenues dans le métacontexte **Représente**, tandis que le métacontexte **Implémente** inclut les règles concernant la phase d'implantation. Le métacontexte **Utilise-Supporte** spécifie l'utilisation, la sécurité et l'entretien de la connaissance des applications. Le métacontexte **Transforme** entrepose les algorithmes (théorie des dépendances et règles heuristiques) qui sont utilisés pour dériver les concepts du OER à partir du SER. Le métacontexte **Répertoire** indique où ces concepts sont entreposés physiquement (localisation et chemin d'accès).

En somme, les informations concernant les ressources sont représentées au niveau du modèle fonctionnel du GIRD (GIRD/SER) en utilisant quatre métasujets et cinq métacontextes qui spécifient les interactions entre les différents métasujets. Le contenu de ces métasujets comprend les ressources informationnelles déterminées par ces trois sources, soit (1) la littérature, (2) le but visé par les paradigmes de modélisation et (3) les études empiriques. La littérature permet d'obtenir une source de base pour catégoriser et comprendre les différentes classes de métadonnées, c'est-à-dire les métadonnées et leur contenu (métaattributs). Ces classes ont en outre été raffinées, et de nouvelles classes ont été rajoutées suite à des études sur les types de métadonnées, telles que les contraintes d'intégrité, les vues et la hiérarchie sémantique contenues dans trois différents paradigmes de modélisation, soit (1) le modèle relationnel, (2) le modèle orienté-objets et (3) l'approche TSER. Les études empiriques ont été conduites dans des environnements qui aident à vérifier que le contenu d'un sujet est suffisant et ce, spécialement pour les règles de représentation.

4.3 Niveau structurel du GIRD (GIRD/OER)

Le modèle fonctionnel fournit un haut niveau de représentation des classes de métadonnées. Ces classes ont des structures de données non-normalisées, ce qui rend difficile l'implantation d'une base de métadonnées qui doit être maintenue cohérente.

Pour qu'une relation (ou métarelation) puisse être traitée par un système de base de données relationnel, il est préférable qu'elle soit à la troisième forme normale (3FN) [Petters84, Ozkarahan86, Date90]. Pour illustrer la structure entière, les quatre prochaines sections décrivent la dérivation des sous-modèles correspondant à chacun des quatre métasujets dans le modèle fonctionnel du GIRD. Le processus de consolidation pour dériver le modèle structurel global intégré du GIRD est discuté à la section 4.3.5. Une description détaillée de la structure des métaentités et des métarelations est fournie à l'annexe A. La définition des métaattributs est fournie à l'annexe B.

4.3.1 Sous-modèle Vue-Application

Les métadonnées contenues dans le métasujet **Vue-Application** (cf. figure 4.3) représentent les fonctions de l'entreprise, qui sont le plus haut niveau d'abstraction des sous-systèmes présents dans l'organisation et les groupes primaires d'utilisateurs. L'entité **APPLICATION** décrit les différentes applications présentes dans l'entreprise, tandis que l'entité **USAGER** décrit les classes et groupes d'utilisateurs dans l'organisation.

La métarelation fonctionnelle **Administrer** détermine l'administrateur d'une application et ce, par le biais du métaattribut **Userid** qui est la clé primaire de la métaentité **USAGER** et est une clé étrangère dans la métaentité **APPLICATION**. La métarelation plurale **Usager_Appl** identifie les différents utilisateurs courants de chaque application avec leur code d'accès et leur mot de passe respectifs. La métarelation plurale récursive **Gérer** démontre la gestion des usagers présents dans l'entreprise, qui est effectué par un administrateur, lui-même considéré comme un usager.

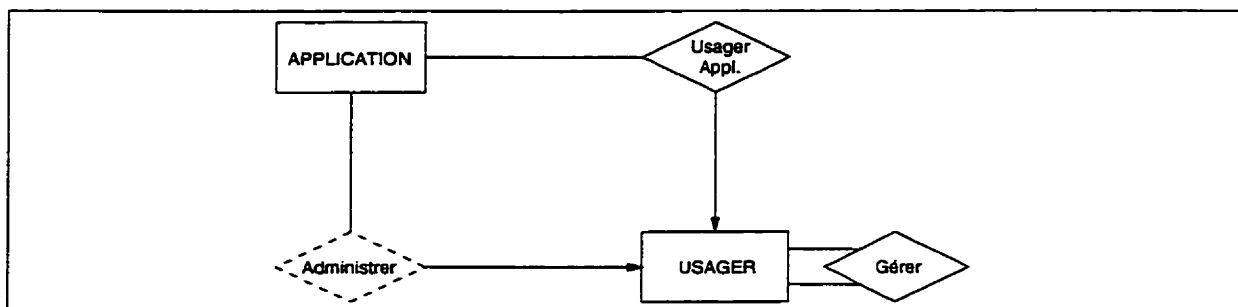


Figure 4.3 : Le sous-modèle **Vue-Application**

4.3.2 Sous-modèle Vue-Fonctionnelle

Le second métasujet, **Vue-Fonctionnel**, contient les concepts décrivant un modèle fonctionnel, tel que décrit par TSER (cf. figure 4.4). Ces concepts sont liés à la sémantique des données et aux aspects dynamiques de l'information. Le savoir dynamique correspond à un ensemble de règles de production de la forme **si-alors**.

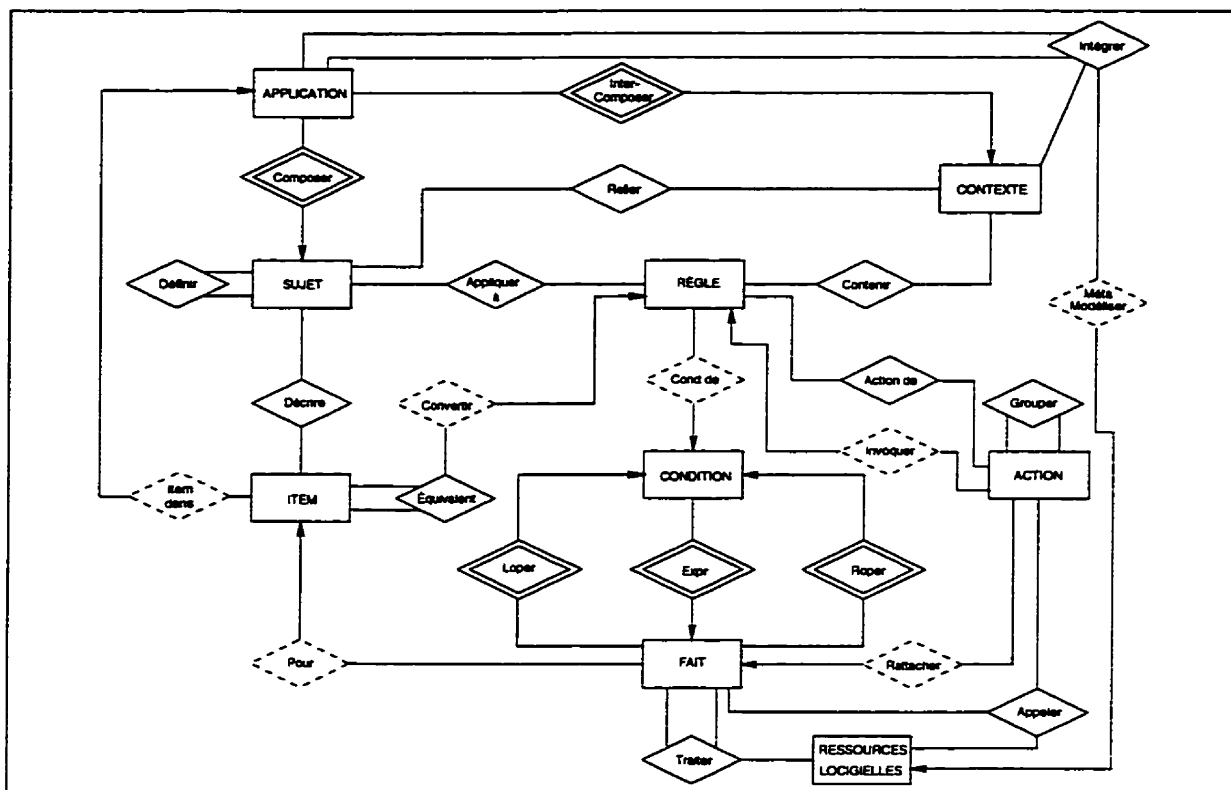


Figure 4.4 : Le sous-modèle **Vue-Fonctionnelle**

La métarelation plurale **Définir**, par sa structure récursive, décrit la hiérarchie avec laquelle les sujets sont modélisés et ce, en appliquant les notions d'agrégation et de généralisation. Cela est réalisé par le biais des métaattributs **Sname** et **SSname**, ce qui implique l'existence de règles de dépendances d'intégrité. La métarelation obligatoire **Composer** raccorde les sujets à leur application correspondante et ce, grâce au métaattribut **Applname** qui est la clé primaire de la métaentité **APPLICATION** et une clé étrangère dans la métaentité **SUJET**. Ces deux métarelations définissent l'architecture hiérarchique d'une application qui est comparable à un arbre dont la racine correspond à une instance de la métaentité **APPLICATION**, et le tronc et les feuilles à des instances de la métaentité **SUJET**.

Similairement, la métarelation obligatoire **Inter-Composer**, qui représente des règles d'intégrité implicites, relie les contextes intra-applications à leur application respective et ce, par le biais du métaattribut **Applname** qui est une clé étrangère dans la métaentité **CONTEXTE**. Les applications reliées par ces contextes peuvent être retrouvées via les sujets correspondants. La métarelation **Relier** raccorde les contextes d'une application aux sujets de celle-ci et ce, grâce aux métaattributs **Cname** (le nom du contexte) et **Sname** (le nom du sujet). Le métaattribut **Direction** indique l'orientation d'un flux d'information entre les sujets et les contextes. Par conséquent, en utilisant **SUJET**, **CONTEXTE** et **Relier**, il est possible de reconstruire le modèle fonctionnel d'une application.

Puisque le schéma du GIRD est conçu pour supporter de multiples systèmes d'application, le métaattribut **Applname** est aussi compris dans la métaentité **ITEM**, qui relie les attributs à leur application respective. À la différence de **SUJET** et de **CONTEXTE**, **Applname** dans **ITEM** implique une règle d'intégrité référentielle qui est représentée par la métarelation fonctionnelle **Item_dans**. Afin de supporter l'autonomie locale des opérations de détermination des noms, cette représentation n'a pas besoin d'utiliser des noms uniques pour les attributs. Les attributs sont uniquement identifiés par un métaattribut **Itemcode** généré par le système comme étant la clé primaire de la métaentité **ITEM**. Cet identifiant est nécessaire pour gérer les attributs reliés logiquement et leurs homonymes et ce, spécialement dans les environnements hétérogènes. Par exemple, lorsqu'un attribut logique est implanté différemment dans plusieurs systèmes, mais avec le même nom, tous les formats doivent être enregistrés dans la base de métadonnées.

Le savoir équivalent aux données globales est représenté par la métarelation plurale **Équivalent**. La clé primaire de cette métarelation, qui correspond à une paire d'identifiants générés par le système, est composée de **Itemcode** et de **EqItemcode**. De plus, **Équivalent** contient deux métaattributs non-clés (**Convertby** et **Reverseby**) qui réfèrent aux règles de traduction des données sur des attributs équivalents. La contrainte d'intégrité référentielle impliquée par ces deux métaattributs est représentée par la métarelation fonctionnelle **Convertir**. La métarelation plurale **Grouper** permet de créer des segments de règles parallélisables ou exécutables en séquence. La métarelation fonctionnelle **Invoquer** implique qu'une action est l'invocation d'une autre règle. La métarelation fonctionnelle **Rattacher** est utilisée lorsque l'action est une assignation, permettant ainsi de savoir à quel fait la variable assignée correspond. La métarelation plurale **Intégrer** spécifie les attributs des applications en relation avec leur contexte. La métarelation fonctionnelle **Métamodéliser** démontre que pour l'intégration d'une application, il existe un ou plusieurs métamodèles décrivant les différents niveaux de celle-ci, et ce, en fonction des ressources logicielles.

4.3.3 Sous-modèle Vue-Structurelle

Le troisième métasujet représente les structures de données sous-jacentes aux schémas de la base de métadonnées et aux bases de connaissances. Les concepts du modèle structurel sont regroupés en deux catégories, selon leurs fonctionnalités. La première catégorie, telle qu'illustrée par la figure 4.5 par la métaentité **ENTITÉ/RELATION**, ne contient que deux types de concepts, l'entité et la relation plurale, ce qui reflète le fait que la structure normalisée se rapporte aux données d'une application. La seconde métaentité, **INTÉGRITÉ**, contient un ensemble de contraintes d'intégrité spécifiées par les concepts du type relation fonctionnelle ou relation obligatoire. La métarelation fonctionnelle **Exister**, par le biais des métaattributs **Master** et **Slave**, renforce la règle qui dit que pour une contrainte d'intégrité déclarée entre deux entités/rerelations, les entités/rerelations elles-mêmes doivent exister.

La métarelation plurale **Transformer** démontre la métamorphose des sujets du modèle fonctionnel d'un système d'information en un ensemble de concepts structurels, tandis que la métarelation plurale **Appartenir à** détermine l'assignation des attributs de ses sujets aux métaentités et métarelations correspondantes, selon leurs dépendances et la théorie de normalisation. Les entités et les relations possédant des clés primaires équivalentes sont simplement fusionnées durant le processus de transformation des sujets. Cela représente le mécanisme de base par lequel l'intégration des vues, des sous-modèles, et des modèles est accomplie. Puisque l'entité ou la relation peut avoir différents noms dans chacune des applications, la métarelation plurale **Nommer** est développée entre les métaentités **ENTITÉ/RELATION** et **APPLICATION** pour maintenir les divers noms (dans les applications locales) des objets logiques globaux.

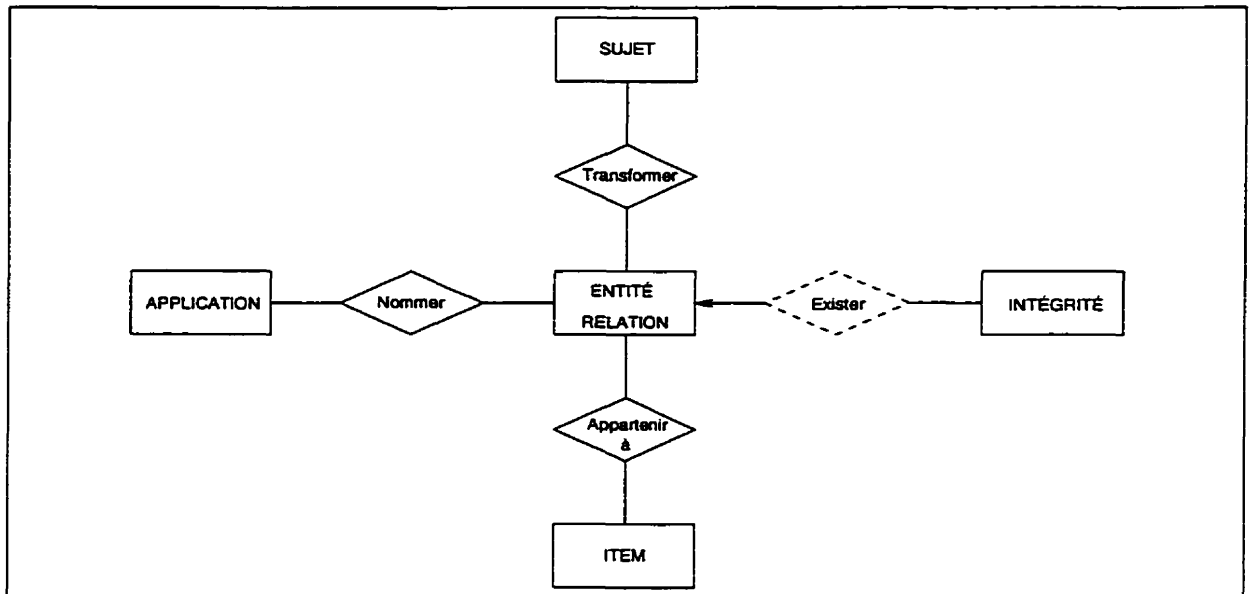
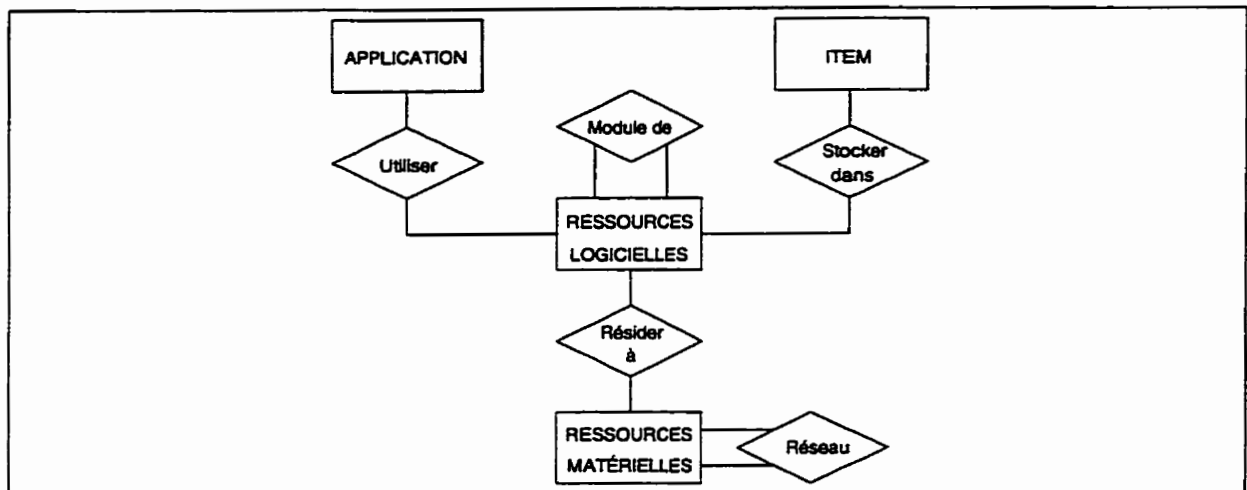


Figure 4.5 : Le sous-modèle **Vue-Structurelle**

4.3.4 Sous-modèle **Vue-Ressource**

Le quatrième et dernier sujet du modèle fonctionnel du GIRD représente les ressources logicielles et matérielles utilisées dans l'entreprise pour opérer ses différentes applications. Ainsi, ce sous-modèle contient deux métaentités principales, soient **RESSOURCES_MATÉRIELLES** et **RESSOURCES_LOGICIELLES** (cf. figure 4.6). La première métaentité décrit les composantes physiques des ressources informationnelles de l'organisation, tandis que la seconde représente les différents types de ressources logicielles tels que les fichiers, les documents et les programmes et ce, par le biais du métaattribut **Restype**. Le métaattribut **Resid**, qui est une clé primaire générée par le système, identifie les ressources logicielles de manière unique, permettant ainsi de relacher la contrainte d'unicité sur le nom des applications et ce, à la grandeur de l'entreprise.

Figure 4.6 : Le sous-modèle **Vue-Ressource**

La métarelation plurale **Stocker_dans** détermine les fichiers où sont emmagasinés les attributs d'une application respective. La métarelation plurale **Module_de** est de type récursive et représente le fait qu'une ressource logicielle peut faire appel à ses congénaires pour effectuer une tâche particulière. La métarelation plurale **Utiliser** identifie les ressources logicielles d'une application et ce, de manière à ce qu'une application puisse être implantée en utilisant plusieurs ressources logicielles ou qu'une ressource logicielle particulière soit exploitable par plusieurs applications. Similairement, la métarelation **Résider_à** indique qu'une ressource logicielle particulière peut résider dans différentes ressources matérielles, et en même temps, chacune d'elles peut contenir plusieurs ressources logicielles. La métarelation plurale **Réseaux**, qui est de type récursive, démontre que les différentes ressources matérielles de l'entreprise sont inter-reliées par le biais de réseaux.

4.3.5 Consolidation des quatre sous-modèles

Le schéma de la base de métadonnées est construit en fusionnant les modèles, les vues (cf. figure 3.4) et différentes classes de métadonnées, telles que les ressources logicielles et les ressources matérielles. La consolidation de ces métadonnées est organisée pour produire un schéma conceptuel, le GIRD, qui est le métamodèle permettant de réaliser l'intégration des différents modèles. Une fois la base de métadonnées instanciée, elle participe de manière directe à la gestion de l'information de l'entreprise et facilite ainsi l'intégration de systèmes locaux. Par conséquent, la base de métadonnées offre une représentation fidèle de l'architecture informationnelle de l'entreprise, permettant de supporter le traitement de requêtes globales, l'intégration de systèmes, les outils CASE et d'autres applications de métadonnées de types passifs.

La figure 4.7 illustre la structure globale du modèle structurel du GIRD (GIRD/OER), qui est le résultat de la consolidation des quatre sous-modèles décrits plus haut. La notion de clé étrangère (contrainte d'intégrité référentielle) apparais-

sant entre des concepts situés dans différents sous-modèles est effectuée en créant de nouvelles métarelations fonctionnelles. Dans le présent cas, la métarelation **Maintenir** a été créée. Elle relie **RESSOURCES_MATÉRIELLES** et **USAGER**, puisque la clé primaire de **USAGER** (Userid) est aussi un métaattribut non-clé de **RESSOURCES_MATÉRIELLES**.

Suite à cette analyse, on peut établir les propriétés de conception du modèle GIRD et les caractéristiques suivantes [Hsu et al.91] :

1. Une hiérarchie de modèles (représentée par les quatre métasujets), correspondant au cycle de vie habituel présent dans le processus d'analyse et de conception de systèmes, est couvert et emmagasiné d'une manière structurée.
2. Trois dimensions de représentation des données sont comprises, soit : (1) les relations de dépendances, (2) les contraintes d'intégrités et (3) l'abstraction hiérarchique des données.
3. La représentation des connaissances englobant les dimensions statiques et dynamiques des classes des connaissances requises pour l'intégration des informations est fournie par le biais de règles intra-sujets (encapsulation), de règles inter-sujets regroupées par contextes et de flux de données bidirectionnels.
4. Les attributs et les prédicats logiques de la forme règles de production, définis dans le modèle structurel, facilitent l'unification des classes de métadonnées.
5. Le schéma du GIRD possède les niveaux fondamentaux nécessaires à la représentation hétérogène de l'information et en facilite ainsi l'intégration.
6. L'intégration de modèles de données hétérogènes est effectuée uniquement au niveau logique et ce, via le schéma informationnel de l'entreprise et cette intégration est implantée par le biais de liens définis dans le GIRD, éliminant ainsi le processus d'intégration physique des schémas locaux.
7. La structure de base du GIRD est enracinée dans la méthodologie TSER qui est directement dérivée de la modélisation logique ; par conséquent, elle est auto-descriptive au niveau de la définition récursive.
8. La méthodologie TSER, lorsque utilisée pour modéliser les applications d'une entreprise, facilite la définition des métadonnées nécessaires à l'instanciation de la structure du GIRD, et le processus de modélisation peut être réversible, c'est-à-dire, que les modèles fonctionnel et structurel d'origine peuvent être reproduits directement et entièrement à partir du contenu de la base de métadonnées.

L'utilisation de TSER n'est pas obligatoire pour le développement du modèle GIRD, car la structure, la définition et les conventions du GIRD deviennent indépendantes après que leur développement ait été complété.

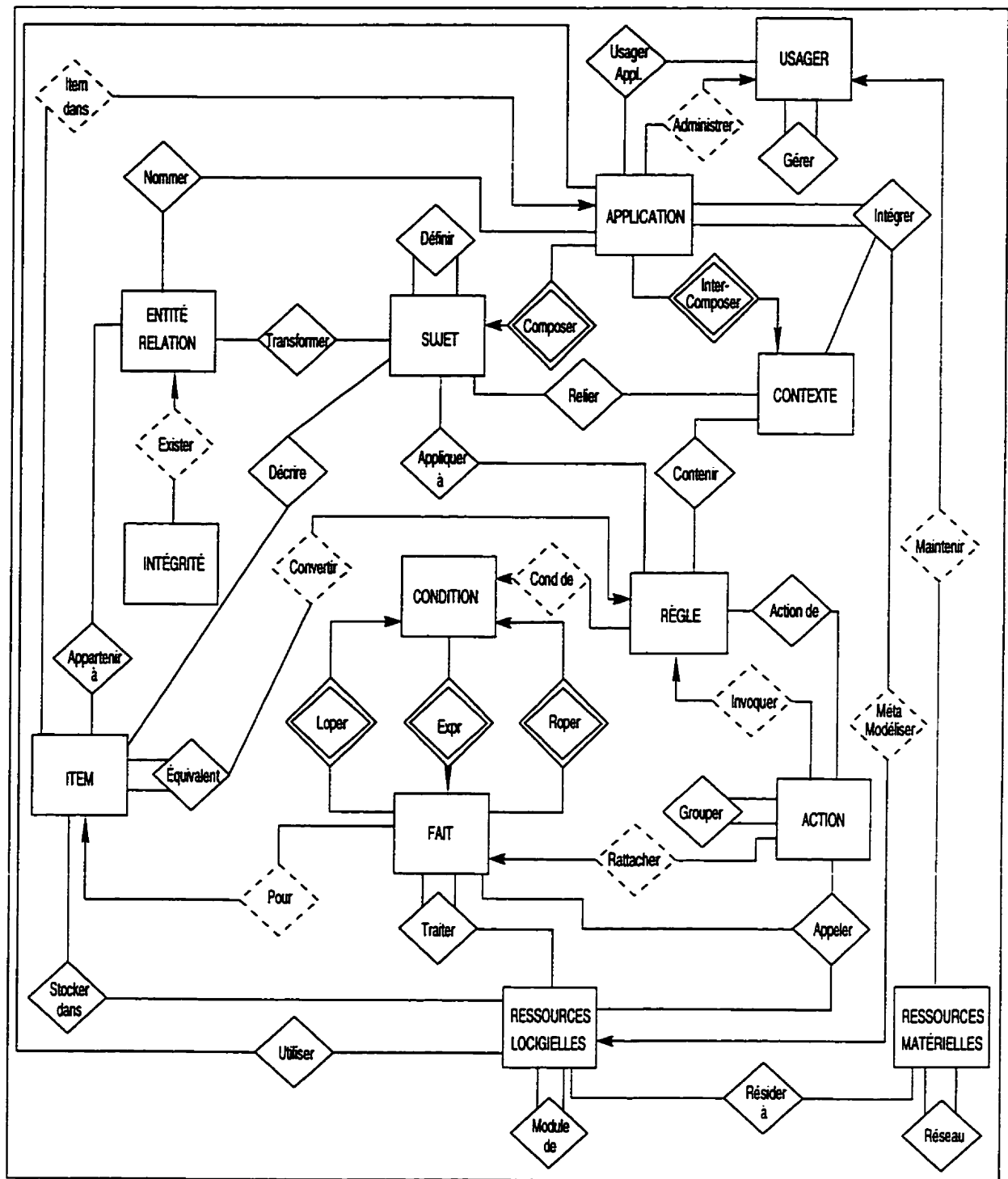


Figure 4.7 : Le modèle structurel du GIRD (GIRD/OER)

Chapitre 5

Système de requêtes globales

La formulation d'une requête globale dans un environnement hétérogène et distribué peut demander des connaissances techniques concernant l'architecture des systèmes d'information présents dans l'entreprise. Cette contrainte a pour effet de limiter l'accès de cette tâche à une clientèle spécialisée. Le développement d'un système de requêtes globales rend le savoir de l'entreprise accessible aux utilisateurs novices, car ce logiciel prend en charge tous les aspects techniques du système d'information global. Dans ce chapitre, nous présentons une description des caractéristiques que ce genre de système devrait posséder, et nous expliquons comment la base de métadonnées peut être exploitée dans ce contexte. L'architecture du système de requêtes globales est également présentée, ainsi que les différentes étapes nécessaires à la résolution d'une requête globale.

5.1 Définition d'un système de requête générique

L'intégration de plusieurs systèmes est un problème omniprésent pour les entreprises [Rattner90]. Un objectif primordial au niveau de l'intégration des systèmes est de fournir une structure logique permettant d'intégrer ces ressources informationnelles et ainsi créer un système d'information global pour toute l'entreprise, ce qui facilite la distribution du savoir de celle-ci entre ses différentes sous-entités. Un système d'information intégré doit être capable :

1. de supporter les besoins des utilisateurs lors de la recherche d'information au niveau global de l'entreprise ;
2. de faciliter les accès des applications à l'ensemble des informations maintenues dans les systèmes locaux ;
3. de maintenir la cohérence de l'information malgré les différentes mises à jour possibles.

La clé fondamentale de cette réussite est ce qui peut être appelée une assistance en direct. En particulier, les utilisateurs devraient être capables d'établir des requêtes comme si le système était un tout bien soudé et ce, sans avoir à connaître les détails

techniques concernant les différents systèmes locaux qui composent l'architecture informationnelle de l'entreprise. De même, un sous-système devrait pouvoir interagir avec ses congénères sans devoir se soucier de l'architecture des ressources logicielles et matérielles de l'entreprise. Un système ne possédant pas ces caractéristiques est dit statique et il devient alors difficile d'effectuer des changements, car la maintenance peut être fastidieuse. Un système possédant une architecture dynamique serait plus efficient pour répondre à ce problème ; cependant, un tel système n'est pas encore disponible, si ce n'est au niveau expérimental.

Une opération globale de requête comprend deux étapes : la **formulation** de la requête globale et le **traitement** de celle-ci. Dans la première étape, la requête de l'utilisateur est articulée et représentée de manière à ce que le système de requête globale puisse la comprendre. Dans la seconde étape, qui est accomplie à l'intérieur du système, les différentes sous-requêtes sont envoyées aux systèmes locaux appropriés pour retrouver l'information pertinente et formuler une réponse compréhensible par l'utilisateur.

Un système utilisant une assistance intelligente en direct permet de supporter la formulation de requêtes de haut niveau, ce qui rend l'utilisation de ce système beaucoup plus conviviale. De cette façon, il peut manipuler toutes sortes d'interprétations et de transformations entre les requêtes globales formulées, et supporter les différentes implantations au niveau des systèmes locaux. Pourvu de ses capacités, le traitement des requêtes globales devrait être accompli dans l'ordre suivant [Cheung91] :

1. **Optimisation de la requête**

La requête globale est premièrement décomposée de manière optimale en un ensemble de requêtes locales et ce, en tenant compte de la sémantique et de la performance.

2. **Traduction de la requête**

Les requêtes locales sont encodées dans le langage source de manipulation de données.

3. **Exécution de la requête**

Les requêtes locales, qui ont été encodées, sont expédiées et traitées à leur site respectif.

4. **Résultat de l'intégration**

Les résultats des requêtes locales sont assemblés pour répondre à la requête globale.

Chacune de ces étapes utilise les métadonnées présentes dans l'entreprise. Par exemple, le schéma local de la base de données, les répertoires, les informations réseaux et les connaissances contextuelles des données sont requises pour l'optimisation de la requête ; les informations concernant les systèmes de gestion de base de données locaux pour la traduction de la requête ; les règles d'opérations sur les flux d'information pour l'exécution de la requête ; et les connaissances concernant les équivalents de données,

les incompatibilités et la conversion des données pour l'intégration des résultats. La formulation d'une requête globale est effectuée par un utilisateur via une interface prévue à cette effet.

5.1.1 Interface usager pour la formulation de requêtes

La création d'interfaces conviviales pour les différentes fonctionnalités de la base de métadonnées est importante, car elle facilite l'utilisation de celles-ci, rendant ainsi ces services accessibles à la majorité des utilisateurs de l'entreprise. Bien sûr, il n'existe pas d'interface parfaite répondant aux exigences de tous. Les techniques telles que les fenêtres, les icônes, les menus, les graphiques, la visualisation [Nilan92, Robertson et al.93] et les autres formes d'interfaces personnes/machines devraient respecter les onze règles de l'art suivantes [Apple92, Nielsen93, Shneiderman92] :

1. Favoriser la simplicité
2. Parler le langage de l'utilisateur
3. Minimiser la charge mentale
4. Demeurer cohérente
5. Informer l'utilisateur
6. Indiquer clairement les sorties
7. Soutenir les novices et les experts
8. Offrir un traitement simple des erreurs
9. Prévenir les erreurs
10. Utiliser des métaphores
11. Ne rien cacher, être transparent envers l'utilisateur

La création d'interfaces d'applications (*Application Program Interface*; API) fait appel aux principes théoriques cognitifs et peut ainsi être une ligne de conduite pour faciliter les tâches de requêtes dans les bases de données [Gamache95]. Une API est une bibliothèque de fonctions normalisées et utilisées par les applications pour interfacer avec n'importe quel SGBD. Ces fonctions exploitent les ordres DML (*Data Manipulation Language*) propres à un SGBD via un pilote spécifique. Cela renforce l'indépendance de l'application vis-à-vis du SGBD. Les interfaces ODBC (*Open Data Base Connectivity*) et SAG (*SQL Access Group*) sont des efforts de normalisation qui prennent cette direction pour les architectures de type client/serveur. Le DML constitue un langage autonome auquel on a ajouté les structures de contrôle connues : itération et alternative. Un bon exemple de langage de quatrième génération (L4G) est le logiciel SQLFORM 4.5 de ORACLE. Ce L4G permet la création de programmes

complets avec des interfaces graphiques conviviales. Le logiciel est disponible sur plusieurs plate-formes (UNIX, Windows et autres) et ce, grâce à l'utilisation de bibliothèques complètes. Il facilite aussi la formulation de requêtes ad hoc pour les utilisateurs.

La formulation d'une requête globale dans la base de métadonnées peut être écrite manuellement dans un langage approprié (*Metadatabase Query Language*; MQL) ou être effectuée via l'interface du système de requêtes globales (*Global Query System*; GQS).

Une question intéressante qui se pose, concernant spécialement le traitement des requêtes globales, est comment peut-on réconcilier et consolider les différentes vues systèmes et les méthodes de représentation et ce, à la grandeur de l'entreprise, tout en maintenant les différences entre les sous-systèmes pour en assurer la flexibilité ? Des recherches ont révélé [Hsu96] que l'utilisation de métadonnées dans une entreprise s'avère une approche intéressante pour la représentation des vues globales et la gestion des transactions de requêtes dans un environnement de bases de données hétérogènes et distribuées.

5.1.2 Objectifs du système de requêtes globales

L'objectif principal des travaux de Cheung [Cheung91] est de développer une nouvelle approche fondamentale pour les requêtes globales et ainsi permettre aux utilisateurs de l'entreprise d'accéder à l'information contenue dans les multiples sous-systèmes hétérogènes, sans devoir changer les sous-systèmes déjà existants ou obliger les utilisateurs à acquérir des connaissances techniques complexes. L'approche de base est d'employer une nouvelle classe de métadonnées comme base de connaissances en direct de l'entreprise, et par la suite créer une nouvelle classe de méthodes qui utiliserait cette base de connaissances pour résoudre le problème. De plus, la technologie de la base de métadonnées est incorporée dans ces méthodes et permet ainsi le développement d'un nouveau concept, le système de requêtes globales (GQS).

Dans la mesure où l'on désire maintenir la transparence au niveau des opérations de requêtes globales pour les utilisateurs, le GQS a besoin d'utiliser l'entrepôt des métadonnées de l'entreprise qui comprend :

1. **La connaissance des usagers et des applications**

L'information en relation avec les règles d'affaires et les perspectives des usagers concernant les systèmes d'information de l'entreprise (incluant les privilèges d'accès aux bases de données locales et l'information dirigeant vers des modèles spécifiques d'usagers, supportant ainsi des interfaces adaptées et personnalisées selon le type d'utilisateur) ;

2. **La connaissance pour résoudre une requête**

L'information des équivalences de données globales et les règles de conversion, ainsi que le dictionnaire et le répertoire de l'information ;

3. La connaissance sur les systèmes d'information de l'entreprise L'information auto-descriptive sur les modèles de l'entreprise sous-jacents.

L'approche est d'utiliser la base de métadonnées comme un entrepôt pour assiter le GQS. Cette approche emploie un principe unique qui est le modèle de la base de métadonnées visant la connaissance de l'entreprise. Cette connaissance permet d'obtenir la représentation de la structure des métadonnées, et par surcroît, le support des besoins nécessaires pour la formulation des requêtes globales et le traitement de celles-ci.

5.1.3 Exigences de base pour une requête globale

Une hypothèse fondamentale est que les organisations ont besoin d'exploiter et d'interrelier les systèmes d'information qui ont été développés et administrés de façon indépendante. Il existe un besoin substantiel pour la flexibilité de gestion, le partage de l'information et l'autonomie dans les systèmes de bases de données interopérables. Les technologies existantes n'offrent presque pas d'alternatives ou de solutions aux opérations de traitement des requêtes globales. Par exemple, les systèmes distribués traditionnels s'appuient sur les schémas intégrés, ce qui produit des difficultés lors de la conversion de ces schémas et impose ainsi des changements dans les systèmes locaux. Les interfaces de requêtes déjà existantes doivent posséder tous les détails techniques concernant les systèmes avec lesquels elles doivent interagir ; par conséquent, elles sont conçues exclusivement pour les informaticiens. Les besoins fondamentaux pour les requêtes globales peuvent être spécifiés comme suit :

1. Le modèle globale

Il représente toutes les ressources de données partagées à travers l'entreprise, ce qui est une exigence unique et essentielle pour les opérations de requêtes globales. Le modèle fournit une vue de l'information contenue dans les différents systèmes locaux. Une requête globale est formulée selon le modèle global.

2. Une interface utilisateur

Elle facilite la formulation de la requête globale par un utilisateur et affiche les résultats de cette même requête. Une interface usager est un besoin générique pour la gestion des requêtes dans une base de données. Cependant, l'hétérogénéité de l'environnement informatisé augmente la complexité de ces supports visuels. L'interface idéale devrait fournir une assistance intelligente pour aider les utilisateurs novices lors de la formulation de leurs requêtes et supporter les utilisateurs experts en leur permettant de formuler des requêtes globales efficaces.

3. La décomposition et l'optimisation des requêtes globales

La requête globale peut faire appel à plusieurs systèmes locaux, ce qui nécessite une décomposition de cette dernière en sous-requêtes conformément au schéma local. Chacune d'elles devrait être associée à un seul système local. Deux critères sont souvent utilisés pour l'optimisation de la décomposition de la requête globale : (1) minimiser le nombre de systèmes locaux interrogés et (2) minimiser le

nombre de données devant être transférées. L'optimisation d'une requête locale est accomplie efficacement par le SGBD.

4. Traduction de la requête

Une sous-requête se rapporte à un système local qui est implanté en utilisant soit un SGBD, soit un système de traitement de fichiers. Pour les systèmes utilisant une base de données, la sous-requête est traduite en DML local. Pour ceux qui emploient un gestionnaire de fichiers, du code adapté est généré pour effectuer la recherche dans les fichiers.

5. Intégration des résultats

Les résultats intermédiaires des sous-requêtes sont premièrement interprétés et par la suite, assemblés selon les conditions de jointure. La conversion des données peut être effectuée à cette étape pour résoudre les incompatibilités de données entre ces résultats. La plupart des systèmes de requêtes globales existants assurement que les incompatibilités soient résolues lors du processus d'intégration des schémas. Sinon, ces systèmes imposent une standardisation des données et ce, à la grandeur de l'entreprise, ce qui permet d'éliminer le processus de conversion des données. Cependant, cette approche n'est pas très réaliste, car elle impose une structure statique des données.

5.2 Traitement d'une requête

Le MGQS (*Model-assisted Global Query System*) [Cheung91] utilise une approche basée sur la connaissance, améliorant ainsi la formulation et le traitement des requêtes. Cependant, la connaissance est une notion abstraite qui peut avoir plusieurs interprétations. Dans une perspective de l'intégration de l'information, la connaissance est définie comme étant les métadonnées de l'entreprise qui sont requises au niveau du système global pour atteindre cet objectif [Hsu et al.94]. Les métadonnées peuvent être cataloguées en quatre classes, soit : (1) la définition des données, (2) la définition des schémas, (3) la définition des vues et (4) la connaissance contextuelle qui inclut les règles d'opérations, règles de contrôles et les règles de décisions.

Le concept d'une assistance pour le QQS dans une base de métadonnées est développé selon les principes logiques suivants. Toutes les classes de métadonnées de l'entreprise sont spécifiées et structurées selon une méthode de représentation. Cette structure de métadonnées peut alors servir de base pour l'organisation et l'implantation des ressources informationnelles de l'entreprise dans une base de métadonnées partagées et non-disponible en direct. Ainsi, les vues fonctionnelles, les modèles de données, les règles d'affaires et les autres connaissances concernant les opérations du système global sont facilement accessibles pour le système et les usagers par le biais de la base de métadonnées. Par conséquent, l'assistance en direct (*online*) pour la formulation et le traitement des requêtes devient un concept réalisable et parfaitement caractérisé. Des méthodes spécifiques basées sur cette connaissance peuvent être définies et développées en fonction des exigences des métadonnées et des services dans chacune des tâches.

5.2.1 Algorithme de résolution d'une requête globale

Il faut premièrement caractériser le rôle de la base de métadonnées comme étant une base de connaissances en direct pour les opérations globales. Cette caractérisation débute avec une analyse technique des tâches majeures au niveau des requêtes globales et leurs exigences de base concernant les métadonnées.

Supposons une requête globale GQ caractérisée par un ensemble d'attributs A impliqués dans une requête ; un ensemble d'entités ou de relations persistantes D emmagasiné avec ses attributs ; et une expression $\langle C \rangle$ spécifiant les conditions de recouvrement. L'expression $\langle C \rangle$ est constituée de sous-expressions qui sont des conditions de sélection $\langle SC \rangle$ et des conditions de jointures $\langle JC \rangle$. Tous les attributs, les objets et les expressions sont sujet à des spécifications en terme de métadonnées du système $\langle M \rangle$ (ex : les noms et les structures). Ces métadonnées, pouvant être fournies par l'utilisateur ou par le système d'assistance en direct, doivent satisfaire au minimum exigé pour chaque requête particulière.

$$\begin{aligned} GQ &= (A, D, \langle C \rangle \mid \langle M \rangle) \text{ où} \\ A &= A^u \cup A^s \text{ avec } A^u \neq \emptyset, \\ D &= D^u \cup D^s \text{ avec } D \neq \emptyset \text{ et } D^u \cap D^s = \emptyset, \\ \langle C \rangle &::= [\langle SC \rangle \mid \langle JC \rangle \mid \langle SC \rangle \text{ AND } \langle JC \rangle], \\ \langle M \rangle &::= \{\langle M^u \rangle \cup \langle M^s \rangle\}. \end{aligned}$$

Voici la définition détaillée de chacune des variables :

- A^u Cette variable représente un ensemble d'attributs sélectionnés par l'utilisateur. Elle comprend les attributs demandés directement pour le recouvrement et les attributs indiqués dans les conditions de sélection. Une requête globale doit posséder au moins chaque attribut de A^u .
- A^s Le MGQS peut déterminer que des attributs additionnels sont nécessaires ou implicites pour la requête, pour ainsi compléter la définition de la requête au niveau des attributs manquants.
- D^u L'utilisateur peut aussi spécifier un ensemble d'informations (entités ou relations) D^u contenant les attributs sélectionnés (A^u).
- D^s Puisque D^u peut ne pas contenir tous les attributs de A^u , le système doit déterminer les informations restantes qui sont impliquées dans l'opération de requête globale.

- $\langle M^u \rangle$ Cette expression, correspondant aux métadonnées fournies par l'utilisateur, représente les connaissances techniques minimales des différents systèmes et des modèles informationnels de l'entreprise que l'utilisateur doit posséder pour formuler une requête complète. En général, ces connaissances sont exprimées par le biais d'un langage de spécification adapté. L'approche MGQS vise l'élimination totale de cette partie.
- $\langle M^s \rangle$ Cette expression, correspondant aux métadonnées fournies par le système, représente un complément de l'expression $\{\langle M^u \rangle\}$ qui doit respecter les exigences minimales au niveau des métadonnées pour une requête particulière. Dans MGQS, $\langle M^s \rangle$ satisfait à toutes les exigences de $\langle M \rangle$.

L'opération de requête globale est décrite comme suit [Cheung et Hsu96] :

- Étape 1 : Formulation de la requête globale
- Étape 2 : Détermination des conditions de jointure
- Étape 3 : Traitement de la requête globale
- Étape 4 : Génération des requêtes locales
- Étape 5 : Exécution des requêtes locales
- Étape 6 : Intégration des résultats

Étape 1 : Formulation d'une requête globale

Pour formuler une requête globale, l'utilisateur spécifie les attributs A^u devant être retrouvées dans la réponse. Les informations contenant A^u ne sont pas requises pour être identifiées. Cependant, les informations dénotées D^u peuvent être utilisées pour clarifier les ambiguïtés sémantiques potentielles. Les conditions de sélection $\langle SC \rangle$, lorsqu'il y en a, sont aussi spécifiées dans cette étape. Puisque l'intervention de l'utilisateur n'est pas nécessaire pour spécifier tous les détails techniques (les détails manquants seront automatiquement complétés par le système de requête), le résultat de la formulation sera probablement une requête globale incomplète IGQ définie comme suit :

$$IGQ = (A^u, [D^u], [\langle SC \rangle] \mid [\langle M \rangle]).$$

Une approche directe est utilisée dans MGQS pour les formulations des requêtes globales par laquelle le contenu des informations pertinentes et des modèles des systèmes locaux hétérogènes sont fournis en mode interactif. L'utilisateur peut naviguer à travers le modèle informationnel de l'entreprise pour reconnaître et visiter dans n'importe quel ordre, les données projetées. Cette méthode de navigation est conçue selon les caractéristiques des schémas et les associations logiques entre les modèles informationnels

stockés dans la base de métadonnées. La formulation de requêtes globales se compose des étapes suivantes :

- **étape 1.1 :**
Sélection des attributs (a_i^u) de d_i^u pour le recouvrement. Les métadonnées requises sont : les noms des applications, les vues fonctionnelles, les concepts de données, les attributs et leurs associations (ex : le modèle global de l'entreprise) ;
- **étape 1.2 :**
Spécification des conditions de sélection $\langle SC \rangle$ comprises entre les attributs choisis. Les métadonnées requises sont : les formats, les domaines et les contraintes opérationnelles des attributs ;
- **étape 1.3 :**
Résolution des ambiguïtés sémantiques. Les métadonnées requises sont : les contraintes sémantiques telles que les dépendances fonctionnelles, et les règles d'affaires qui décrivent l'utilisation prévue des données.

Un langage de programmation alternatif de requête peut être développé en se basant directement sur l'approche spécifiée ci-haut, qui est effectué en MGQS.

Étape 2 : Détermination des conditions de jointure

Une requête globale peut impliquer plusieurs éléments par l'entremise des entités et des relations. Normalement, l'utilisateur doit explicitement spécifier les conditions de jointure équivalentes entre ces éléments pour obtenir une requête globale complète. Les spécifications devraient nécessiter les détails pour la compréhension du modèle informationnel. Un algorithme de détermination des conditions de jointure utilisant les métadonnées est employé pour alléger cette contrainte :

- **étape 2.1 :**
Déterminer l'ensemble des informations contenant tous les attributs sélectionnés par l'utilisateur et leurs attributs équivalents. Cette étape établit l'espace de recherche maximal des attributs impliqués dans la requête.

Pour chaque attribut $a_k^u \in A^u$ faire

$O_k = \text{DataObject}(\text{Equivalent}(a_k^u)) :$

$\text{Equivalent}()$ est une fonction qui prend un attribut saisi en entrée et retourne un ensemble d'attributs équivalents à a_k^u incluant a_k^u lui-même.

$\text{DataObject}()$ est une fonction qui prend un ensemble d'attributs ($\text{Equivalent}(a_k^u)$) comme entrée et retourne un groupe d'entités/rerelations plurales (O_k), de telle sorte que toutes les données résultantes contiennent au moins un attribut de l'ensemble fourni en entrée. Les métadonnées requises sont : les associations entre les entités/rerelations et leurs données, et les informations de données équivalentes.

- **étape 2.2 :**

Déterminer un ensemble minimal de données (O_{min}) nécessaire au système pour chercher à combler les exigences de la requête globale.

$$O_{min} = Min \parallel \{O_h | \forall (a_k^u) \exists (d_0) (d_0 \in O_h) \wedge (d_0 \in O_k) \wedge (D^u \subseteq O_h)\} \parallel$$

- **étape 2.3 :**

Identifier le chemin d'accès le plus court (SP_{min}) qui raccorde tous les $d_m \in O_{min}$. Les métadonnées requises sont : les associations entre les entités et les relations dans le modèle global de l'entreprise.

- **étape 2.4 :**

Insérer les conditions de jointure pour chaque paire d'objets raccordée dans D , ce qui permet de déterminer les conditions de jointure pour chaque paire d'éléments d_i et d_j qui est raccordée par un lien $l \in L_{min}$. Une condition de jointure est insérée pour chacun des attributs clés de d_i qui est aussi un attribut clé ou une clé étrangère dans d_j et vice versa. L'équivalence de données est considérée pour la comparaison. La clé ou les attributs de clés étrangères ajoutés à la requête globale pour les conditions de jointure sont A^* . Les métadonnées requises sont : les clés primaires, les clés étrangères et les équivalences de données.

Étape 3 : Le traitement de la requête globale

Une requête globale formulée $GQ = (A, D, < C >)$ est décomposée en une série de requêtes locales LQ_i où i indique le système local. La décomposition est basée sur l'entreposage physique et la distribution des données recouvrées. Chacune des requêtes locales LQ_i se rapporte à un seul système local.

$$LQ_i = (LA_i, LF_i, < LC_i >) \text{ où}$$

LA_i est un ensemble d'attributs locaux avec $\cup_i(LA_i) = A$,
 LF_i est un ensemble de fichiers locaux,
 $< LC_i >$ est l'expression d'une condition qui concerne seulement les données qui sont contenues dans le système local i

Pour un fichier système, LF est un ensemble de fichiers locaux qui contient toutes les données locales dans LA_i . LF_i peut aussi être une série de relations de bases locales si le système i est une base de données relationnelle, ou peut être une série d'enregistrements si i est une base de données hiérarchique. Pour effectuer le traitement d'une requête globale, les étapes suivantes doivent être effectuées :

- **étape 3.1 :**

Déterminer tous les fichiers de données qui contiennent $a_j \in A$

Pour chaque attribut local $a_j \in A$ faire

$$F_j = \text{GET_FILE}(\text{Equivalent}(a_j))$$

$\text{GET_FILE}()$ est une fonction qui prend un ensemble d'attributs ($\text{Equivalent}(a_j)$) en entrée et retourne un groupe de fichiers de données (F_j), de telle manière que chaque fichier résultant contient au moins un attribut de l'ensemble d'attributs

fourni en entrée. Les métadonnées requises sont : les équivalences de données, la description de l'entrepasage physique telle que les fichiers, les tables de relations et leurs données, c'est-à-dire l'implémentation des modèles.

● **étape 3.2 :**

Déterminer l'ensemble minimal de fichiers de données (F_{min}) par lequel le système de requête recouvre les données (A).

$$F_{min} = Min_k \parallel \{F_k | \forall (a_j^u) \exists (f_0) (f_0 \in F_k) \wedge (f_0 \in F_j)\} \parallel$$

● **étape 3.3 :**

Formuler une requête locale LQ_i pour le système local i tel que

1. $\forall (lf_i) (lf_i \in LF_i) \wedge (lf_i \in F_{min}),$
2. $\forall (la_i) \exists (lf_i) (la_i \in LA_i) \wedge (lf_i \text{ contient } la_i),$
3. tous les attributs impliqués dans $\langle LC_i \rangle$ sont des éléments de LA_i .

Les métadonnées requises sont : la description de l'entrepasage physique telle que les fichiers, les tables de relations et leurs données.

● **étape 3.4 :**

Préserver les conditions de jointure globales $\langle GJC \rangle$ telles que

$$\langle GJC \rangle ::= \langle j_condition \rangle [\text{AND } \langle j_condition \rangle]$$

$$\langle j_condition \rangle ::= \text{item1} = \text{item2} \text{ où}$$

item1 et item2 appartiennent à deux systèmes différents. Les métadonnées requises sont : la description de l'entrepasage physique telle que les fichiers, les tables de relations et leurs données.

L'étape 3 est critique pour obtenir un rendement efficient du système. Un certain nombre de stratégies de conception peuvent être utilisées pour optimiser la performance. Quelques-unes de ces stratégies pourraient impliquer plus de métadonnées que nécessaires pour le traitement de base.

Étape 4 : La génération de la requête locale

La génération de la requête locale dans MGQS est extensible. Un générateur de code est fourni pour chacun des langages de manipulation de données utilisés dans l'entreprise. Une requête locale est générée en utilisant le langage local spécifique et ce, pour chacune des requêtes locales devant être traitées. Les métadonnées requises sont : l'implantation des modèles (le SGBD local, le DML local et les chemins d'accès) et les informations concernant la sécurité des métadonnées (l'accès de l'utilisateur autorisé et son mot de passe).

Étape 5 : Exécution des requêtes locales

Une requête locale LQ est envoyée, par le biais du réseau, à son destinataire pour être traitée par la base de données locale. Après traitement, les résultats sont retournés à l'expéditeur.

● étape 5.1 :

Un message est produit pour chacune des requêtes locales qui ont été générées dans l'étape 4. Ce message contient le destinataire, sa priorité, des métadonnées et la requête locale en traitement.

● étape 5.2 :

Ces messages sont transmis aux systèmes locaux appropriés par le moniteur de réseau.

● étape 5.3 :

Le moniteur du *shell* du système local reçoit le message et expédie la requête au SGBD correspondant.

● étape 5.4 :

Les résultats obtenus, après l'exécution de la requête locale, sont retournés au point d'origine pour post-traitement.

Les métadonnées requises sont : les protocoles de communication entre les systèmes locaux.

Étape 6 : Intégration des résultats

Les résultats des requêtes locales (LQ) doivent être interprétés et assemblés selon les conditions de jointure globales ($< GJC >$) résultants de l'étape 3.4. L'équivalent logique des attributs peut être implémenté directement en termes de format et d'échelle, et être encodé dans différents systèmes locaux. Supposons $R = \{R_1, R_2, \dots, R_n\}$ comme étant les résultats d'un ensemble de requêtes locales.

```
Result_Integrator( $R, < GJC >$ )
begin
   $i = 1$  ;
  FinalResult =  $R_i$  ;
   $R = R - R_i$  ;
  while  $R \neq \emptyset$  ;
    begin
       $i = i + 1$  ;
      Convert_Equivalent_Data(FinalResult,  $R_i, < GJC >$ ) ;
      FinalResult = Join(FinalResult,  $R_i, < GJC >$ ) ;
       $R = R - R_i$  ;
    end ;
  end ;
```

Convert_Equivalent_Data() est une fonction qui :

1. identifie l'existence d'équivalence(s) de données entre deux attributs impliqués dans la condition de jointure globale. Les métadonnées requises sont : les informations d'équivalence de données ;

2. trouve la règle de conversion correspondante. Les métadonnées requises sont : la connaissance de la métadonnée et la règle de conversion ;
3. déclenche le processeur de règles qui en retour va lancer la règle pour effectuer la conversion.

Join() est une fonction qui joint deux tables selon les conditions de jointure.

En plus, si chacun des attributs dérivés est sélectionné dans la requête globale, leur valeur est calculée après l'assemblage des résultats locaux. Les métadonnées requises sont : les équivalences de données et les connaissances contextuelles (règles de conversion, règles d'opération).

5.2.2 Modèle minimal pour une base de connaissances en direct

Définition : Les métadonnées de l'entreprise, telles que définies dans l'algorithme précédent et dans le modèle du GIRD, sont les connaissances requises lors de la formulation d'une requête globale et de son traitement, peu importe si cette requête est créée par un utilisateur ou par une application. Les métadonnées sont organisées en un modèle permettant l'identification et la caractérisation des connaissances nécessaires pour une opération de requête globale.

Connaissances pour la formulation d'une requête globale

- **Le modèle global de l'entreprise**

Il s'agit d'un modèle représentant les ressources informationnelles de l'entreprise. Le nom des applications ou de leurs fonctions ainsi que les entités et leurs relations sont particulièrement nécessaires lors la navigation du modèle ; le format et les domaines des attributs sont utilisés pour la spécification des conditions de sélection ; les dépendances fonctionnelles permettent de vérifier l'intégrité des conditions de sélection ; les clés primaires et les clés étrangères servent à déterminer les conditions de jointure implicites.

- **L'équivalence des données**

Ces connaissances sont nécessaires dans la plupart des étapes pour convertir et identifier les différentes définitions de données.

- **Connaissances contextuelles**

Les règles d'affaires, décrivant l'utilisation projetée des attributs et les besoins des utilisateurs, sont aussi utilisées pour vérifier les ambiguïtés au niveau des conditions de sélection.

Connaissances pour l'optimisation et la décomposition d'une requête globale

- **L'équivalence des données**

- **Implantation des modèles**

Il représente la méthode d'entreposage physique des attributs logiques qui est, soit la grandeur des fichiers, soit les tables de relation qui sont utilisées pour la décomposition et l'optimisation.

Connaissances pour la génération de la requête globale

- **Implantation des modèles**

Il correspond aux métadonnées décrivant l'environnement de manipulation des données locales et les chemins d'accès des générateurs de langage utilisés, ainsi que l'entête du programme de requête.

- **La sécurité des métadonnées**

Les privilèges d'accès de l'utilisateur sont utilisés pour déterminer si le recouvrement d'une requête est permis pour celui-ci. Le mot de passe est nécessaire pour obtenir les autorisations d'accès.

Connaissances pour l'intégration des résultats

- **L'équivalence des données**

- **Connaissances contextuelles**

Les règles de conversion et les règles opérationnelles sont utilisées pour résoudre les conflits entre les définitions des données et pour traiter les attributs dérivés.

5.3 Utilisation de MDB pour l'implantation d'une assistance en direct

Les méthodes de MGQS ont pour but d'utiliser la base de métadonnées pour fournir une assistance intelligente en direct, facilitant ainsi la formulation et le traitement des requêtes globales.

5.3.1 Formulation de la requête

Navigation du modèle

Il s'agit d'une approche par laquelle les usagers accèdent aux métadonnées de l'entreprise et les articulent directement en termes de modèle d'information pour la formulation des requêtes. Le contenu pertinent et la sémantique des systèmes hétérogènes sont fournis interactivement. La syntaxe et les erreurs lexicales sont réduites grâce à l'interface de requête interactive. Les erreurs sémantiques les plus communes causées par la

traduction syntaxique, ou par l'interprétation des modèles informationnels selon les approches conventionnelles indirectes, sont évitées par l'utilisateur, puisque celui-ci visualise et manipule directement les modèles de systèmes qui sont compréhensibles et sans équivoque. Par conséquent, les détails techniques (définitions des données, dépendances fonctionnelles, les règles d'affaires, etc.) ne sont plus des pré-requis que les utilisateurs doivent posséder pour opérer le système.

L'approche de navigation du modèle fournit aux utilisateurs une manière logique de scruter le modèle informationnel. Dans le processus, l'utilisateur recherche et sélectionne les concepts de modélisation et les données pour ainsi représenter la requête désirée. De cette façon, la navigation du modèle est conçue selon les caractéristiques conceptuelles du gabarit (méthode TSER) et les associations logiques entre les modèles d'information emmagasinés dans la base de métadonnées. Tel que le montre la figure 5.1, une application (système local) est premièrement représentée en utilisant la hiérarchie des SUJETS et des CONTEXTES. Le contenu des données est défini avec les SUJETS, tandis que les CONTEXTES définissent seulement la connaissance contextuelle. Chacun des SUJETS feuilles est transformé en un sous-modèle structural. Les sous-modèles de toutes les applications sont alors consolidés en un modèle d'entreprise global, c'est-à-dire, qu'ils sont fusionnés au niveau du modèle global.

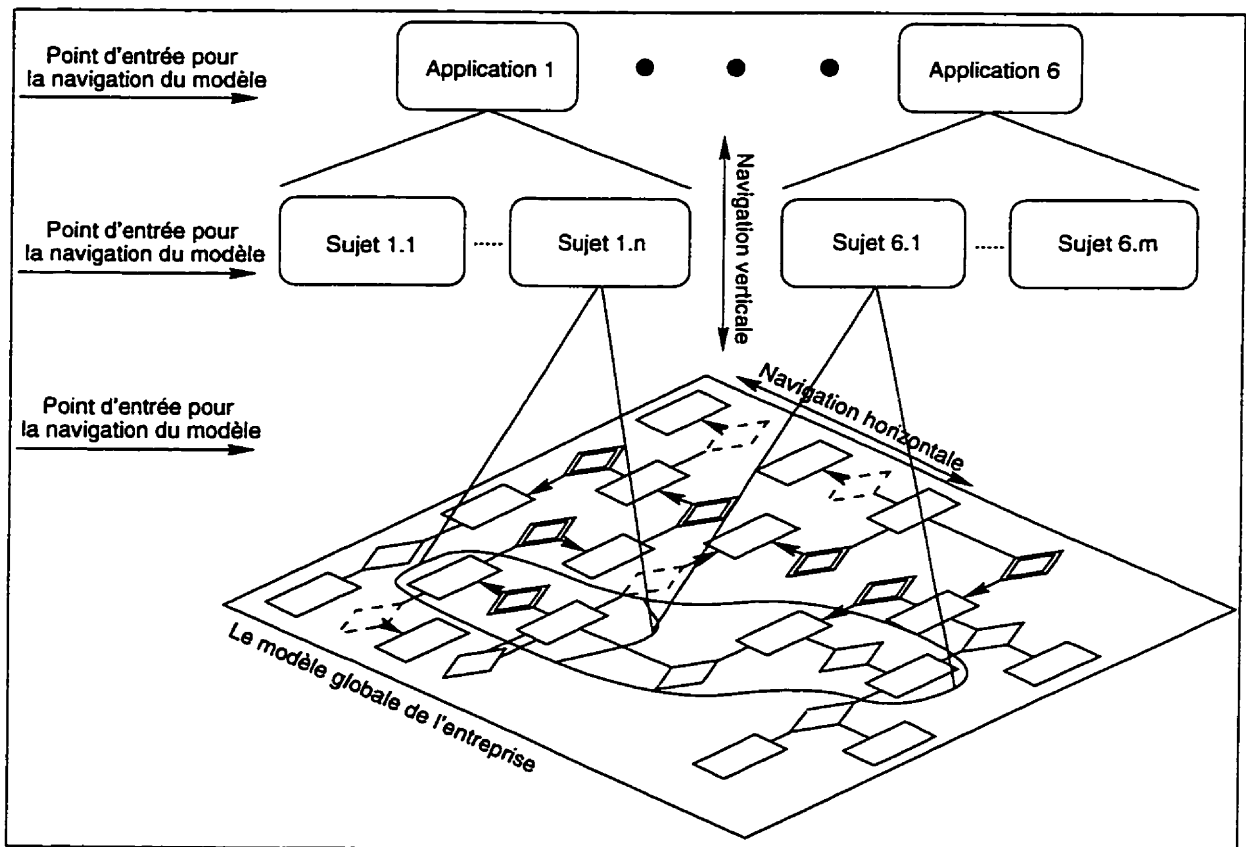


Figure 5.1 : Les deux méthodes de navigation du modèle

Deux méthodes de base sont employées pour la navigation du modèle, soient la navigation verticale et la navigation horizontale. La méthode verticale spécifie le domaine de recherche au niveau vertical. Par exemple : si un processus X est spécifié comme étant une application initiale, le parcours sera restreint à cette application. La méthode horizontale permet à un utilisateur de parcourir horizontalement le modèle global (ex : un réseau) et ce, à partir d'une entité opérationnelle (*Operational Entity*; OE) ou d'une relation plurale (*Plural Relationship*; PR) vers n'importe quelle autre OE/PR immédiatement inter-reliée.

La méthode horizontale débute seulement lorsqu'une OE/PR est sélectionnée. Une fonction appelée `get_related_entrel()` est utilisée pour retrouver la liste de toutes les OE/PR qui sont immédiatement inter-reliées à la OE/PR courante. Ainsi, l'utilisateur peut parcourir le modèle global selon les associations logiques entre les concepts modélisés (OE/PR). Cette méthode n'est pas limitée par les frontières logiques d'une application ou d'un sujet. Par conséquent, un utilisateur pourrait parcourir le modèle global d'une application vers une autre, sans devoir explicitement spécifier ses intentions. Une exception se produit lorsqu'une relation obligatoire (*Mandatory Relationship*; MR) ou une relation fonctionnelle (*Functional Relationship*; FR) est inter-reliée à la OE/PR. Dans cette circonstance, la fonction retourne la OE/PR qui se trouve à l'autre bout de la FR/MR.

La méthode verticale fournit un accès facile aux données qui sont en corrélation avec la requête globale. L'utilisateur peut limiter l'aire de navigation en parcourant le modèle informationnel selon l'un des trois niveaux suivants, soit : (1) le niveau application, (2) le niveau sujet ou (3) le niveau OE/PR (cf. figure 5.1) ; cette restriction est optionnelle. Tout dépendant de son expérience avec le modèle informationnel, l'utilisateur peut pointer sur une donnée en spécifiant verticalement l'un des trois niveaux et naviguer au travers d'une liste plus longue de données contenant des spécifications moins précises. En plus de cela, l'utilisateur n'est pas limité à employer une méthode ou l'autre. Une utilisation adéquate des deux méthodes, selon la connaissance qu'ont les usagers du modèle informationnel, permettra d'obtenir de meilleurs résultats lors de la formulation de requêtes globales.

Validation des requêtes

Un champ particulièrement significatif est la validation sémantique des requêtes globales. La sémantique est nécessaire pour la génération des requêtes selon les frontières techniques locales. MGQS détecte et prévient la spécification des conditions de sélection inconsistantes sémantiquement et de conditions de jointure ayant des domaines incompatibles, ce qui est réalisé en examinant le contenu du dictionnaire des métadonnées (ex : les dépendances fonctionnelles et les règles d'affaires d'une connaissance contextuelle). Une assistance est spécialement fournie pour les trois problèmes potentiels suivants :

- **Les domaines incompatibles des conditions**

Les propriétés du domaine sont utilisées pour guider l'utilisateur lors de la sai-

sie des valeurs de la condition de sélection. Si le domaine D^I des attributs I impliqué dans la condition de sélection est explicitement défini et emmagasiné en un ensemble de valeurs restrictives dans la base de métadonnées, l'utilisateur peut seulement sélectionner une valeur v de cette série où $v \in D^I$. Si le domaine D^I n'est pas défini explicitement, MGQS va seulement permettre la comparaison entre des valeurs ayant le même format (ex : réel, entier, caractère, etc.). Similairement, si la condition de sélection implique une comparaison entre deux attributs $I1$ et $I2$ ayant des domaines respectifs D^{I1} et D^{I2} , alors elle sera acceptée seulement si $D^{I1} \cap D^{I2} \neq \emptyset$. Dans le cas où un ou les deux domaines ne sont pas explicitement définis, MGQS va rejeter la condition si les formats des deux données sont incompatibles.

- **Les inconsistances sémantiques des conditions**

Les inconsistances sémantiques peuvent se produire dans une condition de sélection d'une requête globale. Considérons par exemple les conditions de sélection suivantes : `PARTID = "PZ51" AND WEIGHT = 100`. Ces conditions sont sémantiquement inconsistantes, ou du moins redondantes, si `WEIGHT` est en dépendance fonctionnelle avec `PARTID` (`PARTID  $\Rightarrow$  WEIGHT`). Les dépendances fonctionnelles représentées par le modèle global de l'entreprise sont utilisées pour détecter et prévenir les inconsistances potentielles. La base logique de ce principe est que pour une donnée i qui est en dépendance fonctionnelle avec un ensemble d'attributs I (où $I \Rightarrow i$, et $i \notin I$), la condition de sélection concernant i n'est pas permise si tous les membres de I sont liés par des conditions déjà existantes.

- **Les conditions discordantes au niveau des règles d'affaires**

Les règles d'affaires sont une sous-série des connaissances contextuelles et sont emmagasinées en utilisant le format des règles de la base de métadonnées. Un avertissement devrait être émis si la condition définie est en contradiction avec une règle d'affaires ; par exemple, une règle d'affaires spécifiant que "Une opération de mouture sera débutée seulement si elle peut être complétée dans la même journée."

```
If WO_SEQ.WS_ID = '0001'
    WO_SEQ.START_DATE := WO_SEQ.END_DATE
```

MGQS va émettre un avertissement à la condition conflictuelle suivante :

```
WO_SEQ.WS_ID = '0001' AND
WO_SEQ.START_DATE  $\neq$  WO_SEQ.END_DATE
```

5.3.2 Conversion à base de règles

Un autre exemple utilisant la base de connaissances en direct est la conversion automatique des formats entre des attributs hétérogènes équivalents à partir d'une base de règles. La fonction `find_convert_rules(itemA, itemB)` recherche la table d'équivalence

de la base de métadonnées pour obtenir les règles de conversion nécessaires à la conversion de la valeur *itemA* dans le format *itemB* (cf. figure 5.3). Par exemple : dans la figure 5.2, *find_convert_rules(itemA, itemB)* retourne deux règles (règle2 et règle4), qui vont premièrement convertir la valeur *itemA* dans le format de *itemC* en utilisant règle2, et par la suite vers le format de *itemB* en utilisant règle4. La fonction retourne "NULL" si aucune conversion est nécessaire pour la comparaison entre *itemA* et *itemB*. La fonction *convert_item()* déclenche le processeur de règles qui va chercher et lancer les actions de la règle de conversion. Les deux fonctions font partie des ressources logicielles dans la base de métadonnées.

Table des Équivalences			
Itemcode	Eqitemcode	reverse_by	convert_by
ItemA	ItemC	Règle1	Règle2
ItemB	ItemC	Règle4	Règle3

Figure 5.2 : Table des équivalences

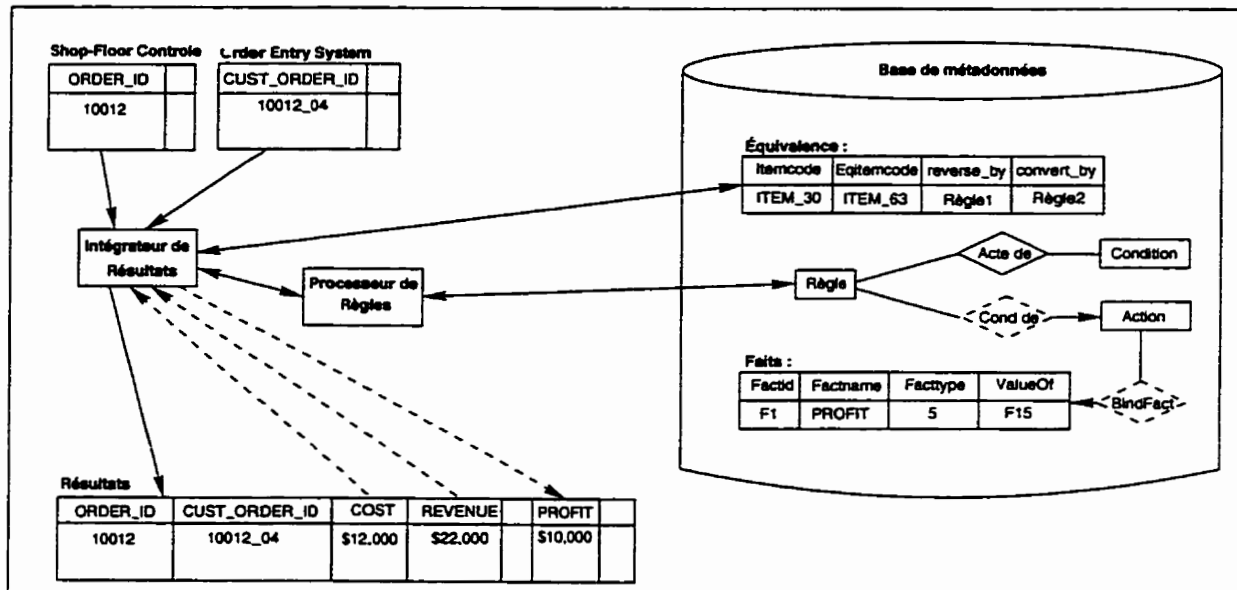


Figure 5.3 : Conversion et dérivation des données selon l'approche à base de règles

En 1991, le laboratoire de recherche CIM (*Computer Integrated Manufacturing*), de l'Institut *Rensselaer*, a fourni un environnement avec un niveau de complexité raisonnable pour tester le prototype de MGQS et les concepts du modèle d'assistance. Tous les objectifs majeurs (partage de l'information, autonomie des systèmes locaux et modèle d'assistance) du MGQS ont été réalisés et testés grâce au prototype, ce qui permet d'affirmer qu'ils sont réalisables et exploitables en milieu de travail. La plupart des fonctionnalités envisagées ont été implantées, à l'exception de la vérification des ambiguïtés et de l'interrogation des attributs dérivés. En plus, la capacité de conversion

des données a démontré qu'une utilisation de la connaissance contextuelle et du processeur de règles peut fournir une assistance intelligente en direct. Les mêmes méthodes peuvent être appliquées au niveau de la vérification des ambiguïtés et de l'interrogation des attributs dérivés.

Il existe cependant encore des besoins à explorer au niveau de la performance, surtout le temps d'exécution pour la conversion des données et les opérations des règles qui y sont rattachées. L'étude empirique ne permet pas l'esquisse de conclusions scientifiques complètes à cause du manque d'observations. Les résultats primaires permettent par contre de croire que ces opérations ne semblent pas créer de congestion (*bottle-neck*) dans le système global. Une explication possible de ce phénomène est peut-être le fait que la conversion des données dans le MGQS est effectuée dans les noeuds locaux et ce, d'une manière distribuée par les systèmes locaux en utilisant simultanément les représentations globales communes comme but. La plupart des autres règles sont traitées selon une conception concurrente similaire en utilisant les *shells* distribués de la base de métadonnées pour les systèmes locaux.

5.3.3 Langage de requête globale

La formulation des requêtes globales au niveau de la base de métadonnées se fait en MQL. Ce langage permet à l'utilisateur d'interroger une collection de systèmes d'informations autonomes de façon non-procédurale. Le MQL permet de localiser des données résidant dans différents sites et sous différents systèmes d'exploitation. En général, il offre la possibilité (1) de retrouver des métadonnées, (2) de spécifier des requêtes de jointure, (3) de formuler des requêtes impliquant des données avec de multiples définitions et (4) de minimiser les détails techniques pour l'utilisateur dans sa formulation. Les métadonnées que le MQL utilise sont de trois types :

1. **Métadonnées locales**
elles concernent la modélisation et l'implantation d'une application :
2. **Métadonnées globales**
elles touchent les ressources informationnelles de l'entreprise et les relations entre les applications ;
3. **Métadonnées détaillées**
elles portent sur les spécificités des structures de données.

Chapitre 6

Architecture concurrente adaptable

Pour faciliter l'intégration des différents systèmes d'information d'une entreprise, qui sont complexes et diversifiés, une architecture concurrente adaptable, telle que ROPE (*Rule-Oriented Programming Environment*), peut s'avérer une solution intéressante. Dans ce chapitre nous présentons une description des composantes de cette architecture et des explications concernant l'approche de ROPE.

6.1 Architecture concurrente pour l'intégration adaptable

L'intégration des applications recherchée par les entreprises doit être adaptable pour assurer l'autonomie de ses systèmes d'information. Les résultats préliminaires pour le EIM (*Enterprise Information Manager*) ont permis d'établir plusieurs bonnes architectures, utilisant des principes tels que les schémas intégrés, le traitement des bases de connaissances et les *shells*. Ces résultats ont cependant laissé sans réponse une question fondamentale concernant l'intégration adaptable : comment développer l'architecture globale d'un système lorsque des applications locales ont été modifiées, détruites ou ajoutées ?

Peu de ces systèmes locaux semblent être en mesure de fournir une solution à ce problème. Il est impératif pour l'adaptabilité de bien évaluer le degré d'autonomie locale offerte par une solution ; l'autonomie est l'une des caractéristiques les plus significatives de la base de métadonnées. Elle est associée au niveau de la coopération nécessaire pour le contrôle des différentes opérations à travers le système intégré. Par contre, une grande coopération implique une autonomie locale réduite. Du point de vue de l'utilisateur, l'autonomie locale est toujours indispensable. D'un point de vue technique, à cause de l'hétérogénéité, il est difficile d'imposer les différents comportements nécessaires au bon fonctionnement des applications. Une application est considérée comme étant complètement autonome lorsqu'elle n'a pas besoin [Hsu96] :

1. de se conformer à un schéma intégré (convertir un schéma hérité à un standard global quelconque) ;

2. de coopérer directement avec un contrôleur global (sérialisation des programmes ou toute forme de supervision directe des transactions et de leurs communications) ;
3. d'avoir recours à un usager pour traiter des connaissances spécifiques à l'environnement global sur lequel les systèmes locaux doivent opérer.

Les résultats disponibles sur les multibases de données ne supportent pas encore la pleine autonomie locale des applications.

Un autre problème évident est celui de la performance, qui est directement proportionnelle à la capacité du traitement concurrentiel des systèmes locaux, ou réciproquement, à l'exigence de la présence d'un contrôleur global. À un certain point, dépendant du nombre de canaux qui est offert aux systèmes locaux, virtuellement toutes les opérations gérant les appels de mise à jour des données démontrent une sérialisation globale de ces transactions. Les facteurs provoquant ce goulot d'étranglement sont les problèmes d'échelonnement des réseaux, les requêtes distribuées et le traitement des transactions. Les résultats préliminaires ont été obtenus à partir d'expérimentations effectuées sur des réseaux de petite envergure et avec un nombre limité de transactions. De cette manière, on se questionne sur la maniabilité des solutions dans un environnement plus étendu. La concurrence et le contrôle cohérent sont au coeur de la notion de traitement distribué.

Dans la littérature informatique, le critère le plus accepté pour atteindre la cohérence entre les différentes tâches concurrentes est la **sérialisation**, c'est-à-dire le modèle de Von-Neumann. John Von-Neumann, mathématicien de génie, a rédigé un document où il suggérait [Sanders83] : (1) l'emploi du système de numération binaire dans la construction des ordinateurs et (2) le stockage interne des instructions de l'ordinateur aussi bien que des données manipulées. Bien que l'origine de ces idées soit sujette à caution, elles constituaient néanmoins la philosophie de base de conception des ordinateurs jusqu'à nos jours. La sérialisation des transactions est dirigée par les objectifs qui assurent l'exactitude (*correctness*) de toutes les données du système. Les critères de base pour la sérialisation des méthodes de contrôle sont les mêmes utilisés par un gestionnaire de multibases de données que par un gestionnaire d'une simple base de données. Cependant, un système de gestion de multibases de données entrave sérieusement la performance des applications locales lors de la sérialisation des transactions, car pour atteindre ce but, le gestionnaire doit verrouiller certaines données durant le traitement de la transaction. Avec l'avènement de nouvelles applications et une demande toujours grandissante au niveau de la disponibilité des données, il devient impératif de définir un nouveau critère pour l'exactitude des données (*data correctness*).

Pour répondre aux défis fondamentaux et implanter une base de métadonnées conceptuelle, un environnement de programmation orienté-règles, qui se nomme ROPE (*Rule-Oriented Programming Environment*), a été développé pour l'implantation et la gestion de l'architecture concurrente de cette base. L'objectif principal de ROPE est

la décomposition et la distribution des connaissances contextuelles pertinentes contenues dans la base de métadonnées, pour augmenter le traitement concurrent et faciliter l'adaptabilité de l'architecture dans un environnement de contrôle, distribué et autonome. L'objectif mentionné ci-haut est atteint par la création de *shells* locaux contenant la connaissance contextuelle. Cet environnement permet de :

1. contrôler la logique des systèmes locaux sans avoir recours à l'aide d'un contrôleur central ;
2. accroître les connaissances distribuées des applications pour obtenir un comportement global, et ce, sans devoir les modifier ;
3. transférer les connaissances nécessaires entre la base de métadonnées et les applications locales ainsi que les connaissances inter-applications nécessaires.

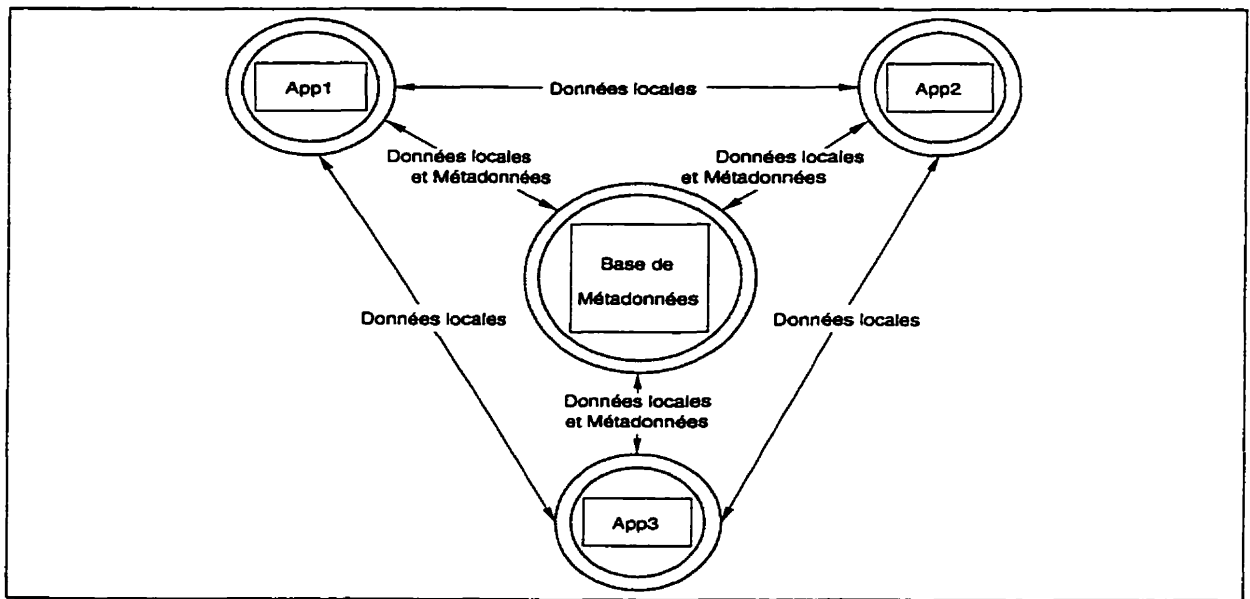


Figure 6.1 : L'architecture concurrente utilisant la base de métadonnées

La base de métadonnées elle-même fournit un modèle intégré de l'entreprise pour les différents systèmes d'information, leurs bases de données, et les interactions entre les différents systèmes (les contenus informationnels et leurs connaissances contextuelles), à l'aide d'une architecture concurrente (cf. figure 6.1). L'approche de la base de métadonnées possède deux caractéristiques [Hsu91] :

1. Elle s'occupe des détails techniques et/ou de la hiérarchisation des schémas intégrés, facilitant ainsi la génération de requêtes globales par les utilisateurs.
2. Elle distribue la connaissance contextuelle, ce qui augmente le rendement des systèmes locaux lors de la mise à jour des données et de la communication entre ceux-ci. Le tout doit être effectué sans le contrôle d'une base de données centralisée.

3. Elle incorpore l'héritage des modèles locaux modifiés, ou des nouveaux modèles, en une structure générique de métadonnées pour supporter l'évolution du système global sans devoir effectuer la reconceptualisation des systèmes locaux ou la recompilation de ceux-ci.

Les *shells* permettent ainsi l'implantation de la connaissance locale distribuée qui, en retour, est gérée par la base de métadonnées.

L'architecture concurrente du modèle de la base de métadonnées (cf. figure 6.1) est employée comme étant la structure de base pour développer les nouvelles méthodes et répondre aux exigences de ce genre d'architecture. Au lieu d'avoir des applications modifiées pour s'adapter à une méthode de contrôle particulière (ex : la sérialisation), on utilise la connaissance concernant le modèle (règles d'opération et règles d'intégrité) dans la base de métadonnées pour construire des *shells* de contrôle personnalisés autour de chacune des applications. Ces *shells* peuvent ainsi opérer indépendamment et concurremment avec leurs propres connaissances de contrôle ; la connaissance de contrôle est coordonnée par la base de métadonnées. La fonctionnalité de chacun des *shells* dépend des connaissances spécifiques de leur application particulière ; par conséquent, chaque *shell* est personnalisé. Cependant, la création de ces *shells* peut s'effectuer de manière à obtenir des structures de base identiques, avec comme seule différence des connaissances spécifiques à leurs processus. Les *shells* peuvent [Babin93] :

1. communiquer avec chacun de leurs confrères en ayant un comportement concerté ;
2. réagir aux changements dans les applications locales qui ont une répercussion globale ;
3. traiter les connaissances gérées par la base de métadonnées.

De plus, les *shells* devraient être efficaces lors de leur implantation, de leurs modifications et de leur exécution. ROPE est une méthode développée pour satisfaire à ces exigences.

6.2 ROPE : méthode de gestion informationnelle dans l'entreprise

La méthode ROPE permet de développer la technologie des *shells* nécessaire à l'architecture concurrente de la base de métadonnées intégrée dans l'entreprise. Cette méthode définit :

1. comment les règles sont emmagasinées dans les *shells* locaux (formats et représentation) ;
2. comment les règles sont traitées, distribuées et gérées ;
3. comment les *shells* interagissent avec leur application ;

4. comment les *shells*, y compris celui de la base de métadonnées, interagissent entre eux ;
5. comment les *shells* sont structurés.

De plus, l'approche ROPE impose trois principes, soit :

1. les règles représentant le traitement logique sont séparées du code source du programme et sont insérées dans une section à base de règles pour faciliter ainsi les modifications possibles ;
2. les communications entre les *shells* sont effectuées par un système de messages ;
3. le traitement des règles et le système de messages sont implantés dans un environnement local (processeur de règles et de messages) et sont reliés aux *shells*.

De cette manière, les *shells* locaux sont invisibles aux utilisateurs, mais contrôlent le comportement externe des systèmes locaux. Dans ROPE, les règles possèdent une des structures suivantes :

1. [Action]
2. IF [Expression] THEN [Action]
3. IF [Event] THEN [Action]
4. IF [Event] AND [Expression] THEN [Action]

6.2.1 Modèle ROPE

Dans l'environnement de ROPE, les capacités de gestion des informations dans l'entreprise peuvent être réalisées de trois manières différentes, c'est-à-dire : (1) par le traitement des requêtes globales, (2) par les mises à jour globales et le traitement événementiel ou (3) par l'adaptabilité et la flexibilité.

Traitement des requêtes globales

ROPE traite les requêtes globales au niveau du noeud local. La figure 6.2 démontre les différents parcours possibles des requêtes et des flux de données dans le *shell*. Le système de requêtes globales du gestionnaire de la base de métadonnées (*MetaData-Base Management System*; MDBMS) qui est représenté par le cercle du milieu de la figure 6.2, envoie des demandes pour l'exécution des requêtes locales dans les différents *shells* concernés. Les données résultantes sont, par la suite, assemblées par le système de requêtes globales (*Global Query System*; GQS). Les différents *shells* locaux peuvent aussi initier des demandes de requête globale, qui seront traitées par le système de gestion de la base de métadonnées.

Le traitement général se déroule de la façon suivante. Un utilisateur formule une requête globale, soit à partir du gestionnaire de la base de données (MDBMS), soit à

partir de l'interface de requête globale localisée dans l'environnement de l'application locale. Cette requête est envoyée au gestionnaire de la base de métadonnées où elle est traitée ; sa syntaxe et sa sémantique sont validées ; après cela, les requêtes locales sont générées. Les requêtes sont alors envoyées à leur *shell* respectif où elles seront exécutées par le SGBD local. Les résultats des requêtes locales sont ultérieurement renvoyés au gestionnaire de la base de métadonnées pour y être assemblés et post-traités.

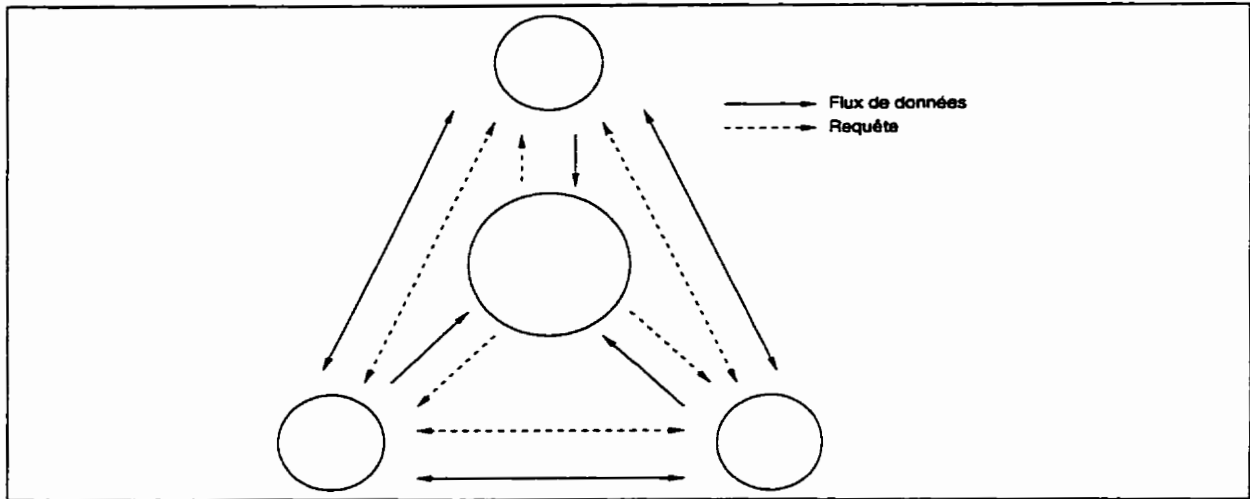


Figure 6.2 : Le traitement des requêtes globales

Mises à jour globales et le traitement événementiel

ROPE traite les mises à jour globales de l'information et les flux d'information orientés-événements entre les bases de données locales ; cette façon implique l'utilisation directe des sections à base de règles des *shells* de ROPE, mais elle ne requiert pas d'initiation ou de contrôle venant du gestionnaire de la base de métadonnées (cf. figure 6.3). Les parties du *shell* concernées par cette classe de capacités sont le contrôleur d'applications et l'interface de la base de données locale. Les règles sont déclenchées et basées sur le statut de l'application ou de la base de données. Ces règles peuvent nécessiter la mise à jour automatique de la base de données locale. Le *shell* formulera des commandes de modification qui seront envoyées aux autres *shells* pour effectuer les mises à jour nécessaires.

Adaptabilité et flexibilité

ROPE assure l'adaptabilité (1) en sectionnant en modules les comportements globaux des systèmes locaux et en les stockant dans la base de métadonnées, ce qui permet de supporter les *shells* individuels ; (2) en stockant ces connaissances sous la forme de règles dans les *shells* ; (3) en traitant directement ces règles dans le langage local ; et (4) en effectuant la mise à jour automatique de ces règles lorsque leur comportement logique est modifié dans la base de métadonnées. De cette manière, le traitement logique de l'information (les règles d'intégrité sémantique et les connaissances contextuelles) des

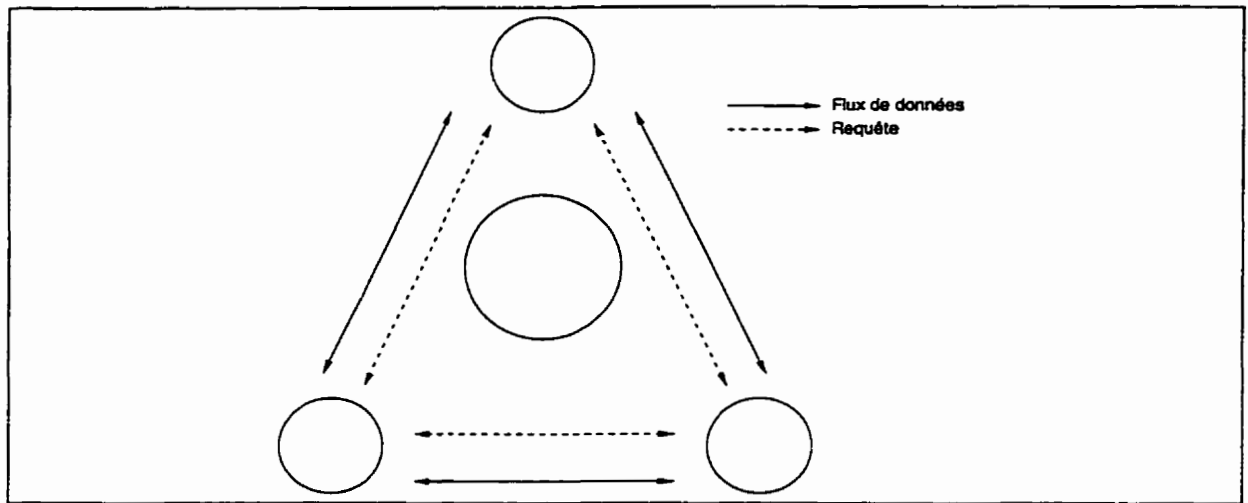


Figure 6.3 : Le traitement automatique de la connaissance de l'entreprise

différents systèmes est géré par le biais du traitement à base de règles. Cette approche facilite l'adaptation des *shells* lorsque de nouvelles situations se produisent, et fournit ainsi une interface flexible entre les différentes applications et le gestionnaire de la base de métadonnées. Étant représenté par le contenu de la base de métadonnées, tout changement dans le comportement global des systèmes locaux va déclencher la mise à jour des règles présentes dans celle-ci ou va provoquer la création de nouvelles règles dans une forme neutre. Par la suite, ces modifications seront propagées à tous les *shells* pertinents et implantées dans la section à base de règles du *shell* local, sous une forme locale par ROPE (cf. figure 6.4). Seul le traitement des métadonnées est impliqué dans la gestion des règles.

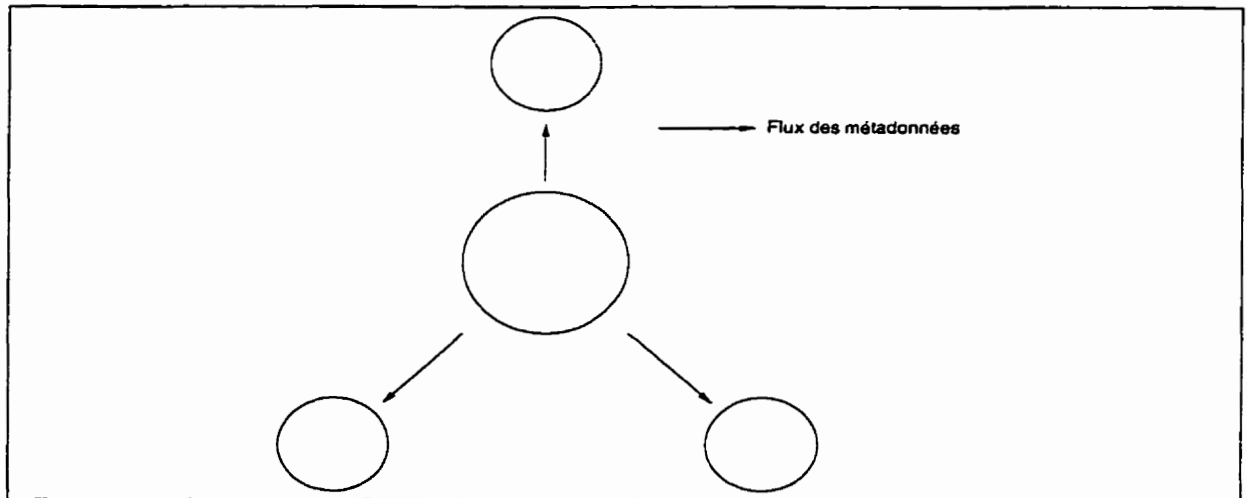


Figure 6.4 : Le processus de mise à jour des règles

Lorsqu'un nouveau système est ajouté ou qu'un système existant est détruit dans l'environnement global, seulement trois étapes doivent être franchies pour effectuer

la mise à jour de manière modulaire : (1) la base de métadonnées doit être mise à jour (mise à jour des métadonnées concernées), (2) un nouveau *shell* doit être créé ou détruit et (3) les règles pertinentes dans les autres *shells* doivent être mises à jour. ROPE, en collaboration avec le système de gestion de la base de métadonnées, facilite et automatise ces trois étapes.

Modèle informationnel

Le modèle informationnel de la méthode ROPE est la pierre angulaire du *shell*. Basé sur la description fonctionnelle faite précédemment, il permet la création de *shells* et la justification des informations contenues dans ceux-ci. La connaissance pertinente dans la base de métadonnées est distribuée dans ces *shells* lorsque les règles globales sont décomposées pour la distribution. Une règle est disséquée en utilisant les algorithmes de décomposition. Cela va générer une série de requêtes locales et de sous-règles qui seront envoyées à différents *shells*. Cependant, une attention particulière doit être portée lors de l'entreposage de cette information dans les *shells* locaux pour ainsi minimiser le traitement requis pour chaque *shell*. Cela peut être réalisé en stockant séparément les éléments appartenant à une règle unique des éléments appartenant à plusieurs règles. Par conséquent, chaque *shell* doit avoir suffisamment d'information pour effectuer correctement ses tâches locales et être capable d'interagir avec ses congénaires.

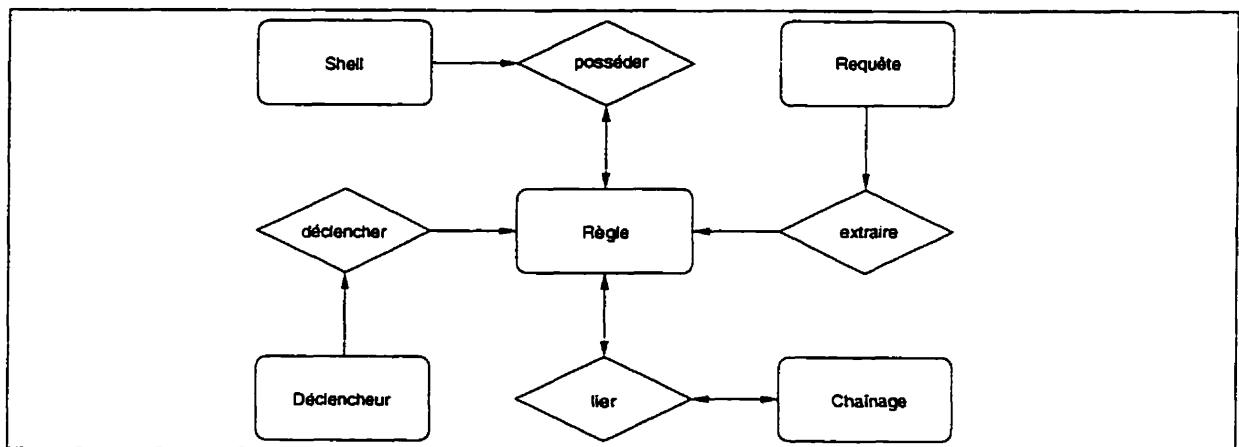


Figure 6.5 : Le modèle fonctionnel du *shell* de ROPE

Description du modèle fonctionnel de ROPE

Le modèle fonctionnel de ROPE est divisé en cinq parties informationnelles : (1) le *shell* lui-même, (2) l'information sur les règles, (3) l'information sur les déclencheurs, (4) les requêtes utilisées pour surveiller l'application locale dans le but d'extraire les données utilisées par les règles et d'emmagasiner le résultat des requêtes. et (5) l'information concernant le chaînage des règles. Le modèle fonctionnel est illustré à la figure 6.5.

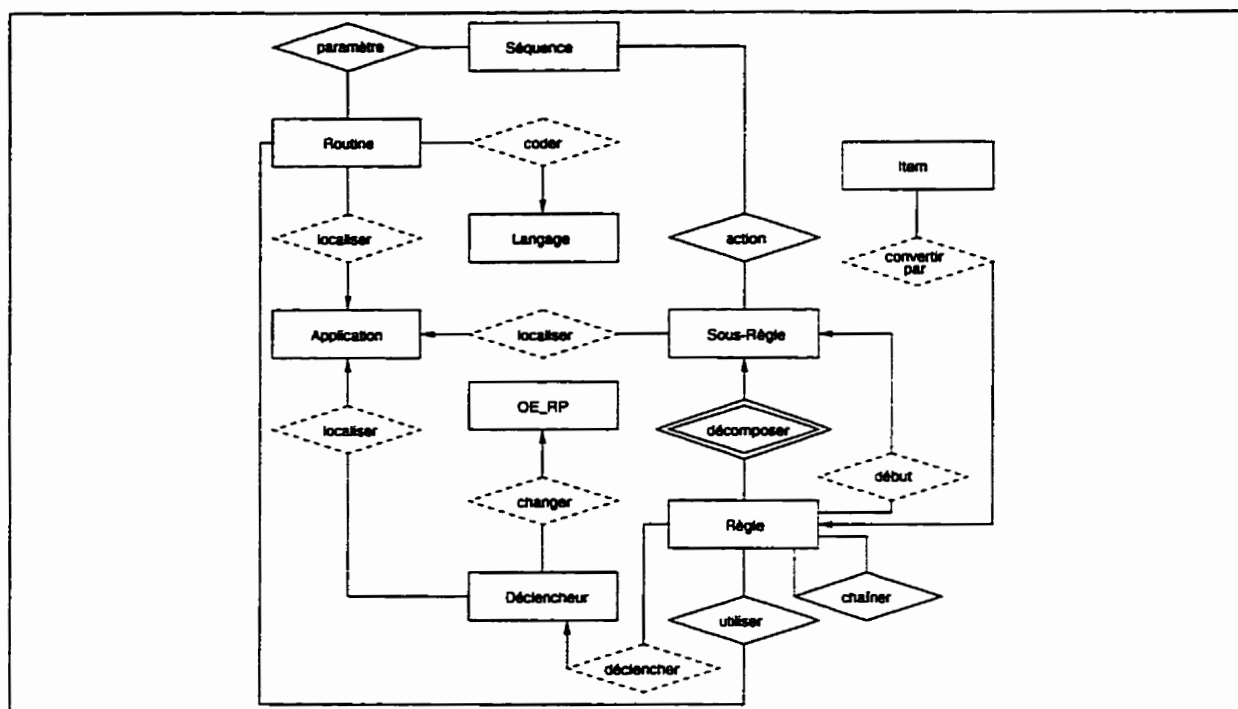


Figure 6.6 : Le modèle structurel du *shell* de ROPE

Description du modèle structurel de ROPE

Le modèle structurel de ROPE est créé à partir des dépendances fonctionnelles extraites du modèle fonctionnel. Ce modèle est utilisé pour emmagasiner l'information nécessaire à ROPE lors de l'exécution des règles globales. Le modèle structurel de ROPE est représenté à la figure 6.6.

6.2.2 Structure statique de ROPE

La structure statique de ROPE est composée de 7 éléments (cf. figure 6.7) qui sont identiques d'un *shell* à l'autre et sont définis comme suit :

- **Segment à base de règles**

Le segment est l'élément clé du *shell* puisqu'il contient les règles. Logiquement, il implante l'intersection du modèle à base de règles et la logique de l'application locale ; physiquement, il emploie une structure de données en corrélation avec l'environnement logiciel local.

- **Contrôleur de réseau**

Le contrôleur est l'interface du *shell* avec le réseau de communications : il est en quelque sorte la fenêtre ouvrant sur le monde. Le contrôleur de réseau reçoit les messages en entrée et les envoie ensuite au processeur règles/messages. Il est également responsable de l'envoi de messages à d'autres *shells*.

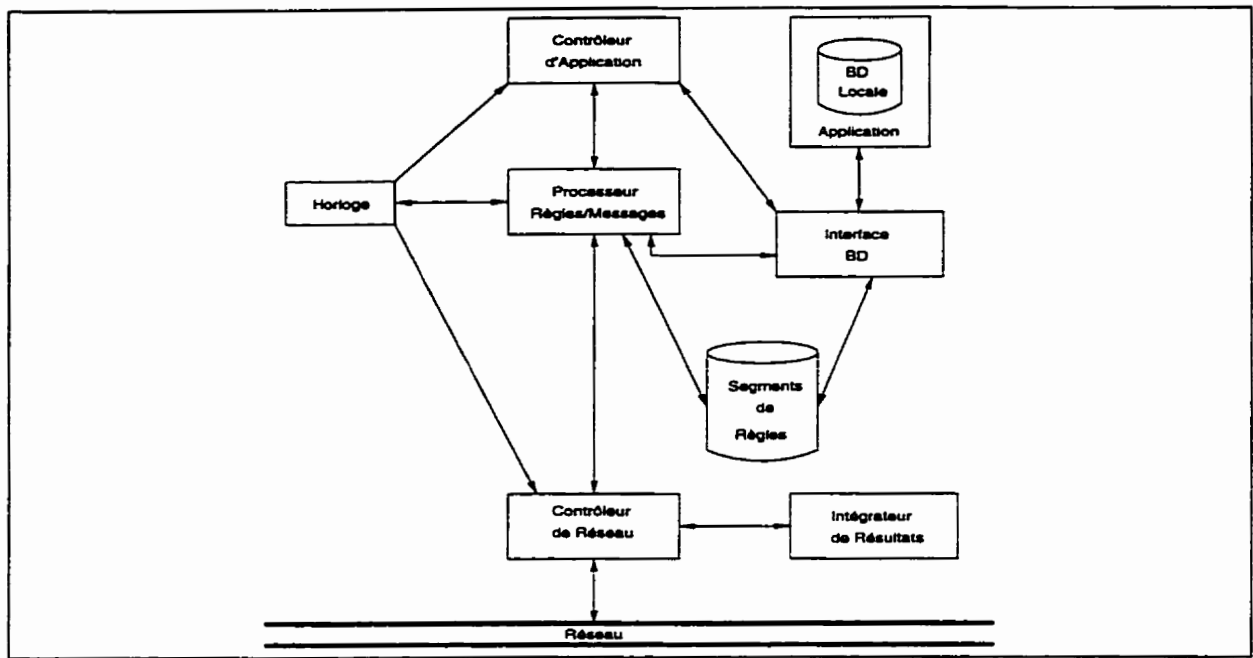


Figure 6.7 : La structure statique d'un *shell* de ROPE

- **Contrôleur d'application**

Ce module est en communication avec l'application locale. Il surveille le comportement de l'application en détectant les changements afin de les signaler au processeur règles/messages.

- **Processeur de règles/messages**

Le processeur de règles/messages est le moteur d'inférence du *shell*. En se basant sur les événements qu'il reçoit du contrôleur de réseau, du contrôleur d'application locale, de l'horloge et de l'application locale, il déclenche les règles emmagasinées dans le segment de règles ou exécute la fonction appropriée du *shell*. Le processeur de règles/messages est constitué de deux unités de traitement, une pour les messages et l'autre pour les règles. Le processeur de messages transige avec les messages de régularisation et les mises à jour dans le segment de règles. Le processeur de règles exécute des sous-règles du *shell*.

- **Horloge**

L'horloge gère la liste des événements temporels. Lorsqu'un événement doit être déclenché, l'horloge initie la commande associée à cet événement.

- **Intégrateur de résultats**

L'intégrateur est utilisé par le *shell* pour assembler les résultats des requêtes locales et pour réaliser d'autres transformations sur ces résultats. La fusion des résultats partiels est faite à partir du *script* d'intégration.

- **Interface de la base de données**

L'interface permet de convertir les valeurs locales en des valeurs globales et vice

versa. Cet élément reçoit deux types de requêtes : (1) extraction des données et (2) stockage des données. Généralement, le *shell* garde une liste des attributs à convertir ainsi que les règles de conversion nécessaires. L'équivalent global permet au *shell* de reconnaître les différentes représentations (nom, format ou structure) qui existent entre deux attributs sémantiquement équivalents.

6.2.3 Structure dynamique de ROPE

La création des *shells*, ainsi que l'exécution des tâches spécifiques à chacune des composantes, se fait par le développement d'algorithmes et de langages. Ces derniers représentent l'aspect dynamique de ROPE. Les algorithmes comprennent un noyau générique et une interface spécifique d'implantation qui représente, d'une manière appropriée, les différentes applications. Les algorithmes suggèrent la façon de structurer la connaissance et quelle savoir est nécessaire pour le fonctionnement des *shells* [Hsu et Babin93]. Il existe trois champs majeurs où les nouveaux langages sont nécessaires pour fournir les comportements globaux des applications concernées, c'est-à-dire : (1) la création des *shells* pour les nouvelles applications qui sont dans l'environnement intégré, (2) les opérations d'intégration devant être exécutées et (3) les communications *inter-shells*. En fonction de l'évolution de ROPE, trois classes de langages sont définies :

1. Langage de définition des shells

Les *shells* sont créés par un langage de définition, qui peut être exploité par le système de gestion de la base de métadonnées ou par l'utilisateur de ROPE. Le langage définit premièrement un *shell* en termes génériques dans un environnement global, et par la suite, utilise un générateur de code pour implanter la structure générique dans l'environnement local cible. Les éléments définis par ce langage sont :

- (a) les fonctions systèmes définissant les cinq éléments de la structure statique et leurs algorithmes correspondants ;
- (b) les spécifications définissant les interfaces avec les applications locales ;
- (c) les paramètres systèmes définissant les besoins des interfaces et des algorithmes du *shell*.

Ces concepts permettent la réalisation d'un *shell* générique ne possédant aucune dépendance système. Le résultat est transformé pour un environnement logiciel spécifique, et ce, par le biais du générateur de code, ce qui alloue un maximum de portabilité.

2. Langage de règles et de modélisation

Ce langage aide à décrire la connaissance contenue dans la base de métadonnées et les *shells*. Il y a deux types d'actions à considérer dans les règles :

- (a) les actions systèmes : ce sont des fonctions et des procédures comprises dans chaque *shell*, permettant de traiter et de générer les messages que le *shell* reçoit ou transmet aux autres *shells* d'applications ;

- (b) les actions utilisateurs : ce sont les autres fonctions et procédures impliquées dans les règles.

Le langage de modélisation de ROPE fournit les structures qui définissent : (1) les conditions de déclenchement d'une règle, (2) les actions à réaliser lorsqu'une règle est déclenchée, (3) les événements significatifs et (4) l'emplacement des données nécessaires pour exécuter une règle.

3. Langage et protocole de messages

Les messages permettent de réaliser la communication entre les différents systèmes et la connection du *shell* de gestion de la base de métadonnées à ceux des applications locales. Un message doit être composé d'un minimum d'information pour être considéré, soit : (1) un identificateur de message, (2) la fonction qui doit être exécutée lors de la réception du message ainsi que les paramètres nécessaires, et (3) le point d'origine et la destination du message. Les fonctions peuvent être classifiées comme étant les fonctions des métadonnées ou les fonctions des données. Les fonctions des métadonnées sont utilisées pour le segment de règles du *shell*. Les fonctions des données sont utilisées pour permettre la coopération entre les *shells* et la base de métadonnées. Les types de messages sont déterminés à partir de la tâche fonctionnelle du message. Deux types de messages sont définis :

- (a) type correspondant aux fonctions assurées par le processeur règles/messages ;
- (b) type correspondant aux résultats des requêtes locales, à l'intégration des résultats et à la demande des requêtes globales.

Ces différents types de langages supportent les transactions entre la paire *shell/applications* locales pour l'exécution des règles et les opérations de gestion de la connaissance, et la paire *shell/système* de gestion de la base de métadonnées pour la maintenance du schéma de la base de métadonnées. L'intérêt pour un environnement global d'intégration nécessite des extensions à ces langages, spécialement pour le protocole de communication. La grammaire du langage de messages définie dans [Babin93] supporte :

1. le traitement des règles par les shells

Il inclut l'exécution des sous-règles, le déclenchement des règles, l'information sur le chaînage et les modifications apportées par les règles aux systèmes locaux ;

2. les requêtes sur les données locales

Elles fournissent les résultats pour l'assemblage et le traitement local (présenter le résultat final selon les besoins des utilisateurs) ;

3. la modification des structures d'un shell

Ayant trait essentiellement au traitement des règles globales, à l'adaptabilité et la flexibilité de l'architecture concurrente des bases de métadonnées, ce type de message indique la modification du contrôle d'observation de l'application locale et du modèle structurel ;

4. les requêtes globales

Elles permettent à l'utilisateur d'extraire et de mettre à jour les données distribuées.

6.3 Création d'un nouveau shell

L'insertion d'une nouvelle application implique deux tâches principales : (1) la modélisation de cette application et son intégration dans le modèle global de l'entreprise, tel que décrit à la section 1.2 ; et (2) la création d'un nouveau *shell* pour opérationnaliser l'intégration.

Il y a quatre étapes spécifiques que l'administrateur de système doit effectuer pour créer un *shell* : (1) préparer les liens de communication entre le nouveau système et ceux déjà existants, (2) définir les liens entre le nouveau *shell* et le *shell* de la base de métadonnées, (3) générer le *shell* en utilisant le langage de définition, (4) instancier les différentes règles dans le *shell*, incluant les règles de conversion. Une fois toutes ses actions terminées, le *shell* peut être mis en opération.

1. Préparation du réseau

Pour le bon fonctionnement des liens de communication de ROPE, l'administrateur doit répondre à deux questions, pour chaque paire de *shell* : (1) comment les messages quittent-ils le *shell*, et (2) comment les messages arrivent-ils au *shell* ? Dans un environnement idéal, tous les *shells* devraient avoir la capacité d'envoyer et de recevoir des messages de leurs congénaires. Dans ce cas particulier, on aurait seulement besoin de définir les commandes d'envoi et de réception de messages. Cela implique que le *shell* peut recevoir ces messages d'une manière passive (mode asynchrone). Puisque cela n'est pas toujours le cas, on peut permettre aux *shells* d'aller chercher leurs messages (mode synchrone).

2. Appel au SGBD

Les accès à la base de données locale sont effectués au niveau du SGBD de celle-ci. Cependant, chaque SGBD possède ses propres interfaces programmes/usagers. Pour séparer la conception des *shells* de leurs environnements locaux spécifiques, on définit une commande générique permettant l'accès aux bases de données. Cette commande doit prendre deux paramètres en entrée, soit (1) le nom du fichier de requête et (2) le nom du fichier des résultats. Le second paramètre est toujours manquant quand il s'agit d'une requête de modification du contenu de la base de données (destruction, mise à jour ou insertion).

3. Description du shell

Lorsque les liens de communications sont établis et que les commandes d'accès à la base de données sont écrites, l'administrateur peut utiliser le langage de définition des *shells* pour générer (1) le fichier "rope.h", qui contient tous les éléments spécifiques à l'environnement local, et (2) le fichier de données du *shell*. Tous ces éléments sont essentiels pour la création et la mise en opération d'un

Une fois la description du *shell* analysée, les fichiers sources (les fichiers “.h” et “.c”) et les fichiers de données (les fichiers “.dat”) devraient être importés dans leurs répertoires respectifs. Par la suite, l'administrateur doit compiler les modules de ROPE et établir les liens avec les librairies et les fichiers spécifiques à l'application locale.

4. L'instanciation des règles

Au moment où les deux modèles sont intégrés, que le nouveau *shell* est créé et que les liens de communication sont mis à jour, on peut instancier les règles contenues dans la base de métadonnées avec les nouvelles connaissances. Les nouvelles connaissances seront détectées par le contrôleur de la base de métadonnées et les changements seront transmis au gestionnaire d'évolution. Cela va automatiquement peupler le nouveau *shell* avec toutes les règles pertinentes et effectuer la mise à jour des règles d'opérations et des règles d'intégrité des *shells* déjà existants.

Chapitre 7

Cadre d'intégration

L'intégration d'une nouvelle ressource informationnelle dans la base de métadonnées (MDB) doit être effectuée en suivant un cadre opérationnel bien établi. Une mauvaise définition des métadonnées représentant cette application pourrait provoquer la création d'incohérences dans le contenu de la MDB. Il est donc important de définir les métadonnées minimales nécessaires à la représentation d'une application et ce, dans le contexte du schéma du dictionnaire global des ressources informationnelles (GIRD ; voir chapitre 4). On doit également respecter les contraintes de structuration définissant le comportement d'une application dans son environnement d'exploitation. Dans ce chapitre, nous présentons la définition des contraintes de structuration d'une cellule d'intégration et une analyse complète des métadonnées — métaentités et métarelations — du GIRD et définissons les informations minimales nécessaires à l'intégration d'une application dans l'architecture d'intégration locale. Nous expliquons également les modifications fonctionnelles devant être apportées à l'outil de modélisation IBMS.

7.1 Architecture d'intégration locale et globale

La base de métadonnées est l'outil d'intégration qui consolide toutes les ressources informationnelles d'une organisation. Les métadonnées sont utilisées pour décrire une application et ses liens avec les autres systèmes existants. Elles sont en fait emmagasinées en tuples dans une base de données, qui est définie comme étant la MDB (cf. figure 6.1). L'intégration d'une application dans le contexte d'une seule base de métadonnées est réalisée comme suit :

- La MDB stocke les métadonnées concernant un ensemble d'applications déterminé et ce, par le biais d'un système de gestion prévu à cet effet (*Meta-database Management System*; MDBMS ; [Hsu et al.91, Hsu et al.92b]).
- Les métadonnées décrivent chacune des applications au niveau de leurs modèles fonctionnel, structurel et des ressources (logicielles et matérielles), ainsi que les interactions entre les applications [Hsu et al.91, Hsu96].

- Des agents de type réactif (voir chapitre 6) sont créés pour chacune des applications du système global. Un agent préserve l'autonomie de la ressource informationnelle dont il est responsable. De plus, il rend opérationnel l'intégration de cette application en utilisant le contenu de la base de métadonnées [Babin et al.97].

Lors de l'utilisation de plusieurs bases de métadonnées (cf. figure 7.1), les applications sont regroupées en agrégat de cellules d'intégration locales (*Local Integration Cells*; LIC). Chaque LIC est autonome et gère l'intégration des applications qu'elle détient. En ce qui concerne l'intégration globale des LIC dans un système réparti, les MDB des différentes cellules partagent certaines métadonnées représentant des liens sémantiques inter-applications sous leur contrôle respectif.

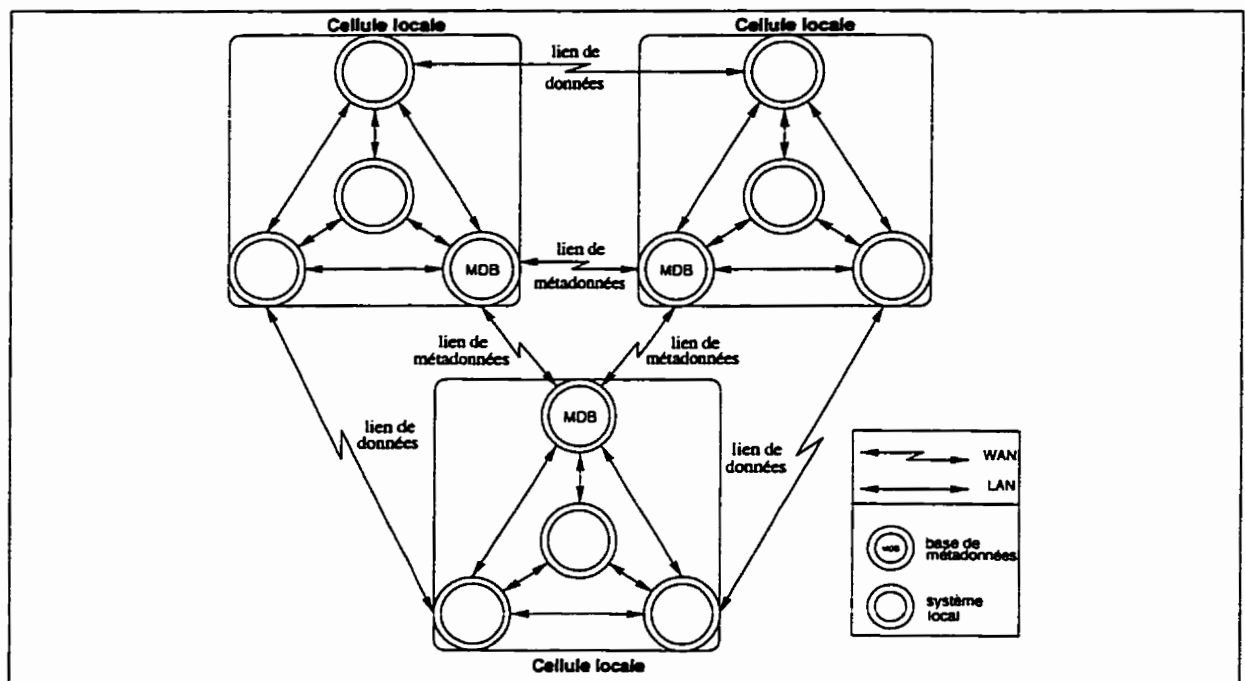


Figure 7.1 : L'architecture d'intégration globale

Le modèle autonomie-coopération, qui est un protocole de gestion de partage de métadonnées entre de multiples cellules d'intégration locales, règle les interactions entre les MDBMS, assurant ainsi la cohérence des métadonnées partagées. Le protocole définit : (1) les règles d'appartenance, qui identifient le MDBMS responsable du contrôle de reproduction des métadonnées; (2) les privilèges d'accès, qui limitent l'accès aux métadonnées; (3) les protocoles de contrôle, qui spécifient comment accéder aux métadonnées; et (4) les protocoles de mise à jour, qui assurent l'uniformité des métadonnées.

7.2 Modèle autonomie-coopération

La conception du modèle autonomie-coopération est basée sur les facteurs suivants [Maamar95, Kane95] :

- **Les frontières systèmes de l'organisation**
Les contraintes sont différentes, selon qu'il s'agisse d'un système d'information intégré évoluant à l'intérieur d'une seule ou de plusieurs entreprises.
- **Confidentialité des données**
Diverses cellules d'intégration locales qui évoluent dans une ou plusieurs organisations, peuvent avoir différents privilèges d'accès pour la même métadonnée. Le schéma d'intégration global défini par le modèle autonomie-coopération doit prendre en considération ces restrictions.
- **Topologie réseau**
La tâche de gestion des métadonnées diffère nécessairement lorsque exploitées dans un réseau local (LAN) versus un réseau à grande échelle (WAN).

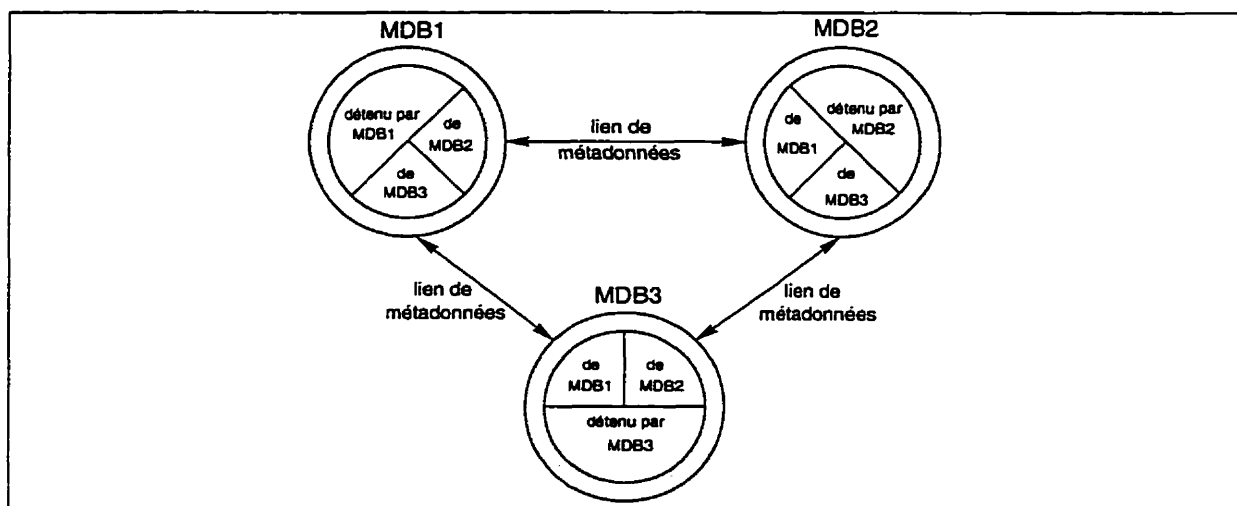


Figure 7.2 : Approche d'intégration autonomie-coopération

La figure 7.2 illustre la fragmentation de l'ensemble des métadonnées d'un système global entre différentes LIC définies dans le modèle autonomie-coopération. Les cercles représentent des systèmes de gestion des bases de métadonnées se retrouvant dans des LIC distinctes. La propriétaire d'une métadonnée est assignée au MDBMS ayant des droits de mises à jour sur celle-ci. Les métadonnées modifiées sont propagées aux autres MDBMS possédant des droits de duplication sur celles-ci. Tout changement en relation avec une métadonnée particulière, doit premièrement être communiqué au détenteur de la métadonnée en traitement [Andleigh et al.92].

La possession d'une métadonnée est déterminée en utilisant un ensemble de règles prédéfinies [Maamar95]. Ces règles dépendent de la structure et de la sémantique des

métadonnées. Une règle d'appartenance doit définir l'unique possesseur en se basant sur la valeur de la métadonnée. En supposant que les métadonnées sont identifiées par la clé primaire *metadata_key*, on obtient :

$$\text{is_Owner_of} : MD \longrightarrow MDBMS,$$

où *MD* représente l'ensemble de toutes les métadonnées et *MDBMS* l'ensemble de toutes les bases de métadonnées.

7.3 Cellule d'intégration locale autonome

Les métadonnées, qui décrivent le comportement et la structure des applications, sont inter-reliées dynamiquement par le biais d'un modèle consolidé [Hsu et al.91, Hsu96], fournissant ainsi au niveau conceptuel une vision globale cohérente de l'infrastructure informationnelle d'une organisation. Cependant, cette approche implique une gestion complexe lors de l'ajout et de la destruction des caractéristiques (métadonnées) d'une application contenues dans une base de métadonnées.

En appliquant certaines contraintes de structuration des métadonnées, nous pouvons réaliser l'intégration sans empiéter sur les opérations d'insertion et de suppression d'une ressource informationnelle. Le but est de permettre d'effectuer certaines transactions sur les métadonnées sans qu'il y ait une influence néfaste sur la cohérence du contenu d'une MDB. Les contraintes utilisent la notion de métadonnées *internes* et de *liaisons*. Les métadonnées *internes* réfèrent à l'ensemble de métadonnées décrivant un système dit isolé, c'est-à-dire, un système n'ayant aucune inter-relation avec d'autres systèmes existants. Les métadonnées de *liaisons* réfèrent à un ensemble de métadonnées décrivant les interactions d'un système avec ses confrères. À partir de ces spécifications, nous définissons les contraintes suivantes :

APPL : l'ensemble de toutes les applications ;

L_{kl} : l'ensemble de métadonnées de *liaisons* entre les applications *k* et *l*, où $L_{kl} = L_{lk}$;

I_k : l'ensemble de métadonnées *internes* de l'application *k*, où $L_{kk} = I_k$;

MDB(k) : fonction retournant le MDBMS qui gère la LIC contenant l'application *k*, où $MDB : APPL \longrightarrow MDBMS$;

app(i) : fonction retournant l'ensemble d'applications contenu dans la LIC gérée par MDBMS *i*, où $app : MDBMS \longrightarrow \mathcal{P}(APPL)$; notons que $MDB(k) = i \Leftrightarrow k \in app(i)$;

L'_{ij} : l'ensemble de métadonnées utilisées pour intégrer la cellule LIC gérée par MDBMS i à la LIC gérée par MDBMS j , défini comme :

$$L'_{ij} = \bigcup_{k \in \text{app}(i) \mid l \in \text{app}(j)} L_{kl} ;$$

I'_i : l'ensemble de métadonnées utilisé à l'intérieur de la LIC gérée par MDBMS i , défini comme :

$$I'_i = L'_{ii} = \bigcup_{k \in \text{app}(i) \mid l \in \text{app}(i)} L_{kl}.$$

Pour assurer une autonomie locale, les métadonnées *internes* de différentes applications devraient être disjointes. Il en est de même pour les métadonnées intra-applications qui devraient être disjointes des métadonnées inter-applications. Ces contraintes sont spécifiées comme suit :

$$\begin{aligned} I_k \cap I_l &= \emptyset, \quad \forall k, l \in \text{APPL}, \\ I_k \cap L_{kl} &= \emptyset, \quad \forall k, l \in \text{APPL}, \\ L_{kl} \cap L_{k'l'} &= \emptyset, \quad \forall k, k', l, l' \in \text{APPL}, \text{ où } k \neq k' \vee l \neq l'. \end{aligned}$$

De même, les métadonnées intra-cellulaires et inter-cellulaires devraient être totalement disjointes les unes des autres, ce qui nous permet d'affirmer :

$$\begin{aligned} I'_i \cap I'_j &= \emptyset, \quad \forall i, j \in \text{MDBMS}, \\ I'_i \cap L'_{ij} &= \emptyset, \quad \forall i, j \in \text{MDBMS}, \\ L'_{ij} \cap L'_{i'j'} &= \emptyset, \quad \forall i, i', j, j' \in \text{MDBMS}, \text{ où } i \neq i' \vee j \neq j'. \end{aligned}$$

En résumé, les métadonnées devraient être disposées en couches successives, de telle sorte que les couches inférieures représentent les métadonnées des applications et que les couches supérieures représentent les métadonnées d'intégration (cf. figure 7.3). Les niveaux obtenus garantissent que le processus d'intégration ou de suppression d'une application dans une base de métadonnées est effectué sans porter atteinte à la cohérence du contenu de celle-ci. Cependant, cette architecture reste très complexe à gérer et à maintenir au niveau des jonctions entre les métadonnées appartenant à différents niveaux. Il est également évident que les métadonnées *internes* d'un MDBMS et ses LIC associées, doivent être détenues par ce MDBMS, puisqu'elles décrivent les applications sous le contrôle de ce dernier.

7.4 Fonctions des métaentités et métarelations

La détermination de l'information nécessaire pour l'intégration d'une ressource informationnelle permet de conserver la cohérence du contenu courant de la base

de métadonnées. Une approche systématique, qui facilite la détermination des métadonnées minimales dans les environnements coopératifs MDB/GQS (*MetaDataBase/Global Query System*) et MDB/ROPE (*MetaDataBase/Rule-Oriented Programming Environment*), consiste à cataloguer les métaentités et les métarelations dans le cadre du modèle fonctionnel du GIRD, qui se divise en quatre métasujets : (1) la Vue-Application, (2) la Vue-Ressource, (3) la Vue-Fonctionnelle et (4) la Vue-Structurelle. De plus, ce modèle est composé de cinq métacontextes inter-reliant ces quatre vues fonctionnelles : (1) Représente, (2) Implémente, (3) Utilise-Supporte, (4) Transforme et (5) Répertoire.

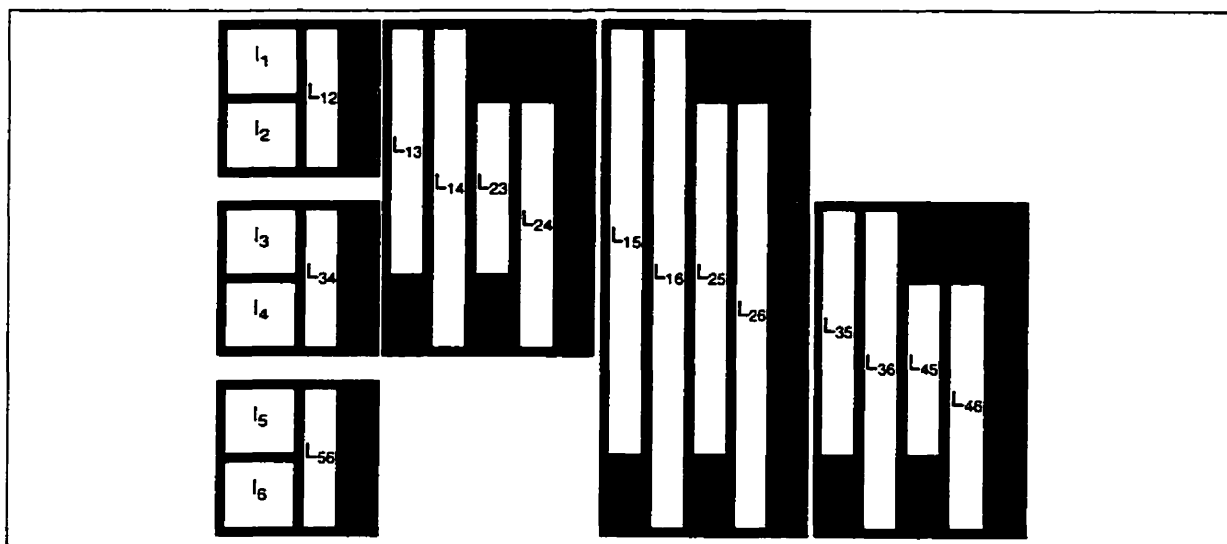


Figure 7.3 : Couches de métadonnées

Pour interfacer avec ses congénaires ou la base de métadonnées, chacune des applications évoluant dans l'entreprise doit posséder un *shell*. L'exploitation de cette architecture nécessite un ensemble de métadonnées spécifiques, qui est composé au minimum de toutes les métadonnées nécessaires au fonctionnement du système de requêtes globales, puisque ROPE fait appel aux services de celui-ci.

Lors de l'ajout d'un système dans une MDB, on doit établir des liaisons entre celui-ci et les applications déjà existantes. Il existe trois niveaux de liaisons distincts, soit : (1) les contextes fonctionnels, qui définissent les règles d'exploitation inter-applications, (2) les données équivalentes et (3) le modèle structurel consolidé de l'entreprise. La principale difficulté du processus d'intégration se situe au niveau de la consolidation des modèles structurels de l'organisation.

La résolution de ce problème passe assurément par une étude complète de toutes les composantes du schéma structurel du GIRD. À partir de cette analyse, il est possible d'identifier les métaentités et les métarelations ayant un rôle clef dans le processus d'intégration et ainsi définir les informations minimales (voir section 7.5) nécessaires pour l'exploitation d'une application dans un contexte local (voir section 7.3). La justification des métadonnées minimales s'appuie sur l'architecture de ROPE (voir cha-

pitre 6) et sur les étapes de résolution d'une requête globale dans GQS (voir chapitre 5), dont voici une récapitulation :

- Étape 1 : Formulation de la requête globale
- Étape 2 : Détermination des conditions de jointure
- Étape 3 : Traitement de la requête globale
- Étape 4 : Génération des requêtes locales
- Étape 5 : Exécution des requêtes locales
- Étape 6 : Intégration des résultats

7.4.1 Vue Application

MétaEntité

- **Application**
Les métadonnées sont nécessaires lors de la réalisation de l'étape 1.1. *Applname* permet d'obtenir le nom de l'application concernée par la requête. Un seul tuple est nécessaire. Il doit contenir l'information concernant l'application devant être intégrée dans la base de métadonnées. Les métadonnées requises sont *Applname*, *descript*, *applcode* et *userid* (l'administrateur de l'application).
- **Usager**
Un seul tuple est nécessaire. Il doit contenir l'information concernant l'administrateur de l'application pour en assurer une gestion sécuritaire. Les métadonnées requises sont *userid*, *lastname* et *firstname*.

MétaRelation Plurale

- **ApplUsager**
Les métadonnées sont nécessaires lors de la réalisation de l'étape 4. Cette métarelation permet d'obtenir les informations concernant les droits d'accès (*accesscode*) pour un utilisateur donné (*Userid*). Ainsi, la sécurité des métadonnées est assurée et ce, à la grandeur du système informationnel de l'entreprise. Les métadonnées requises sont *Applname*, *Userid*, *upassword* et *accesscode*.

7.4.2 Vue Fonctionnelle

Lors de l'ajout d'une application, il n'est pas nécessaire de définir des règles opérationnelles dans l'architecture de ROPE, ce qui implique une définition optionnelle des métadonnées au niveau sémantique de la nouvelle application. Il n'existe aucun conflit en relation avec les règles d'intégrité, puisque chacun des sujets possède un identifiant unique (*Sname*) et ce, à la grandeur de l'organisation. S'il n'existe pas

de modèle fonctionnel en correspondance avec la ressource informationnelle, un sujet simple sera créé et il ne pourra pas être décomposé. Le contenu de ce sujet sera principalement constitué des attributs et des règles de l'application en traitement. Advenant l'existence d'un schéma sémantique, par exemple la définition des vues, il sera possible de l'insérer dans le contenu courant de la base de métadonnées, en créant les métadonnées et les métarelations plures nécessaires, sans générer d'incohérence.

MétaEntités

- **Application**

Voir la description dans la section Vue Application (section 7.4.1).

- **Action**

Les métadonnées sont nécessaires lors d'échanges informationnels entre les *shells* de ROPE. Cette métaentité contient l'information concernant les actions d'une règle qui doivent être exécutées lorsque les conditions de celle-ci ont été évaluées. Les métadonnées requises sont *Actid*, *acttype*, *factid* et *decvalue*.

- **Condition**

Les métadonnées sont nécessaires lors d'échanges informationnels entre les *shells* de ROPE. Cette métaentité contient l'information concernant les différentes conditions qui sont comprises lors de la définition d'une règle. Les métadonnées requises sont *Condid*, qui est aussi un attribut de Règle, *leftfact*, *operator* et *rightfact*.

- **Contexte**

Cette métaentité contient de l'information en relation avec les contextes d'une application, c'est-à-dire, le cadre environnemental, et ce, au niveau local par le biais de la métarelation plure Intégrer et au niveau global par le biais de la métarelation obligatoire Inter.Composer, qui est apparentée à *Applname*, définissant la métaentité Application comme étant le possesseur et la métaentité Contexte comme étant le possédé. Les métadonnées requises sont *Cname*, *Applname* et *descript*, qui sont définies dans la métaentité Application.

- **Fait**

Les métadonnées sont nécessaires lors d'échanges informationnels entre les *shells* de ROPE. Cette métaentité contient l'information concernant les faits qui peuvent exister lors de la définition d'une action dans une règle. Les métadonnées requises sont *Factid*, qui est aussi un attribut de Action, *factname*, qui est soit un *Itemcode* ou un *Condid*, *factype* et *factvalue*.

- **Item**

Les métadonnées sont nécessaires lors de la réalisation des étapes 1.1, 1.2 et 6. Grâce à ces métadonnées, il est possible d'obtenir des informations concernant le type, la description, le nom, le format et le domaine des attributs, ainsi que leurs associations respectives. Un nouveau tuple doit être créé pour tous les attributs dans le modèle de données. On doit s'assurer que les nouveaux *Itemcode*

ne sont pas en conflit avec d'autres déjà existants. Une bonne manière pour encoder les *Itemcode* est d'utiliser le *applcode* et de lui concaténer un numéro séquentiel. Cela permet d'assurer la génération d'identifiants uniques pour les attributs des différentes applications. L'automatisation de cette tâche serait un avantage intéressant. La métadonnée *Applname* est implicite, puisqu'on intègre qu'une application à la fois. Les métadonnées requises sont *Itemcode*, *itemname*, *itemtype*, *descript*, *iformat*, *ilength* et *Applname*.

- **Règle**

Les métadonnées sont nécessaires lors de la réalisation des étapes 1.3 et 6. Ces métadonnées permettent d'obtenir de l'information concernant les règles d'affaires (*rtype*) et les règles de conversion. Les métadonnées requises sont *Rname*, *rtype* et *descript*.

- **Ressources Logicielles**

Les métadonnées sont nécessaires lors de la réalisation des étapes 1.1, 3 et 4. Cette métaentité contient l'information en relation avec les ressources logicielles — par exemple : programmes, fichiers de données, réseaux, documents et autres — qui composent une application. Les métadonnées requises sont *Resid*, *resname*, *extension*, *restype* et *descript*.

- **Sujet**

Les métadonnées sont nécessaires lors de la réalisation de l'étape 1.1. Le nom des sujets concernant une application permet d'obtenir des informations en relation avec le modèle fonctionnel de celle-ci. À partir de son contenu, il est possible de reconstituer le modèle fonctionnel complet de l'application concernée. Les métadonnées requises sont *Sname* et *Applname*.

MétaRelations Plurales

- **Acte_de**

Cette métarelation permet d'établir la relation de type plusieurs à plusieurs entre une règle et les actions correspondantes, ou vice versa, définissant ainsi l'ordre relatif d'une règle avec ses sujets ou ses conditions. Les métadonnées requises sont *Actid* contenue dans *Action*, *Rname* contenue dans *Règles* et *reorder*.

- **Appeler**

Cette métarelation permet d'établir la relation de type plusieurs à plusieurs entre une action et les procédures (ne retourne pas de résultat), qui doivent être exécutées pour réaliser cette action. Les métadonnées requises sont *Actid* contenue dans *Action*, *Procid* qui est un synonyme de *Resid*, *Parid* qui est un synonyme de *Factid* et *parorder*.

- **Contenir**

Cette métarelation permet d'établir la relation de type plusieurs à plusieurs entre une règle et les contextes correspondants, ou vice versa, définissant ainsi l'ordre

relatif d'une règle par rapport à ses contextes. Les métadonnées requises sont *Cname* contenue dans *Contexte*, *Rname* contenue dans *Règle* et *relorder*.

- **Décrire**

Les métadonnées sont nécessaires lors de la réalisation de l'étape 1.1. Cette métarelation permet d'établir la relation de type plusieurs à plusieurs entre un attribut et les sujets correspondants, ou vice versa. Les métadonnées requises sont *Itemcode* et *Sname*.

- **Équivalent**

Les métadonnées sont nécessaires lors de la réalisation des étapes 2.1, 2.4, 3.1 et 6. *EqItemcode* permet d'obtenir de l'information concernant les données équivalentes pour un attribut donné. L'utilisation de la métaentité *Item* dans l'étape 6 permet (1) d'identifier l'existence d'équivalence de données entre deux attributs impliqués dans une condition de jointure globale et (2) de trouver la règle de conversion correspondante (*convert_by* ou *reverse_by*), qui est nécessaire pour le processeur de règle. Au moment de la définition des équivalences, une attention particulière doit être portée à l'identification des équivalences globales courantes, incluant les règles. Les métadonnées requises sont *Itemcode*, *EqItemcode*, *convert_by* et *reverse_by*.

- **Intégré**

Cette métarelation permet d'établir la relation de type plusieurs à plusieurs entre une application et ses contextes correspondants, ou vice versa. La métarelation spécifie les applications de l'entreprise qui sont intégrées récursivement, c'est-à-dire par niveau et ce, en fonction de leurs contextes ; elle définit donc le cadre environnemental des contextes d'une application au niveau local. Les métadonnées requises sont *Applname*, *Cname*, *metadatabase* et *integration_type*.

- **Traiter**

Cette métarelation est similaire à la métarelation *Appeler*, sauf qu'elle se trouve entre une action et les fonctions (retourne des résultats) qui doivent être exécutées pour la réalisation de cette action. Les métadonnées requises sont *Actid* contenue dans *Action*, *Functid* qui est un synonyme de *Resid*, *Parid* qui est un synonyme de *Factid* et *parorder*.

7.4.3 Vue Structurale

L'information contenue dans cette vue correspond au modèle structurel consolidé (schéma global) de l'entreprise. Par conséquent, une priorité doit être accordée à l'insertion des nouveaux tuples dans cette table. Cependant, certaines tâches doivent être préliminairement effectuées : (1) la normalisation du modèle structurel de la nouvelle application et (2) la détermination des métaentités et des métarelations communes entre les frontières systèmes.

Dans la définition des informations requises pour le modèle structurel, nous supposons que le modèle informationnel a été normalisé et que les métaentités et métarelations plures ont été déterminées.

MétaEntités

- **Entité/Relation**

Les métadonnées sont nécessaires lors de la réalisation des étapes 1.1, 2.1 et 2.3. Pour la métaentité Entité/Relation, il existe deux classes possibles, soit : (1) les OE/PRs communes (métaentités/métarelations plures) avec des OE/PRs déjà existantes et (2) les autres OE/PRs. Elles sont analysées séparément :

- **OE/PRs communes**

La clé alternative (*akey*) des OE/PRs déjà existantes est modifiée pour inclure la clé (liste des *Itemcode*) de la nouvelle OE/PR. Si le nom de la nouvelle OE/PR est différent de celle déjà existante, un tuple doit être créé dans la métarelation plure **Nommer_ainsi**.

- **les autres OE/PRs**

Le minimum défini est : *ERname*, *ertype*, *descript* et *akey*. Si le *ERname* est en conflit avec une OE/PR déjà existante, un nouveau nom doit y être attribué et un tuple doit être créé dans la métarelation plure **Nommer_ainsi**.

Par le biais du nom de la métaentité, il est possible d'obtenir des informations concernant les concepts de données (OE, PR) et ce, grâce à la métadonnée *ertype*. Cela permet d'établir les associations (*akey*) entre les entités et les relations dans le modèle global de l'entreprise. Les métadonnées requises sont *ERname*, *ertype*, *descript* et *akey*.

- **Intégrité**

Les métadonnées sont nécessaires lors de la réalisation de l'étape 1 et 2. Cette métaentité correspond aux métarelations de types obligatoire et fonctionnelle, et aux dépendances fonctionnelles entre les entités et les relations (via **Exister**). Ces informations permettent de conserver l'intégrité du modèle structurel consolidé de l'entreprise lors de la définition de ses métarelations de type obligatoire ou fonctionnel. Les métadonnées requises sont *Intrname*, *intype*, *master* et *slave*.

- **Item**

Voir la description dans la section Vue Fonctionnelle (section 7.4.2).

- **Sujet**

Voir la description dans la section Vue Fonctionnelle (section 7.4.2).

MétaRelations Plures

- **Appartenir_à**

Les métadonnées sont nécessaires lors de la réalisation de l'étape 1.1. Cette

métarelation permet d'établir la relation de type plusieurs à plusieurs entre un attribut et les métaentités ou les métarelations concernées, ou vice versa. On doit créer de nouveaux tuples pour indiquer l'appartenance des attributs à leurs OE/PRs correspondantes. Pour les OE/PRs communes avec des OE/PRs déjà existantes, tous les attributs sont emmagasinés comme étant des attributs non-clés. Pour les autres OE/PRs, la clé est emmagasinée telle quelle. La métadonnée *inpkey* est un drapeau (valeur booléenne) qui indique si un attribut fait partie de la clé primaire de Entité/Relation, permettant ainsi la détermination des dépendances fonctionnelles. Les métadonnées requises sont *Itemcode*, *ERname* et *inpkey*.

- **Nommer_ainsi**

Les métadonnées sont nécessaires lors de la réalisation de l'étape 4. Elles permettent d'obtenir le nom local (*localname*) d'une application (*Applname*) pour une métaentité/relation donnée (*ERname*). Les métadonnées requises sont *ERname*, *Applname* et *localname*.

- **Transformer**

Les métadonnées sont nécessaires lors de la réalisation de l'étape 1.1. Cette métarelation permet d'établir la relation de type plusieurs à plusieurs entre une entité/relation et les sujets correspondants, ou vice versa. Les métadonnées requises sont *Sname* contenue dans *Sujet* et *ERname* contenue dans *Entité/Relation*.

7.4.4 Vue Ressources

La vue des ressources représente un cas spécial. Il existe deux types de métaentités pour cette vue, soit *RessourcesLogicielles* et *RessourcesMatérielles*. En fait, la vue représente l'agrégation de plusieurs types de ressources logicielles et matérielles.

MétaEntités

- **Application**

Voir la description dans la section Vue Application (section 7.4.1).

- **Item**

Voir la description dans la section Vue Fonctionnelle (section 7.4.2).

- **RessourcesMatérielles**

Les métadonnées sont nécessaires lors de la réalisation de l'étape 3. Cette métaentité contient l'information en relation avec les ressources matérielles qui sont nécessaires lors de l'exécution des ressources logicielles qui composent une application. La métarelation plurale *Résider_à* permet de localiser les ressources matérielles nécessaires à une ressource logicielle lors de son exécution. Les métadonnées requises sont *Serialno*, *descript*, *location*, *nodename* et *nodeaddr*.

MétaRelation Plurale

- **Module_de**

Cette méta relation est de type récursif et représente les liens existants entre les ressources logicielles. Les métadonnées requises sont *Resid*, *Subresid* qui est un synonyme de *Resid* et *relationship*.

- **Résider_à**

Cette méta relation permet d'établir la relation de type plusieurs à plusieurs entre une ressource logicielle et les ressources matérielles correspondantes, ou vice versa, ce qui facilite la localisation de l'environnement d'exécution pour les ressources logicielles. Les métadonnées requises sont *Resid* contenue également dans *RessourcesLogicielles*, *Serialno* contenue également dans *RessourcesMatérielles*, *path* et *invoke*.

- **Stocker_dans**

Les métadonnées sont nécessaires lors de la réalisation de l'étape 1.1. Cette méta relation permet d'établir la relation de type plusieurs à plusieurs entre une ressource logicielle et ses attributs correspondants, ou vice versa. Les métadonnées requises sont *Itemcode* contenue dans *Item* et *Resid* contenue dans *RessourcesLogicielles*.

- **Utiliser**

Les métadonnées sont nécessaires lors de la réalisation de l'étape 1.1. Cette méta relation permet d'établir la relation de type plusieurs à plusieurs entre une application et ses ressources logicielles, ou vice versa. Les métadonnées requises sont *Applname* contenue dans *Application* et *Resid* contenue dans *RessourcesLogicielles*.

La figure 7.4 illustre les différentes spécialisations de *RessourcesLogicielles* et *RessourcesMatérielles*, et comment sont définis *Action*, *Fait* et *Application*, et démontre également les relations entre les différentes spécialisations de ressources logicielles et matérielles. Dans le contexte de ROPE, nous voyons que : (1) l'organisation de la base de données locale doit être convenablement encodée, permettant ainsi au générateur de requêtes de localiser les attributs persistents concernés et ce, par le biais des méta relations plurales BD ; (2) les méta relations fonctionnelles *Gérer_par* sont nécessaires au système de requêtes globales (GQS) pour lui permettre de déterminer quel générateur de requêtes doit être utilisé ; (3) les méta relations obligatoires *Objet_dans* et *Fichier_de_BD* représentent des liaisons entre les fichiers et les objets contenant les attributs ; (4) la méta relation obligatoire *Fichier_Pgm* et les méta relations plurales *Résider_à* sont utilisées pour spécifier à ROPE quelles routines sont utilisées dans les règles et dans quels fichiers elles sont localisées.

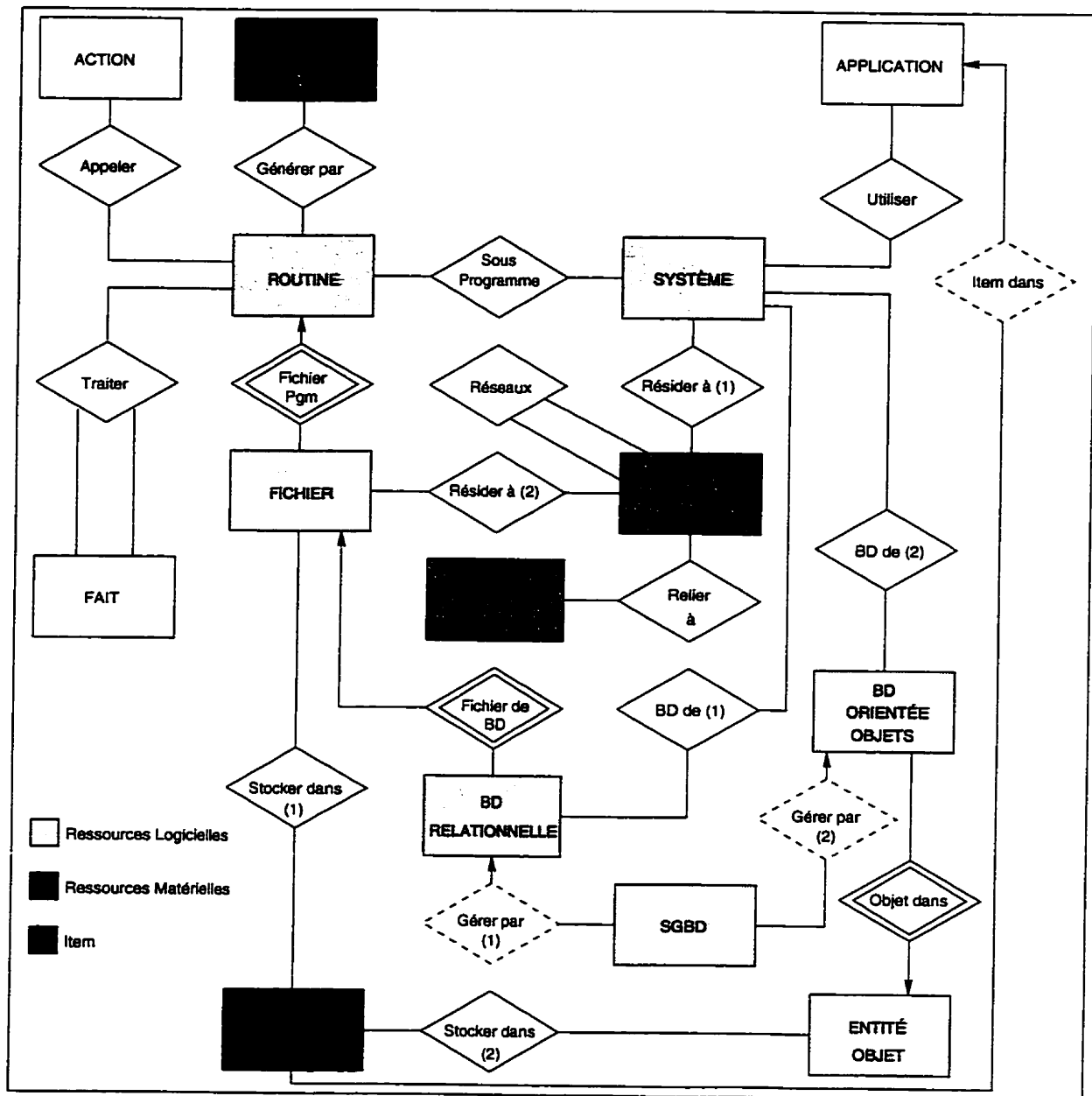


Figure 7.4 : Représentation détaillée de la vue ressource

7.5 Informations minimales

L'analyse effectuée dans la section précédente démontre que la définition du modèle structurel consolidé du GIRD (cf. figure 4.7) est complète et qu'il n'existe aucune information (métadonnées) superflue, puisque la présence de la quasi-totalité des métaentités et des métarelations qui le composent, est justifiée. Dans cette section, le modèle intégré de l'organisation est analysé en se basant sur l'environnement d'exploitation¹ de la base de métadonnées. Les métadonnées minimales requises pour l'intégration d'une ressource informationnelle correspondent à un sous-ensemble des besoins spécifiés à la section 7.4. Pour faciliter la définition de ces métadonnées, nous considérons la saisie de tous les attributs d'un métatuple, permettant ainsi de déterminer ceux qui sont nécessaires dans chacune des métaentités et méta-relations plurales. La figure 7.5 représente le modèle structurel minimal devant être défini pour une intégration cohérente et uniforme d'une application dans la base de métadonnées.

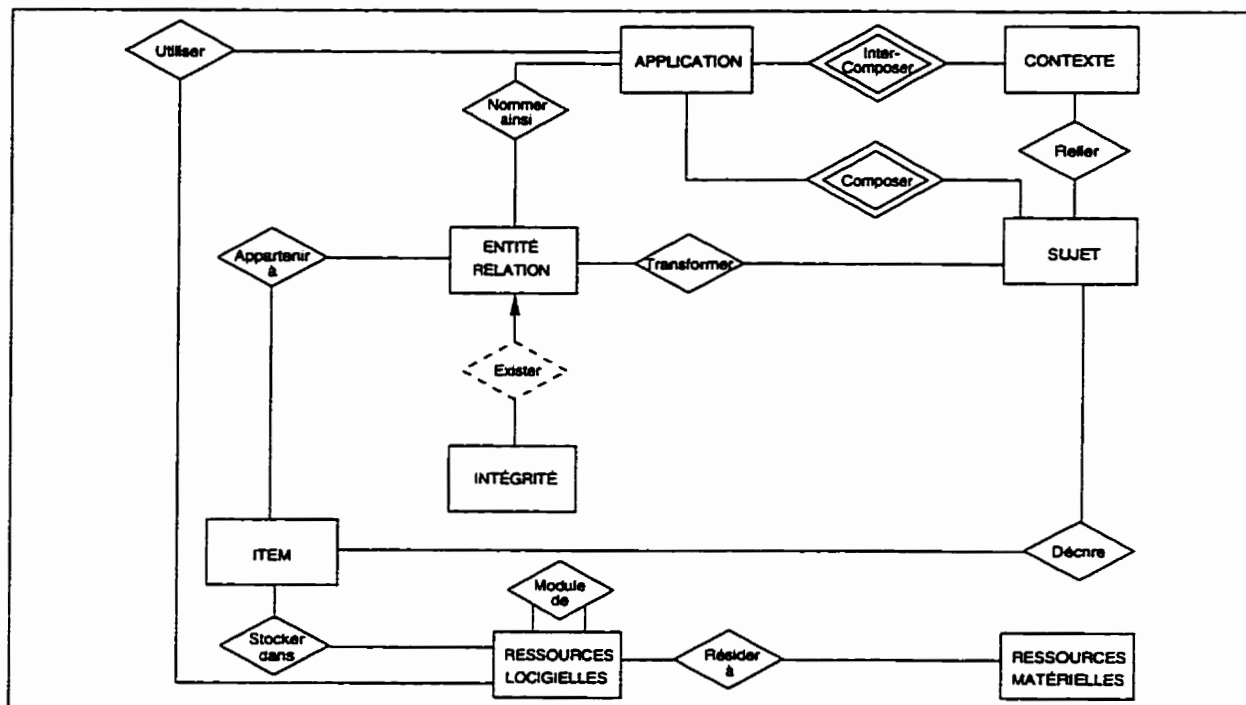


Figure 7.5 : Modèle structurel minimal du GIRD

7.5.1 MétaEntités

● Application

L'intégration d'une nouvelle application nécessite un seul tuple dans cette métaentité. Les métadonnées requises sont *Applname*, *descript*, *applcode*, *userid*, *addedby*, *dateadded*, *modifyby*, *lastmod* et *nummods*.

¹ La version prototypale de MDB installée sur la machine babs.ift.ulaval.ca a servi de fondement à cette analyse.

● Contexte

Cette métaentité contient les contextes reliant : (1) les sujets d'une application et (2) les liaisons inter-applications présentes dans le système global. La présence de cette métaentité est primordiale, puisqu'elle permet l'intégration d'une application dans le schéma global au niveau sémantique. Les métadonnées requises sont *Cname*, *Applname*, *descript*, *xcoord*, *ycoord*, *addedby*, *dateadded*, *modifby*, *lastmod* et *nummods*.

● Entité/Relation

Cette métaentité contient les entités et les relations plures de l'application devant être intégrée. Il y a autant de tuples qu'il y a d'entités et de relations dans l'application. Les métadonnées requises sont *Ername*, *ertype*, *descript*, *akey*, *addedby*, *dateadded*, *modifby*, *lastmod* et *nummods*.

● Intégrité

Cette métaentité contient les contraintes d'intégrité spécifiées par les concepts des modèles structurels de la nouvelle application et les contraintes de définition des utilisateurs. Les métadonnées requises sont *Intrname*, *inttype*, *descript*, *master*, *slave*, *addedby*, *dateadded*, *modifby*, *lastmod* et *nummods*.

● Item

Il y a autant de tuples qu'il y a d'attributs dans la nouvelle application. Dépendant des besoins opérationnels de la base de métadonnées, seuls certains métatuples peuvent être nécessaires. Les métadonnées requises sont *Itemcode*, *itemname*, *itemtype*, *descript*, *iformat*, *ilength*, *domain*, *defvalue*, *addedby*, *dateadded*, *modifby*, *lastmod*, *nummods* et *Applname*.

● RessourcesLogicielles

Cette métaentité représente les différents types de ressources logicielles tels que les fichiers, les documents et les programmes d'une application. Les métadonnées requises sont *Resid*, *resname*, *extension*, *restype*, *descript*, *sizevalue*, *sizeunit*, *coding*, *developedby*, *addedby*, *dateadded*, *modifby*, *lastmod* et *nummods*.

● RessourcesMatérielles

Cette métaentité décrit les composantes physiques d'une application. Les métadonnées requises sont *Serialno*, *hname*, *htype*, *descript*, *location*, *nodename*, *nodeaddr*, *manufacturer*, *purchby*, *datepurch*, *userid*, *addedby*, *dateadded*, *modifby*, *lastmod* et *nummods*.

● Sujet

L'intégration d'une nouvelle application nécessite un seul tuple pour décrire le sujet avec qui il est en relation. Les métadonnées requises sont *Sname*, *descript*, *xcoord*, *ycoord*, *Applname*, *fileid*, *addedby*, *dateadded*, *modifby*, *lastmod* et *nummods*.

7.5.2 MétaRelation

- **Appartenir_à**

Cette métarelation est prise en considération, puisqu'elle relie chacun des attributs avec ses concepts structurels correspondants. Les métadonnées requises sont *Itemcode*, *ERname*, *relpos*, *inpkey* et *posinpk*.

- **Composer**

Cette métarelation est prise en considération, puisqu'elle permet d'obtenir les différents sujets qu'une application peut contenir, par le biais d'une clé étrangère (*Applname*) dans les sujets.

- **Décrire**

Cette métarelation est prise en considération, puisqu'elle permet d'obtenir la description des attributs des sujets de la nouvelle application. Les métadonnées requises sont *Itemcode*, *Sname*, *relpos* et *inherited*.

- **Inter-Composer**

Cette métarelation est prise en considération, puisqu'elle représente les règles d'intégrité implicites reliant les contextes intra-applications à leurs applications respectives et ce, par le biais du métaattribut *Applname*.

- **Module_de**

Cette métarelation récursive est prise en considération, puisqu'elle représente le fait qu'une ressource logicielle peut être utilisée par d'autres ressources logicielles pour effectuer des tâches particulières. Les métadonnées requises sont *Resid*, *Subresid* et *relationship*.

- **Nommer_ainsi**

Cette métarelation est prise en considération, puisqu'elle maintient la relation entre les divers noms des objets logiques globaux dans les applications locales. Les métadonnées requises sont *ERname*, *Applname* et *localname*.

- **Relier**

Cette métarelation est prise en considération, puisqu'elle raccorde les contextes d'une application au niveau sémantique des sujets de celle-ci. Les métadonnées requises sont *Cname*, *Sname* et *direction*.

- **Résider_à**

Cette métarelation est prise en considération, puisqu'elle maintient la relation entre les composantes logicielles et les composantes matérielles d'une application. Les métadonnées requises sont *Resid*, *Serialno*, *path* et *invokecom*.

- **Stocker_dans**

Cette métarelation est prise en considération, puisqu'elle maintient la relation entre les ressources logicielles et le type de support physique utilisé pour les attributs persistants qui sont emmagasinés, soit dans des fichiers, soit dans une base de données relationnelle, ou dans une base de données orientée-objets. Les métadonnées requises sont *Itemcode*, *Resid* et *relpos*.

- **Transformer**

Cette métarelation est prise en considération, puisqu'elle relie chacun des sujets à ses concepts structurels correspondants. Les métadonnées requises sont *Sname*, *ERname*, *addedby*, *dateadded*, *modifby*, *lastmod* et *nummods*.

- **Utiliser**

Cette métarelation est prise en considération, puisqu'elle exprime la relation entre les ressources logicielles utilisées pour l'exploitation d'une application. Les métadonnées requises sont *Applname*, *Resid* et *dataorg*.

7.6 Modifications fonctionnelles de IBMS

Les technologies de l'information, qui font partie de la stratégie de l'entreprise, doivent s'intégrer aux diverses fonctions de l'organisation et appuyer la concertation des efforts organisationnels, d'où l'intérêt de bien les comprendre et les maîtriser. Pour saisir l'architecture informationnelle d'une organisation, il est nécessaire de tracer une image (photographie) très nette des caractéristiques des systèmes d'information, ce qui a donné naissance aux méthodes de modélisation dans le domaine de la représentation des technologies de l'information, fournissant ainsi un médium de communication entre les experts par le biais de modèles (mathématique, graphique, textuel, etc.).

La modélisation des ressources informationnelles présentes dans un système global s'avère le choix de prédilection pour l'intégration d'application [Arens et al.93, Buchmann90, Collet et al.92, Hsu et al.92a, Hsu96, Huhns et al.92, Lawrence et al.96, Spaccapietra et al.92], puisque cette technique permet de capturer et de visualiser les caractéristiques d'une application via des méthodes de représentation graphiques. Pour métamodéliser les systèmes informatiques, nous utilisons le logiciel IBMS (voir section 3.5) qui supporte la méthode TSER (voir chapitre 3). Cet outil CASE assiste l'analyste dans la conception du modèle global de l'entreprise.

Une caractéristique intéressante de la MDB est, qu'à partir de son contenu, il est possible de récupérer tous les modèles fonctionnels et structurels de toutes les applications du schéma consolidé d'une organisation. L'intégration d'une nouvelle application, si on utilise IBMS, peut être effectuée en modélisant le niveau fonctionnel de celle-ci, puisqu'il est possible de générer automatiquement le modèle structurel complet de cette application en se basant sur les spécifications du modèle fonctionnel de cette dernière. Définissons les variables suivantes :

- F , le modèle fonctionnel consolidé d'une MDB ;
- S , le modèle structurel consolidé correspondant à F ;
- F' , le modèle fonctionnel de la nouvelle application devant être intégrée ;
- S' , le modèle structurel correspondant à F' .

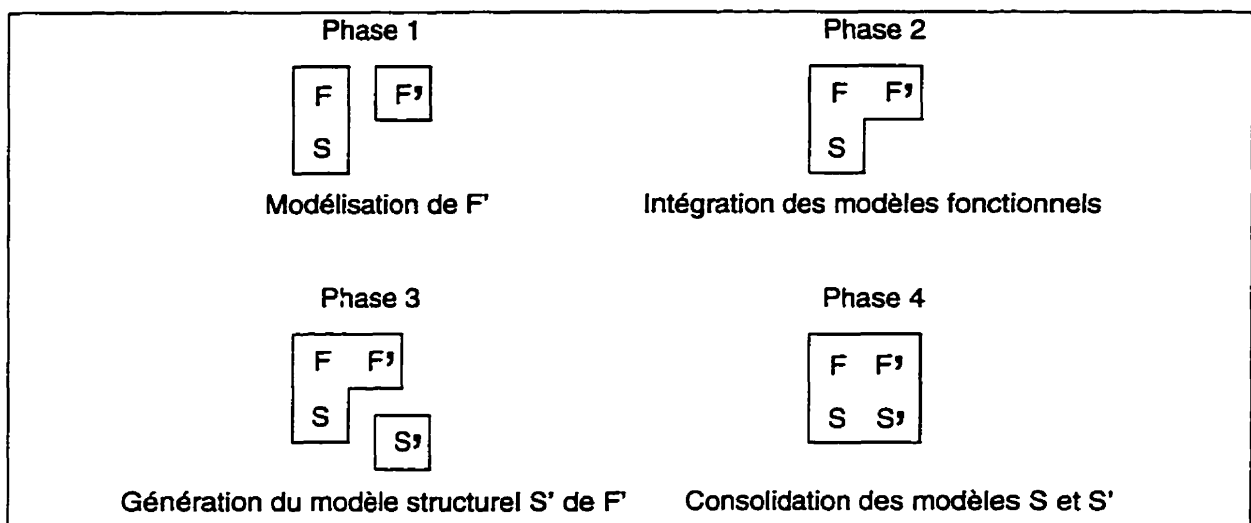


Figure 7.6 : Phases d'intégration d'une nouvelle application

Ainsi, les phases devant être effectuées pour intégrer une application sont (cf. figure 7.6) :

Phase 1 : Modélisation de F'

1. modélisation de la nouvelle application F' en utilisant une méthodologie adaptée, dans notre cas, la méthode TSER.
2. Traduction, si nécessaire, du modèle F' vers un schéma fonctionnel supporté par IBMS. Cette fonction permet de supporter une panoplie de méthodologies ; cependant, sa conception est complexe, en revanche donne la possibilité aux analystes de modéliser les applications dans la méthodologie de leur choix.
3. Récupération des modèles fonctionnel F et structurel S consolidés contenus dans une base de métadonnées. Cette étape est réalisée par le biais du MDBMS qui offre cette option et affiche le schéma consolidé dans IBMS

Phase 2 : Intégration des modèles fonctionnels

4. Intégration des modèles fonctionnels, c'est-à-dire F et F' . On définit les équivalences de données, les règles de conversion et les contextes inter-reliant la nouvelle application avec ses consœurs déjà existantes dans le système global. Par la suite, on doit assurer l'autonomie locale de la nouvelle application en respectant les contraintes de structuration établies dans la section 7.3, ce qui est réalisé en effectuant des disjonctions entre les métadonnées inter-applications et intra-applications dans la cellule locale en traitement.

Phase 3 : Génération du modèle structurel S' de F'

5. Génération du modèle structurel S' correspondant avec F' en effectuant :
 - * la décomposition des sujets en sous-modèles ;

- * la vérification des clés primaires des entités et des relations plurales à partir des dépendances fonctionnelles ;
- * la détermination des relations obligatoires (cardinalité 1, N) et fonctionnelles (cardinalités N , 1), ce qui est réalisé en créant des clés étrangères ;
- * la spécification des liaisons entre les équivalences de données ;
- * la vérification des spécifications minimales de représentation pour assurer la cohérence du modèle structurel en traitement (voir section 7.5) ;
- * la normalisation des sous-modèles en utilisant les théories de dépendance, donnant ainsi des schémas respectant la troisième forme normale ;
- * la consolidation des sous-schémas dérivés du modèle sémantique, en fusionnant les entités et les relations qui ont des clés identiques, incluant les clés ayant des attributs équivalents.

Phase 4 : Consolidation des modèles S et S'

6. Consolidation des modèles structurels, c'est-à-dire S et S' , en favorisant les clés primaires déjà existantes (Phase 4) et en respectant les contraintes de structuration définies dans la section 7.3.

Phase 5 : Implantation

7. Instanciation des métadonnées qui définissent le contenu de la base de métadonnées en utilisant le langage de manipulation de données sous-jacent à la BD, dans notre cas il s'agit de DML pour ORACLE 7.0.
8. Création d'un *shell* pour la nouvelle application en se basant sur le langage de définition des *shells* [Babin93].

7.6.1 Nouvelle version de IBMS

Présentement, il existe deux versions non-compatibles pour IBMS, une exploitable sur Windows de Microsoft et l'autre sur Unix AIX de IBM. Pour remédier à ce problème, on devrait développer une nouvelle version de IBMS en se basant sur les spécifications suivantes :

- Définir une structure commune pour les fichiers de traitement de IBMS. La structure de fichiers utilisée dans la version Windows est relativement complète. Elle pourrait cependant être étendue en définissant des mnémoniques, ce qui faciliterait son utilisation et créerait ainsi un langage opérationnel d'intégration pour la base de métadonnées. Ce langage servirait de passerelle entre différentes versions de IBMS, par exemple la version sur RS/6000 et la version PC déjà existantes.
- Ajouter un traducteur supportant certaines méthodes de modélisation. Il créerait un modèle fonctionnel TSER en correspondance avec un modèle quelconque. Présentement, IBMS supporte les méthodes de modélisation DFD et IDEF, mais est incapable de les traduire.

- Créer un module d'insertion qui consolide le nouveau modèle sémantique avec le schéma fonctionnel global d'une MDB. Par la suite, l'analyste peut définir les spécifications nécessaires pour intégrer, au niveau fonctionnel, la nouvelle application.
- Adapter la fonction "Consolidate OERs" disponible dans le logiciel de modélisation structurel. Pour l'instant, cette fonction génère des modèles structurels consolidés incohérents et non exploitables, c'est-à-dire qu'il est impossible de visualiser ces schémas avec l'outil de modélisation structurel.
- Conceptualiser la fonction "MDB_Populate", qui, en se basant sur les spécifications contenues dans les modèles fonctionnel et structurel en traitement, génère un fichier DML servant à la création des instanciations d'une MDB. Cette fonction pourrait supporter plusieurs langages de manipulation de données, ce qui permettrait d'exporter le schéma consolidé d'une MDB sur différents types de bases de données, telles que les BD objets, relationnelles, hiérarchiques et réseaux. On devrait modifier la fonction pour qu'elle soit conforme à l'architecture actuelle du GIRD.
- Insérer une fonctionnalité dans le module "SER $\Rightarrow$ OER Mapping" pour vérifier les spécifications minimales (voir section 7.5). Cela assure une représentation structurelle cohérente de l'application en traitement.
- Utiliser un langage orienté-objets tel que JAVA pour le développement de la nouvelle version de IBMS, ce qui rendrait cet outil CASE portable sur plusieurs plates-formes.

Les spécifications mentionnées précédemment ne sont pas exhaustives, mais représentent des lignes directrices facilitant le développement d'une nouvelle version de IBMS, qui serait en mesure de réaliser le processus d'intégration d'une nouvelle application dans le schéma consolidé d'une base de métadonnées. Ainsi, ce processus serait grandement simplifié grâce à l'automatisation de fonctionnalités complexes.

7.6.2 Synthèse

Malgré l'automatisation de plusieurs activités du processus d'intégration, ce travail nécessitera toujours l'intervention de spécialistes en ce domaine [Chawathe et al.94, Gracia-Molina et al.95]. Les spécifications fonctionnelles et informationnelles établies dans ce chapitre faciliteront le développement d'une nouvelle version de IBMS qui générera toujours des schémas consolidés cohérents. Elles sont cependant uniquement valides dans le contexte d'une cellule d'intégration locale. L'intégration de plusieurs LIC serait réalisable en se basant sur l'approche autonomie-coopération des bases de métadonnées [Maamar95]. Ainsi, d'un côté, on intègre les MDB locales et les applications, et de l'autre côté, on intègre les applications locales. La même correspondance est faite entre la MDB globale et les MDB résultantes de l'intégration des applications locales.

Chapitre 8

Conclusion

Depuis le début des années 1990, nous avons assisté à une explosion spectaculaire de la quantité d'information disponible sur médium électronique. L'étude de la genèse des technologies de l'information démontre une évolution constante de celles-ci pour s'adapter aux besoins toujours grandissants des organisations. Les systèmes d'information de gestion sont aujourd'hui une ressource indispensable pour le traitement efficient des données de toute entreprise. Chacune des ressources informationnelles possède une panoplie de caractéristiques, souvent très différentes de leurs consoeurs, causant ainsi des problèmes de communication inter-applications. Dans la plupart des entreprises, on gère ce problème en adaptant les applications pour faciliter l'échange d'information disparate, ce qui génère des difficultés au niveau de la gestion et l'opérabilité des systèmes informatiques. De plus, l'installation d'un nouveau système dans cette architecture peut entraîner une suite de mises à jour majeures pour ajuster les systèmes déjà en place. Cette approche est inappropriée pour répondre adéquatement au besoin d'efficience du traitement de l'information, ce qui nous amène à formuler la question suivante : Comment créer un environnement d'intégration évolutif pour la gestion de l'architecture informationnelle d'une organisation ?

Pour augmenter la productivité, la plupart des opérations demandent un environnement d'information intégré, ce qui facilite le développement d'applications accédant aux multiples sources de données de manière efficiente et efficace. Notre but est de fournir un environnement d'intégration ouvert qui supporte l'interopérabilité des ressources informationnelles des années 1990.

Deux forces opposées ont suscité la recherche dans le domaine des bases de données distribuées hétérogènes. La distribution des systèmes centraux (*mainframes*), ainsi que la prolifération des stations de travail et des serveurs, ont provoqué la décentralisation de l'information. D'autre part, le traitement de cette information demande une évolution constante pour la bonne gestion des ressources informationnelles dispersées physiquement. L'échange de ces données engendre des problèmes au niveau de la morphologie et de la fonctionnalité, d'où l'intérêt d'implanter une forme de médiation inter-bases de données. Pour ce faire, on doit capturer les particularités de chacune des bases de données, ce qui peut être réalisé par la modélisation des systèmes

qui composent l'architecture informationnelle de l'organisation.

La modélisation d'information en milieu de production est une activité de corporation produisant des modèles de ressources informationnelles, des flux d'informations et des opérations qui s'effectuent dans une entreprise. Les schémas, qui en résultent, décrivent tous les aspects d'un environnement d'affaires, incluant : (1) les bases de données, (2) les applications, (3) les systèmes experts et les bases de connaissances, (4) les flux de données et (5) l'organisation elle-même, fournissant ainsi une documentation *online* de cet environnement.

Une entreprise possède plusieurs types de modèles, chacun d'eux décrivant une partie de l'organisation ; par exemple, l'information présente dans une base de données relationnelle est modélisée en rapport avec le schéma de cette dernière, tandis que l'information contenue dans une base de connaissances orientée-objets est modélisée en relation avec l'ontologie de ses objets. Il est à déplorer que les schémas sont souvent mutuellement incompatibles au niveau de la syntaxe et de la sémantique. De plus, lors de la modélisation de portions du monde réel, des simplifications et inexactitudes sont nécessairement introduites dans les schémas informationnels, ce qui engendre des incompatibilités fonctionnelles. Toutefois, les modèles individuels doivent être intégrés pour former un schéma global. Pour ce faire, ils doivent être inter-reliés entre eux et leurs incompatibilités doivent être résolues et ce, pour les raisons suivantes [Sheth et Larson90] :

- Une vision cohérente de l'entreprise est souvent indispensable lors de la prise de décision et pour la réalisation d'opérations d'affaires efficaces. Les modèles intégrés fournissent des accès à un environnement global permettant ainsi aux concepteurs d'évaluer les flux d'informations qui entrent et sortent d'une application particulière.
- L'interopérabilité des applications doit être effective, et ce, à la grandeur de l'entreprise. L'accroissement prédominant des applications d'affaires stratégiques requiert des liens d'intégration intercorporatifs ou intracorporatifs.
- Les développeurs ont besoin de valider l'intégrité du schéma global lors de l'insertion de nouveaux modèles ou de mises à jour.
- Les concepteurs doivent détecter et résoudre les inconsistances, non seulement au niveau des modèles, mais aussi au niveau des opérations d'affaires sous-jacentes qui sont modélisées.

D'autres avenues ont déjà été explorées, telles que : DOMS [Buchmann90, Hornick et al.93, Manola et al.92], Infomaster [Duschka et Genesereth97], Information Manifold [Levy et al.96], Occam [Kwok et Weld96], PEGASUS [Rafi et al.91, Shan93, Shan94], SIMS [Ambite et al.95, Arens et al.93].

La métamodélisation par le biais de la méthode TSER [Hsu et al.93], permet de capturer les caractéristiques entourant chacune des ressources qui composent l'infrastructure informationnelle d'une entreprise. En consolidant ces métadonnées dans une base conçue à cet effet, il est possible d'établir une description représentative et fidèle de la réalité de cette architecture informationnelle.

L'avenue que nous considérons utilise des résultats connus dans le domaine de l'intégration et de l'adaptabilité des systèmes d'information [Hsu et al.91, Hsu96]. Cette approche est caractérisée par (1) une modélisation des différentes applications et leur intégration, (2) une gestion des modèles intégrés et (3) une architecture concurrente qui utilise les modèles consolidés pour réaliser l'intégration des opérations. La base de métadonnées (*MetaDataBase*; MDB) contient les métadonnées nécessaires à la définition d'un schéma global représentant l'architecture informationnelle d'une entreprise. Chacune de ces métadonnées est insérée dans le dictionnaire global des ressources informationnelles (*Global Resources Information Dictionary*; GIRD ; [Bouziane91]) qui est un schéma complexe de gestion décrivant toutes les informations contenues dans l'entreprise.

Le principal but de ce travail de recherche est de définir un cadre pour le processus d'intégration d'une nouvelle application dans une MDB. Pour ce faire, nous avons étudié la structure de la base de métadonnées (voir chapitre 4) , le système de requêtes globales (voir chapitre 5) et l'environnement de programmation orienté-règles (voir chapitre 6). Dans le chapitre 7, nous définissons donc quelles sont les métaentités et les métarelations qui doivent être instanciées lors de l'insertion d'une nouvelle application (voir section 7.5) ; ensuite, nous établissons les contraintes de structuration (voir section 7.3) et les modules logiciels nécessaires à la réalisation du processus d'intégration, c'est-à-dire, les modifications fonctionnelles (voir section 7.6). Notons qu'elles sont seulement valides pour un environnement d'intégration local (voir section 7.1). Les résultats obtenus peuvent s'avérer avantageux pour une organisation puisque l'évolution des technologies de l'information ne cesse de progresser, d'où l'intérêt d'adapter le schéma global de l'architecture informationnelle, tout en conservant la cohérence de celui-ci. L'ajout d'une nouvelle application, dans ce cadre bien défini, devient simplement une série de transactions et d'insertions de métadonnées dans une base de métadonnées.

Plusieurs projets de recherche émergent de l'intégration d'applications dans le contexte d'une MDB, soient les thèmes suivants :

1. Établissement des spécifications pour l'insertion de ressources informationnelles dans une architecture d'intégration globale (voir figure 7.1), c'est-à-dire dans le contexte d'un système à grande échelle. On devrait prendre en considération les travaux réalisés par [Maamar95, Kane95].
2. Établissement des spécifications lors de la modification ou de la suppression des caractéristiques d'une ressource informationnelle, tout en maintenant la cohérence du contenu de la base de métadonnées.

3. Développement d'un outil d'intégration venant se greffer aux fonctionnalités du système de gestion de la base de métadonnées. Pour cela, nous conseillons de prendre en considération les travaux de recherche déjà réalisés [Hsu96] et les deux projets cités précédemment.

Avec l'ère de l'Internet et toutes ses implications, l'information est la ressource clé que doit maîtriser toute organisation désirant survivre et évoluer dans un monde économique fortement concurrentiel. La bonne gestion du savoir d'une entreprise passe assurément par une administration efficiente du processus d'intégration de ses ressources informationnelles. Il est exclu de prétendre que nous détenons toutes les solutions par rapport à ce domaine ; cependant, nous croyons fermement que cette étude apporte de nouveaux éléments positifs facilitant la résolution des problèmes d'intégration en milieu d'affaires.

Bibliographie

- [Alsène et Auclair94] Alsène, E. et S. Auclair, *Le degré d'intégration informatique de l'entreprise : proposition d'un mode de mesure*, Rapport technique, EPM/RT-94/10. Département de génie industriel, École polytechnique de Montréal, mars 1994.
- [Alsène et Liman96] Alsène, E. et S. Liman, *Mesure du degré d'intégration informatique d'une entreprise manufacturière*, Rapport technique, EPM/RT-96/13. Département de mathématiques et de génie industriel, École polytechnique de Montréal, juin 1996.
- [Alsène95] Alsène, E., "L'intégratiquette d'aujourd'hui et de demain : du soutien à la prise en charge des activités intellectuelles", *Technologies de l'information et société*. 7(3), pp. 301-319, octobre 1995.
- [Ambite et al.95] Ambite, J.-L., Y. Arens, N. Ashish, C. Y. Chee, C.-N. Hsu, C. A. Knoblock, W.-M. Shen et S. Tejada, *The SIMS manual: Version 1.0*, Rapport technique, ISI/TM-95-428, University of Southern California, Information Sciences Institute, 1995.
- [Amsaleg et al.96a] Amsaleg, L., M. Franklin et A. Tomasic, *Query Scrambling for Bursty Data Arrival*, Rapport technique, octobre 1996a.
- [Amsaleg et al.96b] Amsaleg, L., M. Franklin, A. Tomasic et U. Tolga, "Scrambling Query Plans to Cope with Unexpected Delays", in *International Conference on Parallel and Distribution Information Systems*, Miami Beach, Florida, PDIS. 1996b.
- [Andleigh et al.92] Andleigh, P., R. Michael et M. Gretzinger, *Distributed Object-Oriented Data-Systems Design*, Prentice Hall Inc., Englewood Cliffs, New Jersey 07632, 1992.
- [ANSI89] ANSI, A. N. S. I. I., *Information resources dictionary systems (IRDS)*. Rapport technique, Standard X3.138-1988, ANSI, New York, 1989.
- [Apple92] Apple, *Macintosh Human Interface Guidelines*, Addison Wesley, Reading, MA, 1992.
- [Arens et al.93] Arens, Y., C. Y. Chee, C.-N. Hsu et C. A. Knoblock. "Retrieving and integrating data from multiple information sources", *International Journal of Intelligent and Cooperative Information Systems*, 2(2), pp. 127-158, juin 1993.

- [Babin et al.94] Babin, G., C. Hsu et Z. Maamar, *A Distributed Metadatabase Architecture for an Enterprise Scalable, Adaptive Integration Environment*, Rapport technique, 37-94-424, Department of Decision Sciences and Engineering Systems, Rensselaer Polytechnic Institute, Troy, New York, USA, décembre 1994.
- [Babin et al.97] Babin, G., Z. Maamar et B. Chaib-draa, "Metadatabase Meets Distributed AI", in *International Workshop on Cooperative Information Agents '97 (CIA '97)*, Kandzia, P. et M. Klusch, éditeurs, numéro 1202, Lecture Notes in Artificial Intelligence, pp. 138-147, Kiel, Allemagne, Springer Verlag, février 1997.
- [Babin et Hsu94] Babin, G. et C. Hsu, "A Model-Based Architecture for Adaptive and Scalable Multiple Systems", *soumis à ACM Transactions on Database Systems*, septembre 1994.
- [Babin et Hsu96] Babin, G. et C. Hsu, "Decomposition of Knowledge for Concurrent Processing", *IEEE Transactions on Knowledge and Data Engineering*, 8(5), pp. 758-772, octobre 1996.
- [Babin93] Babin, G., *Adaptiveness in Information System Integration*, Thèse de doctorat, Department of Decision Sciences and Engineering Systems, Rensselaer Polytechnic Institute, Troy, New York, USA, août 1993.
- [Bergamaschi97] Bergamaschi, S., "Extraction of Informations from Highly Heterogeneous Source of Textual Data", in *Cooperative Information Agents: Proceedings of the First International Workshop, CIA '97*, pp. 42-63, Kiel, Allemagne, Peter Kandzia and Matthias Klusch Eds. (Springer), février 1997.
- [Bonnet et Tomasic97] Bonnet, P. et A. Tomasic, *Partial Answers for Unavailable Data Sources*, Rapport technique, RR-3127, INRIA, mars 1997.
- [Bouziane91] Bouziane, M., *Metadata Modeling and Management*, Thèse de doctorat, Department of Decision Sciences and Engineering Systems, Rensselaer Polytechnic Institute, Troy, New York, USA, juin 1991.
- [Buchmann90] Buchmann, A., "Modeling Heterogeneous Systems as an Active Object Space", in *4th International Workshop on Persistent Object Systems*, pp. 279-290, San Monteo, CA, Morgan Kaufmann Publishers, septembre 1990.
- [Chawathe et al.94] Chawathe, S., H. Gracia-Molina, J. Hammer, K. Ireland, Y. Papakonstantinou, J. Ullman et J. Widom, "The TSIMMIS Projet: Integration of Heterogeneous Information Sources", in *Proceedings of IPSJ Conference*, pp. 7-18, Tokyo, Japan, octobre 1994.
- [Chen76] Chen, P.-S., "The Entity-Relationship Model: Toward a Unified View of Data", *ACM Transactions on Database Systems*, 1, pp. 9-36, 1976.
- [Cheung et Hsu96] Cheung, W. et C. Hsu, "The Model-Assisted Global Query System for Multiple Databases in Distributed Enterprises", *soumis à ACM Transactions on Information Systems*, 14(4), pp. 421-470, octobre 1996.

- [Cheung91] Cheung, W., *The Model-Assisted Global Query System*, Thèse de doctorat, Department of Decision Sciences and Engineering Systems, Rensselaer Polytechnic Institute, Troy, New York, USA, novembre 1991.
- [Collet et al.92] Collet, C., N. Michael et W.-M. Shen, "Resource Integration Using a Large Knowledge Base in Carnot", *Computer*, 24(12), pp. 55-62, décembre 1992.
- [Date90] Date, C., *An Introduction to Database Systems*, vol. 1, The Systems Programming Series, Addison-Wesley Publishing Company, fifth edition, 1990.
- [Dilts et Wu91] Dilts, D. et W. Wu, "Using Knowledge-Based Technology to Integrate CIM Databases", *IEEE Transactions on Knowledge and Data Engineering*, 3(2), pp. 237-245, juin 1991.
- [Dilts et Wu92] Dilts, D. et W. Wu, "Integrating Diverse CIM Databases: The Role of Natural Language Interface", *IEEE Transactions on Systems, Man, and Cybernetics*, 22(6), pp. 1331-1347, novembre/décembre 1992.
- [Doug et Guha90] Doug, L. et L. V. Guha, *Building Large Knowledge-Based Systems: Representation and Inference in the Cyc Projet*, Addison-Wesley Publishing Company Inc., Reading, MA, 1990.
- [Duschka et Genesereth97] Duschka, O. M. et M. R. Genesereth, "Infomaster - An Information Integration Tool", in *Proceedings of the International Workshop on Intelligent Information Integration*, Freiburg, Allemagne, septembre 1997.
- [Florescu et al.96] Florescu, D., P. Raschid et P. Valduriez, "Answering Queries Using OQL View Expressions", in *Proc. Int. Workshop on Materialized Views*, Montréal, Canada, ACM SIGMOD, juin 1996.
- [Florescu96] Florescu, D., *Espace de Recherche pour l'Optimisation de Requêtes Objets*. Thèse de doctorat, Université de Paris, VI, octobre 1996.
- [Gamache95] Gamache, A., *Base de données : architectures, modèles, langages*, Presses de l'Université Laval, Canada, Québec, 1995.
- [Gracia-Molina et al.95] Gracia-Molina, H., J. Hammer, K. Ireland, Y. Papakonstantinou, J. Ullman et J. Widom, "Integrating and Accessing Heterogeneous Information Sources in TSIMMIS", in *Proceedings of the AAAI Symposium on Information Gathering*, pp. 61-64, Stanford, California, mars 1995.
- [Heimbigner et McLeod85] Heimbigner, D. et D. McLeod, "A Federated Architecture for Information Management", *ACM Transactions on Office Information Systems*, 3(3), septembre 1985.
- [Henessay et al.96] Hennessay, P., R. Scheid et J. R. Kirkley, "An integration framework for distributed systems", *Object Magazine, SIGS Publication*, pp. 36-42, mars 1996.

- [Hornick et al.93] Hornick, M., D. Georgakopoulos et P. Krychniak, "An Environment for the Specification and Management of External Transactions in DOMS", in *3rd International Workshop on Research Issue on Data Engineering: Interoperability in Multidatabase Systems*, Vienne, Autriche, avril 1993.
- [Hsu et al.88] Hsu, C., A. Perry, M. Bouziane et W. Cheung, "TSER: A Data Modeling System Using the Two-Stages Entity-Relationship Approach", in *Entity-Relationship Approach*, March, S., éditeur, pp. 497–514, Elsevier Science Publishers B.V. (North-Holland), 1988.
- [Hsu et al.91] Hsu, C., M. Bouziane, L. Rattner et L. Yee, "Information Resources Management in Heterogenous, Distributed Environments: A Metadatabase Approach", *IEEE Transactions on Software Engineering*, 17(6), pp. 604–625, juin 1991.
- [Hsu et al.92a] Hsu, C., G. Babin, L. Rattner, L. Yee, M. Bouziane et W. Cheung, "Metadatabase Modeling for Enterprise Information Integration", *Journal of Systems Integration*, 1, pp. 5–37, 1992a.
- [Hsu et al.92b] Hsu, C., A. Rubenstein, L. Yee, G. Babin et M. Lawson, "What is Rensselaer's Metadatabase system?", in *Rensselaer's Third International Conference on Computer Integrated Manufacturing*, pp. 424–433, Troy, NY, mai 1992b.
- [Hsu et al.93] Hsu, C., Y.-C. Tao, M. Bouziane et G. Babin, "Paradigm Translations in Integrating Manufacturing Information Using a Meta-Model: The TSER Approach", *Ingénierie des Systèmes d'informatique*, 1(3), pp. 325–352, 1993.
- [Hsu et al.94] Hsu, C., G. Babin, M. Bouziane, W. Cheung, L. Rattner, A. Rubenstein et L. Yee, "The metadatabase approach to integrating and managing manufacturing information systems", *J. Intel. Manuf.*, 5, pp. 333–345, 1994.
- [Hsu et Babin93] Hsu, C. et G. Babin, "A Rule-Oriented Concurrent Architecture to Effect Adaptiveness for Integrated Manufacturing Enterprises", in *International Conference on Industrial Engineering and Production Management*, pp. 868–877, Mons, Belgique, juin 1993.
- [Hsu et Rattner90] Hsu, C. et L. Rattner, "Information Modeling for Computerized Manufacturing", *IEEE Transactions on Systems, Man, and Cybernetics*, 20(4), pp. 758–776, juillet/août 1990.
- [Hsu et Rattner93] Hsu, C. et L. Rattner, "Metadatabase Solutions for Enterprise Information Integration Problems", *Database, A Quarterly Publication of Sigbit*, 24(1), pp. 23–35, hiver 1993.
- [Hsu91] Hsu, C., "The Metadatabase Project at Rensselaer", *Sigmod Record*, 20(4), pp. 83–90, décembre 1991.
- [Hsu96] Hsu, C., *Enterprise Integration and Modeling: the Metadatabase Approach*. Kluwer Academic Publishers, 1996.

- [Huhns et al.92] Huhns, M., N. Jacobs, T. Ksiezyk, W. Shen, M. Singh et P. Cannata, "Enterprise Information Modeling and Model Integration in Carnot", in *Charles J. Petrie Jr., Enterprise Integration Modeling: Proceedings of the First International Conference*, Cambridge MA, MIT Press, 1992.
- [IEEE87] IEEE, "Special Issue on Distributed Database Systems", *Proceedings of the IEEE*, 75(5), mai 1987.
- [Kane95] Kane, C.-O., *Modèle de coopération et de partage d'information entre bases de métadonnées concurrentes*, Mémoire de maîtrise, Département d'informatique, Université Laval, Québec, Canada, octobre 1995.
- [Kapitskaia et al.97] Kapitskaia, O., A. Tomasic et P. Valduriez, *Dealing with Discrepancies in Wrapper Functionality*, Rapport technique, RR-3138, INRIA, mars 1997.
- [Kroenke et Hatch94] Kroenke, D. et R. Hatch, *Management Information Systems*. Mitchell, McGraw-Hill, New York, USA, third edition, 1994.
- [Kwok et Weld96] Kwok, C. T. et D. S. Weld, "Planning to gather information", in *Proceedings of the AAAI Thirteenth National Conference on Artificial Intelligence*, 1996.
- [Lawrence et al.96] Lawrence, H., C. Fernandes, T. Neild et W.-S. Li, "Modeling Data Correspondence in Multidatabase Systems.", in *Proceedings of the International Conference on Intelligent Information Management Systems (IIMS96)*, pp. 19-23. Washington, DC, USA, juin 1996.
- [Levy et al.96] Levy, A. Y., A. Rajaraman et J. J. Ordille, "Querying heterogeneous information sources using source descriptions", in *Proceedings of the 22nd International Conference on Very Large Databases*, pp. 215-262, Bombay, Inde, 1996.
- [Litwin et al.90] Litwin, W., L. Mark et N. Roussopoulos, "Interoperability of Multiple Autonomous Databases". *ACM Computing Surveys*, 22(3), pp. 267-296, septembre 1990.
- [Litwin88] Litwin, W., "From Database Systems to Multidatabase Systems: Why and How", in *British National Conf. on Databases*, Angleterre, Cardiff, juillet 1988.
- [Maamar95] Maamar, Z., *Dynamique de l'intégration des métabases de données*. Mémoire de maîtrise, Département d'informatique, Université Laval, Québec. Canada, avril 1995.
- [Manola et al.92] Manola, F., S. Heiler, D. Georgakopoulos, M. Hornick et M. Brodie, "Distributed Object Management", *Intelligent and Cooperative Information Systems*, 1(1), mars 1992.
- [Manola et Heiler92] Manola, F. et S. Heiler, "An Approach to interoperable Object Models", in *International Workshop on Distributed Object Management*, Edmonton, Canada, août 1992.

- [Mowbray et Zahavi95] Mowbray, T. et R. Zahavi, *The Essential CORBA: Systems Integration Using Distributed Objects*, John Wiley & Sons, New York, NY, USA, 1995.
- [Naacke et al.97] Naacke, H., G. Gardarin et A. Tomasic, *Leveraging Mediator Cost Models with Heterogeneous Data Sources*, Rapport technique, RR-3143, INRIA, mars 1997.
- [Nielsen93] Nielsen, J., *Usability Engineering*, Academic Press, San Diego, CA, 1993.
- [Nilan92] Nilan, M., "Using virtual reality for large information resource management problems", *Cognitive Space*, 42, pp. 115–135, mars-avril 1992.
- [Nolan79] Nolan, R. L., "Managing the Crisis in Data Processing", *Havard Business Review*, mars-avril 1979.
- [OMG94] OMG, *IDL C++ Language Mapping Specifications*, Rapport technique, OMG RFP Submission, août 1994.
- [OMTF92] OMTF, *Object Management Task Force: The OMG Object Model*, Rapport technique, OMG, mars 1992.
- [Ozkarahan86] Ozkarahan, E., *Database Machines and Database Management*, Prentice-Hall, Inc., Englewood Cliffs, N.J. 07632, 1986.
- [Petters84] Petters, E., *Conception et gestion des banques de données*, Les éditions d'organisation, 5, rue Rousselet, 75007, Paris, 1984.
- [Rafi et al.91] Rafi, A., P. Smedt, W. Du, W. Kent, M. Ketabchi, W. Litwin, A. Rafi et M.-C. Shan, "The Pegasus Heterogeneous Multidatabase System", *Computer*, 24(12), pp. 19–27, décembre 1991.
- [Raschid et al.96] Raschid, L., A. Tomasic et P. Valduriez, "Scaling Heterogeneous Databases and the Design of Disco", in *Proceedings of the International Conference on Distributed Computing Systems*, pp. 449–457, Hong-Kong, 1996.
- [Raschid et Valduriez95] Raschid, L. et P. Valduriez, *Scaling Heterogeneous Distributed Databases and the Design of DISCO*, Rapport technique, RR-2704, INRIA, 1995.
- [Rattner90] Rattner, L., *Information Requirements for Integrated Manufacturing Planning and Control: A Theoretical Model*, Thèse de doctorat, Department of Decision Sciences and Engineering Systems, Rensselaer Polytechnic Institute, Troy, New York, USA, 1990.
- [Reddy et Subhash93] Reddy, P. et B. Subhash, "Deadlock Prevention in a Distributed Database System", *Sigmod Record*, 22(3), pp. 40–46, septembre 1993.

- [Robertson et al.93] Robertson, G., S. CARD et J. Mackinlay, "Information visualization using /D interactive animation", *ACM Commun*, 36(4), pp. 56-71, avril 1993.
- [Sanders83] Sanders, H. D., *Computers Today*, McGraw-Hill Inc., New York, USA, 1983.
- [Shan93] Shan, M.-C., "Pegasus Architecture and Design Principles", in *ACM SIGMOD 1993*, pp. 422-425, 1993.
- [Shan94] Shan, M.-C., *Pegasus: A Heterogeneous Information and Operation Management System*, Working paper, Hewlett-Packard Laboratories, 1501 Page Mill Road, Palo Alto, CA, 1994.
- [Sheth et Larson90] Sheth, A. et J. Larson, "Federated Database Systems For Managing Distributed, Heterogenous, and Autonomous Databases", *ACM Computing Surveys*, 22(3), pp. 183-236, septembre 1990.
- [Shneiderman92] Shneiderman, B., *Designing the User Interface: Strategies for Effective Human-Computer Interaction*, Addison-Wesley Publishing Company, seconde édition edition, 1992.
- [Soley93] Soley, R., éditeur, *Object Management Architecture Guide*, Object Management Group, Framingham, MA, USA, 1993.
- [Spaccapietra et al.92] Spaccapietra, Parent et Dupont, "Model independent assertions for integration of heterogenous schemas.", *VLDB Journal*, pp. 44-98, 81-126 1992.
- [Tanenbaum96] Tanenbaum, A. S., *Computer Networks*, Prentice Hall, International Inc., Englewood Cliffs, New Jersey, third edition, 1996.
- [Thacker89] Thacker, R. M., "A New CIM Model: A Blueprint for The Computer-Integrated Manufacturing Enterprise", in *Dearborn (MI): Society of Manufacturing Engineers*, 1989.
- [Tomlinson et al.92] Tomlinson, C., G. Lavender, G. Meredith, D. Woelk et P. Cannata, "The Carnot Extensible Services Switch (ESS) - Support for Service Execution", in *Charles J. Petrie Jr., Enterprise Integration Modeling: Proceedings of the First International Conference*, Cambridge MA, MIT Press, 1992.
- [Watson96] Watson, A., "The OMG after CORBA 2", *Object Magazine, SIGS Publication*, pp. 58-60, mars 1996.
- [Woelk et al.92] Woelk, D., W. Shen, M. Huhns et P. Cannata, "Model Driven Enterprise Information Management in Carnot", in *Charles J. Petrie Jr., Enterprise Integration Modeling: Proceedings of the First International Conference*, Cambridge MA, MIT Press, 1992.

- [Woelk et al.93] Woelk, D., P. Cannata, M. Huhns, W. Shen et C. Tomlinson, "Using Carnot for Enterprise Information Integration", in *Second International Conference on Parallel and Distributed Information Systems*, pp. 133–136, janvier 1993.

ANNEXES

Annexe A

Définition et visualisation du GIRD Méta-Entités

CLÉ: NOM DU CONCEPTS_MDB (Clé primaire, attribut[1],...,attribut[n])

1. **ACTION** (Actid, acttype, factid, declvalue, *mm_resid*)
2. **APPLICATION** (Applname, descript, applcode, userid, addedby, dateadded, modifby, lastmod, nummods, *mm_resid*)
3. **CONDITION** (Condid, leftfact, operator, rightfact, *mm_resid*)
4. **CONTEXTE** (Cname, Applname, descript, xcoord, ycoord, addedby, dateadded, modifby, lastmod, nummods, *mm_resid*)
5. **ENTITÉ/RELATION** (ERname, ertype, descript, akey, addedby, dateadded, modifby, lastmod, nummods, *mm_resid*)
6. **FAIT** (Factid, factname, facttype, factvalue, *mm_resid*)
7. **INTÉGRITÉ** (Intname, inttype, descript, master, slave, addedby, dateadded, modifby, lastmod, nummods, *mm_resid*)
8. **ITEM** (Itemcode, itemname, itemtype, descript, iformat, ilength, domain, defvalue, addedby, dateadded, modifby, lastmod, nummods, Applname, *mm_resid*)
9. **RÈGLE** (Rname, rtype, descript, condid, addedby, dateadded, modifby, lastmod, nummods, *mm_resid*)
10. **RESSOURCES LOGICIELLES** (Resid, resname, extension, restype, descript, sizevalue, sizeunit, coding, developedby, addedby, dateadded, modifby, lastmod, nummods, *mm_resid*)
11. **RESSOURCES MATÉRIELLES** (Serialno, hname, htype, descript, location, nodename, nodeaddr, manufacturer, purchby, datepurch, userid, addedby, dateadded, modifby, lastmod, nummods, *mm_resid*)

12. **SUJET** (Sname, descript, xcoord, ycoord Applname, fileid, addedby, dateadded, modifby, lastmod, nummods, *mm_resid*)
13. **USAGER** (Userid, lastname, firstname, position phone, office, address, email, skill, addedby, dateadded, modifby, lastmod, nummods, *mm_resid*)

Méta-Relations Plurales

1. **Action_de** (Actid, Rname, relorder)
2. **Appartenir_à** (Itemcode, ERname, relpos, inpkey, posinpk)
3. **Appeler** (Actid, Procid, Parid, parorder)
4. **Appliquer_à** (Sname, Rname, relorder, inherited)
5. **Contenir** (Cname, Rname, relorder)
6. **Décrire** (Itemcode, Sname, relpos, inherited)
7. **Définir** (Sname, SSname, class_type, inherit_type)
8. **Équivalent** (Itemcode, EqItemcode, convert_by, reverse_by, addedby, dateadded)
9. **Intégrer** (Applname, Cname, metadatabase, integration_type, direction, descript, addedby, dateadded)
10. **Module_de** (Resid, Subresid, relationship)
11. **Nommer** (ERname, Applname, localname)
12. **Relier** (Cname, Sname, direction)
13. **Résider_à** (Resid, Serialno, path, invokecom)
14. **Stocker_dans** (Itemcode, Resid, relpos)
15. **Traiter** (Factid, Functid, Parid, parorder)
16. **Transformer** (Sname, ERname, addedby, dateadded, modifby, lastmod, nummods)
17. **Usager_Appl** (Applname, Userid, upassword, accesscode, *user_level*, access_count, addedby, dateadded, modifby, lastmod, nummods)
18. **Utiliser** (Applname, Resid, dataorg)

Méta-Relations Obligatoires

1. **Composer** (APPLICATION, SUJET) apparenté par Applname
Rôle APPLICATION = possesseur
Rôle SUJET = possédé
2. **Express** (CONDITION, FAIT) apparenté par factid
Rôle CONDITION = possesseur
Rôle FAIT = possédé
3. **Inter_Composer** (APPLICATION, CONTEXTE) apparenté par Applname
Rôle APPLICATION = possesseur
Rôle CONTEXTE = possédé
4. **LtOperand** (FAIT, CONDITION) apparenté par factid = leftfact
Rôle FAIT = possesseur
Rôle CONDITION = possédé
5. **RtOperand** (FAIT, CONDITION) apparenté par factid = rightfact
Rôle FAIT = possesseur
Rôle CONDITION = possédé

Méta-Relations Fonctionnelles

1. **Administrer** (APPLICATION, USAGER) apparenté par Userid
Rôle APPLICATION = déterminant
Rôle USAGER = déterminé
2. **Cond_de** (RÈGLE, CONDITION) apparenté par condid
Rôle RÈGLE = déterminant
Rôle CONDITION = déterminé
3. **Convertir** (Équivalent, RÈGLE) apparenté par convert_by = rname et
reverse_by = slave
Rôle Équivalent = déterminant
Rôle RÈGLE = déterminé
4. **Exister** (INTÉGRITÉ, ENTITÉ/RELATION) apparenté par ername = master
et
ername = slave
Rôle INTÉGRITÉ = déterminant
Rôle ENTITÉ/RELATION = déterminé
5. **Item_dans** (ITEM, APPLICATION) apparenté par Applname
Rôle ITEM = déterminant
Rôle APPLICATION = déterminé

6. **Maintenir** (RESSOURCES_MATÉRIELLES, USAGER) apparenté par Userid
Rôle RESSOURCES_MATÉRIELLES = déterminant
Rôle USAGER = déterminé
7. **Méta_Modéliser** (INTÉGRITÉ, RESSOURCES_LOGICIELLES) apparenté par
metadatabase = resid
Rôle INTÉGRITÉ = déterminant
Rôle RESSOURCES_LOGICIELLES = déterminé
8. **Pour** (FAIT, ITEM) apparanté valueof = itemcode
Rôle FAIT = déterminant
Rôle ITEM = déterminé
9. **Rattacher** (ACTION, FAIT) apparenté par factid
Rôle ACTION = déterminant
Rôle FAIT = déterminé

Annexe B

Définition des MétaAttributs

CLÉ (pour le nom du métaattribut) :

attribut - (miniscule) champ non-clé dans les métarelations

Attribut - (majuscule/minicules) champ clé dans les métarelations

Attribut - (majuscule soulignée/minuscules) champ clé et non-clé dans les métarelations

Nom du Métaattribut	Description
accesscode	Un attribut de la métarelation plurale ApplUsager qui identifie les droits d'accès d'un usager sur les données du système ; ex : Read (R), Write (W), Execute (E), Delete (D)
access_count	Un attribut de la métarelation plurale ApplUsager indiquant le nombre de fois qu'un usager a accédé à une application
Actid	Un identifiant unique (clé primaire) pour la métaentité ACTION
acttype	Classe de conséquences des règles de production ; ex : prend la valeur 0 si le résultat de la règle est relié à un fait ou la valeur 1 s'il s'agit d'un appel de procédure
addedby	Nom/Initiales de l'analyste-programmeur ou de l'administrateur qui a saisi la métaentité ou la métarelation dans le GIRD ; permet de garder une trace des transactions de saisie
address	L'adresse personnelle de l'utilisateur ; attribut de la métaentité USAGER
akey	Clé primaire alternative pour la base de relation ENTITÉ/RELATION

Nom du Métaattribut	Description
applcode	Trois caractères représentant un code d'identification pour une application
Applname	Nom unique (clé primaire) pour l'application d'une entreprise
class.type	Champ pour définir les types d'objets pour le support du paradigme orienté-objets
Cname	Nom unique (clé primaire) pour la métaentité CONTEXTE
coding	Le type de représentation physique pour une ressource logicielle : ex : Pascal ou LISP pour les langages de programmation ; ASCII, VSAM, or ISAM pour les fichiers de données
<u>Condid</u>	Identifiant unique (clé primaire) pour la métaentité CONDITION ; aussi un attribut pour la métaentité RÈGLE
convert.by	Règle de conversion pour les données équivalentes de Itemcode et EqItemcode
dataorg	Indique comment la donnée est organisée dans une application ; il est un attribut de la métarelation plurale Utiliser
dateadded	Date de création pour l'instance d'une métaentité ou d'une métarelation dans le GIRD
datepurch	Date à laquelle la ressource matérielle a été achetée/acquise
declvalue	Valeur déclarative pour la métaentité FAIT
defvalue	Valeur par défaut pour la métaentité ITEM, s'il y a lieu
descript	Description de toutes les métaentités et les métarelations définies
developedby	Le nom de la firme ou de la personne qui a développé la ressource logicielle
direction	Indique comment le lien (flux de données) entre un CONTEXTE et un SUJET est dirigé graphiquement, (ex : 1 = vers le SUJET ; 2 = vers le CONTEXTE ; 3 = bidirectionnel ; null = aucun) ; il est un attribut de la métarelation plurale Relier
domain	La série de valeur qui peut être assignée à un attribut (métaentité ITEM)

Nom du Métaattribut	Description
email	Adresse électronique pour un usager
EqItemcode	Synonyme de Itemcode dans la métarelation plurale Equivalent
ERname	Nom unique (clé primaire) pour la métaentité ENITÉ/RELATION
ertype	Le type d'entité/relation ; prend la valeur "OE" pour une entité opérationnelle, ou "PR" pour une relation plurale
extension	L'extension du nom du fichier pour une ressource logicielle, s'il y en a une
<u>Factid</u>	Identifiant unique généré par le système (clé primaire) pour la métaentité FAIT ; il est aussi un attribut pour la métaentité ACTION
Factname	Attribut de fait qui est soit un Itemcode, soit une expression (Condid)
facttype	Attribut de fait qui indique comment la valeur d'un fait doit être assigné ; 0 si la valeur du fait doit être disponible dans la base de données locale, 1 si le résultat est l'évaluation d'une expression, 2 s'il est traité par un appel de procédure
factvalue	La valeur calculée ou référée, ou une constante qui lie un fait durant le processus d'inférence d'une règle
fileid	Attribut de la métaentité SUJET, synonyme pour Resid
firstname	Le prénom d'un usager dans la métaentité USAGER
Functid	Synonyme de Resid ; identifie la fonction qui doit être appelée pour lier un fait ; il est aussi un champ clé dans la métarelation plurale Traiter
hname	Numéro du modèle ou nom de la ressource matérielle
htype	Le type de matériel ; il est un attribut de la métaentité RESSOURCES-LOGICIELLE, ex : mainframe, imprimante à aiguilles, mini ou micro-ordinateur, disque dur, etc
iformat	La représentation du type de données ; il est un attribut pour la métaentité ITEM ex : Character (C), Integer (I), Real (R), BCD (B) EPCDIC (E), etc

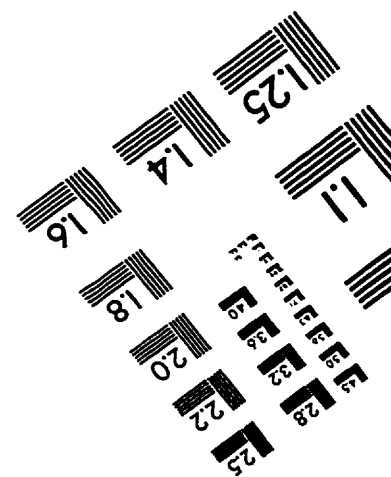
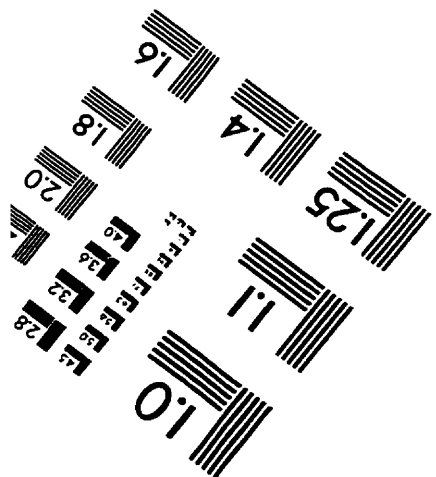
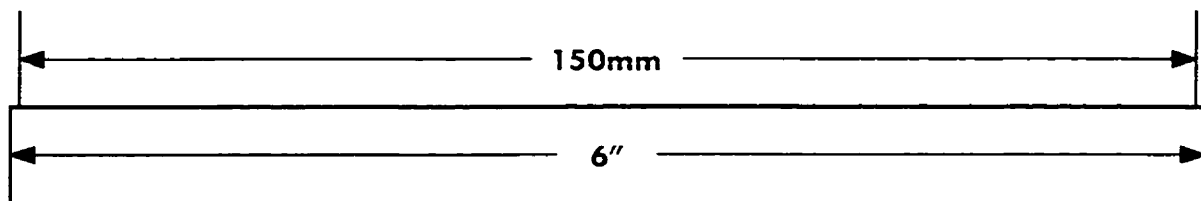
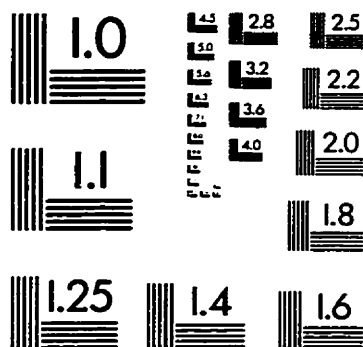
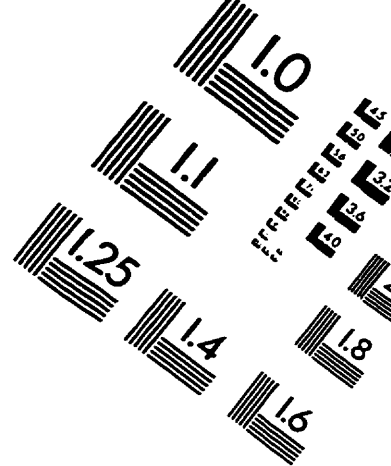
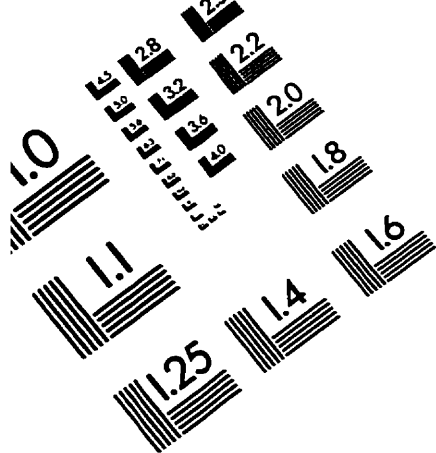
Nom du Métaattribut	Description
ilength	Représente la longueur d'un attribut ; il peut référer à la longueur en nombre de caractères ou en octets dépendant de l'implantation
inherited	Un drapeau qui indique si un attribut hérite du niveau supérieur du sujet
inherit_type	Déclare le type d'héritage dans la métarelation Définir, ex : full, partial, selective
inpkkey	Un drapeau (valeur booléenne) indiquant, si oui ou non un attribut fait partie de la clé primaire de ENTITÉ/RELATION ; il est un attribut de la métarelation plurale Appartenir à
Intname	Nom unique (clé primaire) pour une contrainte d'intégrité
inttype	Le type de la contrainte d'intégrité, "FR" il s'agit d'une relation fonctionnelle, ou "MR" si c'est une relation obligatoire
invokecom	La commande à invoquer pour une ressource logicielle sur une ressource matérielle ; il est un attribut de la métarelation plurale Résider à
Itemcode	Un identifiant généré par le système (clé primaire) pour l'élément de donnée (métaentité ITEM)
Itemname	Le nom de l'attribut dans la métaentité ITEM
itemtype	Un attribut de la métaentité ITEM qui indique si l'attribut est persistant ou s'il est généré lors de l'exécution
lastmod	La date où les métaentités et les métarelations du GIRD ont été modifiées
lastname	Le nom de famille d'un usager dans la métaentité USAGER
leftfact	Synonyme de Factid et il représente l'opérante de gauche dans une expression
localname	Attribut de la métarelation plurale Nommer
location	Localisation physique pour la métaentité RESSOURCES MATÉRIELLES
manufacturer	Le manufacturier de la ressource matérielle

Nom du Métaattribut	Description
master	Un attribut de la métaentité INTÉGRITÉ qui, dans le cas d'une relation fonctionnelle, joue le rôle de déterminant et dans le cas d'une relation obligatoire, le rôle de détenteur
modifby	Identifiant (nom ou initiales) d'un individu qui a modifié pour la dernière fois une instance d'une métarelation donnée
mmresid	Identifiant pour une ressource logicielle correspondant à un élément multimédia ; synonyme avec Resid
nodeaddress	Adresse réseau pour une ressource matérielle
nodename	Le nom du noeud réseau pour une ressource matérielle
nummods	Le nombre de modifications d'une métaentité donnée ; cet attribut se trouve dans toutes les métaentités et dans la plupart des métarelations plures
office	Localisation d'un bureau ou l'adresse d'un usager contenu dans la métaentité USAGER
operator	L'opérateur logique dans l'antécédent d'une règle de production ; cela inclut une série d'opérateurs et de valeurs arithmétiques
Parid	Synonyme de Factid qui représente un paramètre d'une fonction dans la métarelation plurelle Appeler
parorder	La position relative d'un paramètre dans une liste de paramètres dans une fonction ou une procédure
path	Chemin d'accès pour une ressource logicielle qui réside dans une ressource matérielle
phone	Le numéro de téléphone du bureau d'un usager
posinpkey	La position relative d'un champ attribut dans la clé primaire ENTITÉ/RELATION
position	La position organisationnelle d'un usager, ex : président, DBA, opérateur informatique, etc
precision	Précision numérique pour un nombre réel défini dans ITEM
Procid	Synonyme de Resid ; il identifie la procédure qui doit être appelée pour une règle d'action

Nom du Métaattribut	Description
purchby	Identifiant d'un individu responsable des achats pour les ressources matérielles
relationship	La relation avec la ressource logicielle dans la métarelation plurielle Module de
relorder	Ordre relatif d'une règle avec son SUJET ou son CONTEXTE, ou d'une règle de condition
relpos	Position relative d'un attribut dans la métaentité ENTITÉ/RELATION
Resid	Un identifiant unique (clé primaire) pour la métaentité RESSOURCE-LOGICIELLE
resname	Nom d'une ressource logicielle
restype	Le type de la ressource logicielle, ex : programme, fichier de données, réseau, document
reverse_by	Règle de conversion pour des données équivalentes de EqItemcode vers Itemcode
rightfact	Synonyme de Factid et il représente l'opérante de droite dans une expression
<u>Rname</u>	Nom unique (clé primaire) pour la métaentité RÈGLE
rtype	Le type de règle, ex : Modeling (M), Operating (O), Production (P), etc
Serialno	L'identifiant unique (clé primaire) pour la métaentité RESSOURCES MATÉRIELLES
Sizeunit	L'unité de mesure pour décrire le stockage d'une ressource logicielle, ex : KBytes, blocks, cylinders, pages, etc
Sizevalue	Quantité d'unités nécessaires pour une ressource logicielle, exprimé en Sizeunit
skill	Un attribut d'une valeur entière qui assigne un niveau d'expérience ; plus ce nombre est élevé, plus l'utilisateur est expérimenté

Nom du Métaattribut	Description
slave	Un attribut de la métaentité INTÉGRITÉ qui, dans le cas d'une relation de type fonctionnelle, joue le rôle du déterminé et dans le cas d'une relation obligatoire, le rôle du détenteur
Sname	Nom unique (clé primaire) de la métaentité SUJET
SSname	Nom du super-sujet pour la métarelation Définir ; le type et le domaine est le même que Sname
Subresid	Un synonyme pour Resid ; il est le champ clé dans la métarelation plurale Module de
upassword	Le mot de passe pour une application dans la métarelation ApplUsager
<u>Userid</u>	Identifiant unique (clé primaire) pour la métaentité USAGER
xcoord	La coordonnée X de la représentation graphique d'un SUJET ou d'un CONTEXTE
ycoord	La coordonnée Y de la représentation graphique d'un SUJET ou d'un CONTEXTE

TEST TARGET (QA-3)



APPLIED IMAGE, Inc
1653 East Main Street
Rochester, NY 14609 USA
Phone: 716/482-0300
Fax: 716/288-5989

© 1993, Applied Image, Inc., All Rights Reserved